

6 The Transport Layer

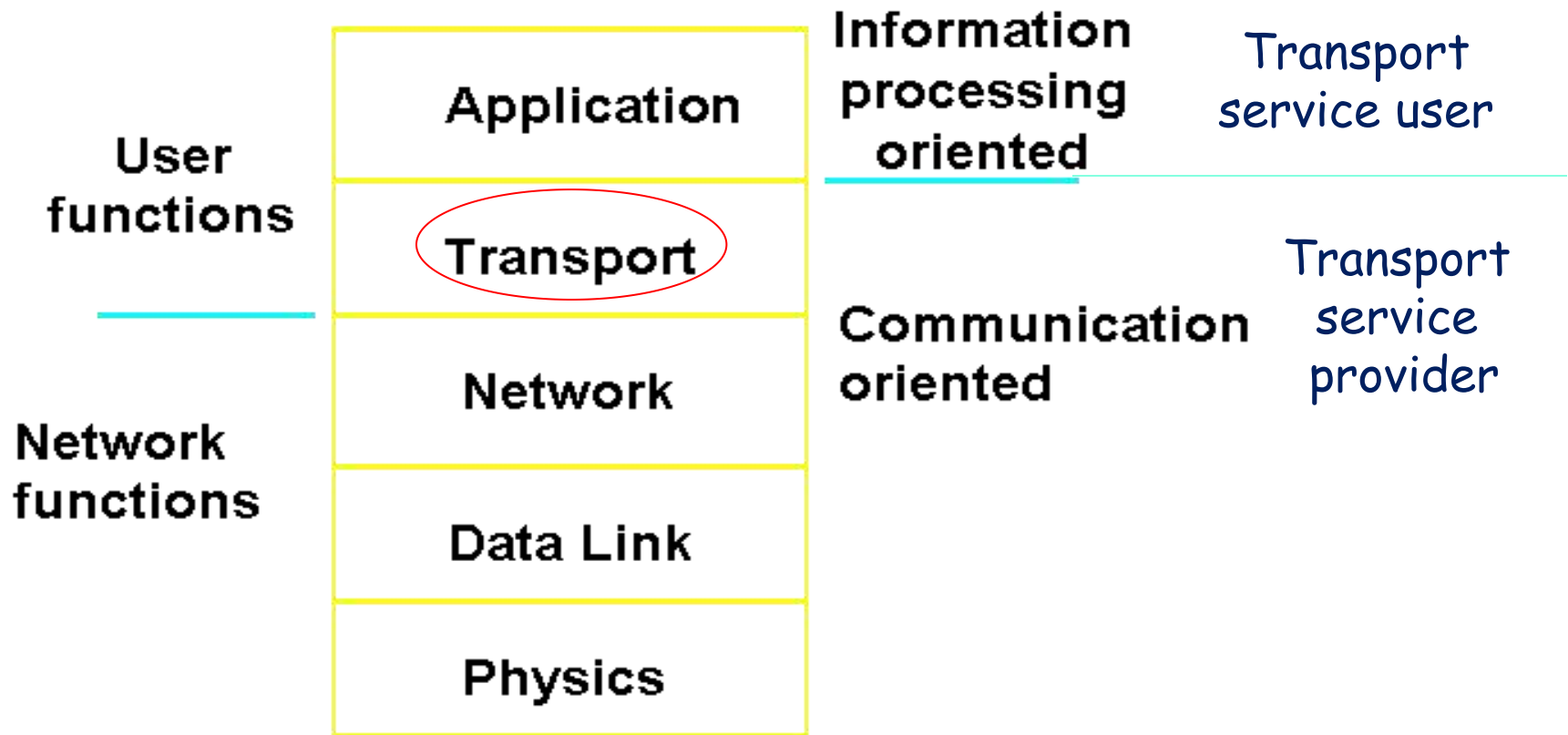
(7–8 hours)

Outline

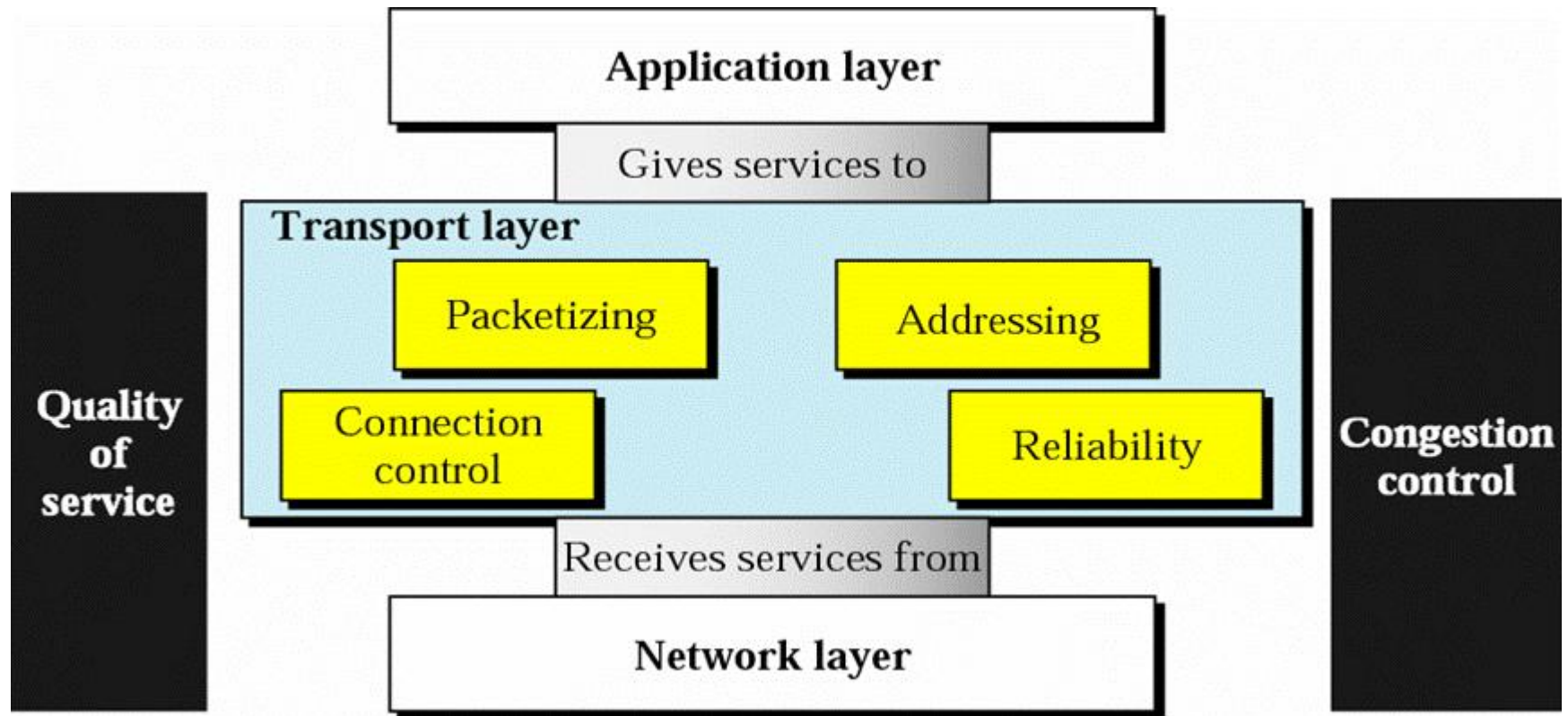
- 6.1 Function and Services to Transport Layer
- 6.2 Elements of Transport Protocols
 - ◆ 6.2.1 Addressing
 - ◆ 6.2.2 Connection Establishment
 - ◆ 6.2.3 Connection Release
 - ◆ 6.2.4 Flow Control and Buffering
- 6.3 Congestion Control
- 6.4 User Datagram Protocol (UDP)
- 6.5 Transmission Control Protocol (TCP)

(1) Position of Transport Layer

- Heart of the whole protocol hierarchy.



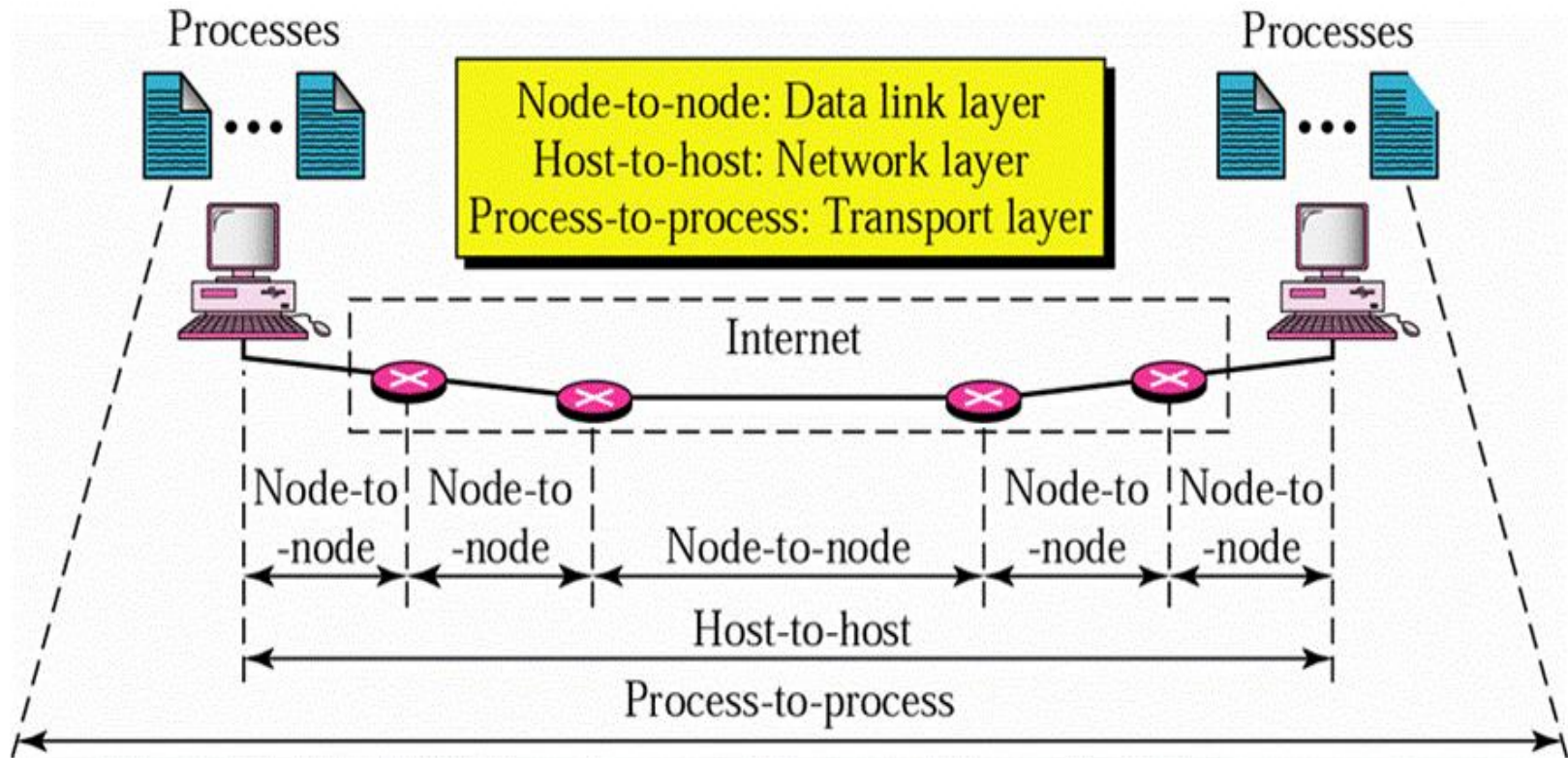
(2) Functions of Transport Layer



- Providing efficient, reliable, and cost-effective data transport from the source to the destination
- Independently of the physical network or networks currently in use.

[Forouzan]

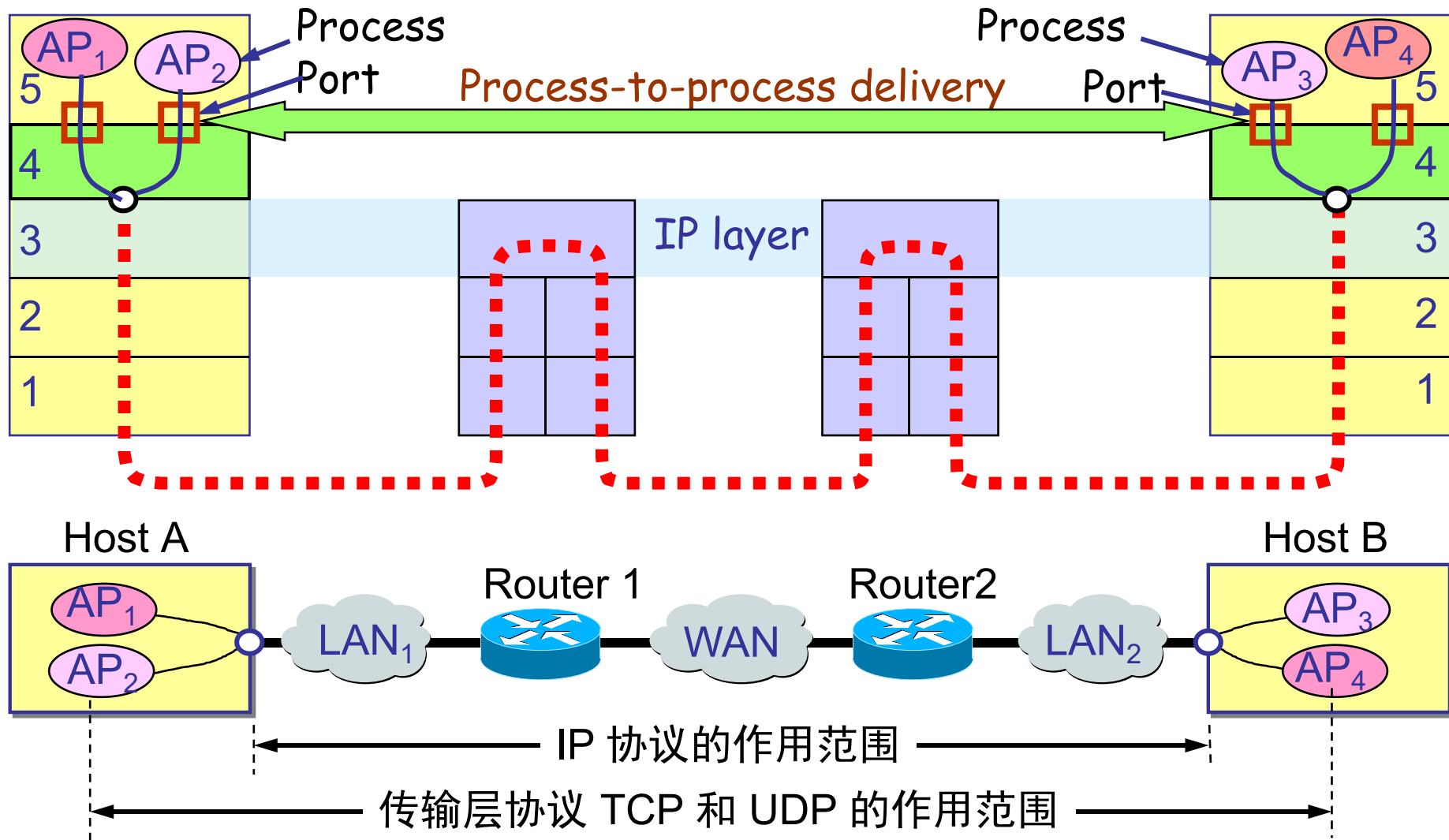
(3) Process-to-Process Delivery



进程-进程

[Forouzan]

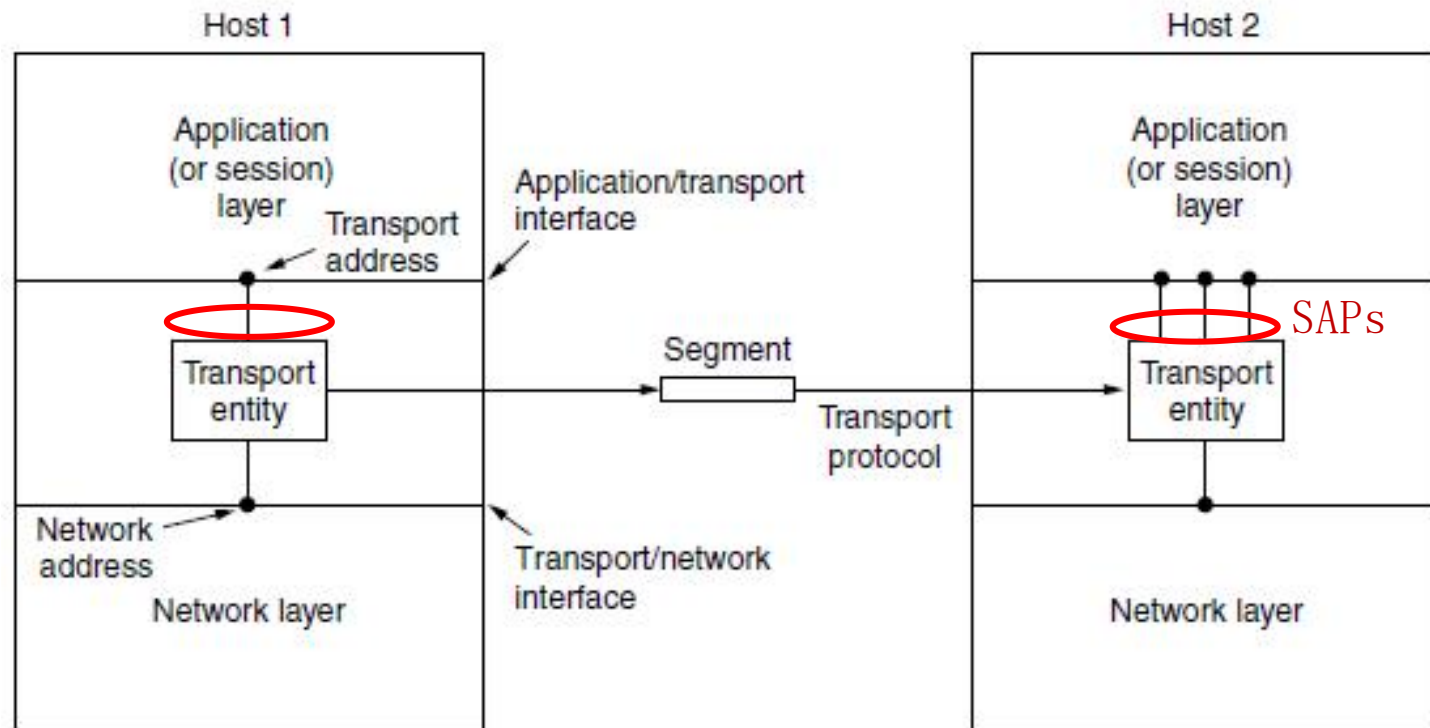
(4) Scope of Transport Layer



[谢]

(5) Services of Transport Layer

- Masking the diversity of communication subnets and providing a **common interface** to the upper layer
- Providing multiple **SAPs**(Service Access Points) **sharing** one network link
- Two type of services: **connectionless, connection-oriented**



(6) Client-Server Model

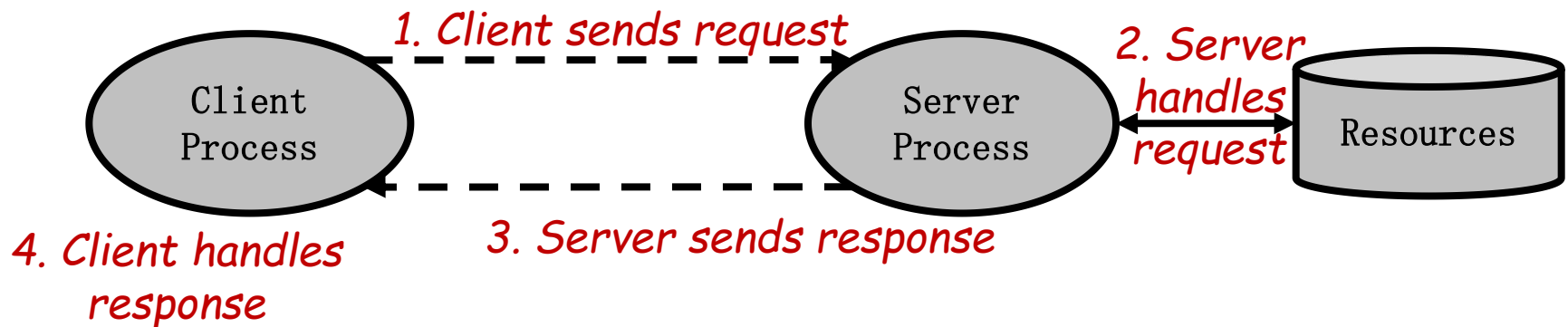
- Used by almost all network applications

- Server

- ◆ Manages some **resources** and provides **service** by manipulating resources for clients
- ◆ Begins execution first
- ◆ Waits **passively** at prearranged location

- Client

- ◆ Begins execution later
- ◆ **Actively initiates** contact with a server

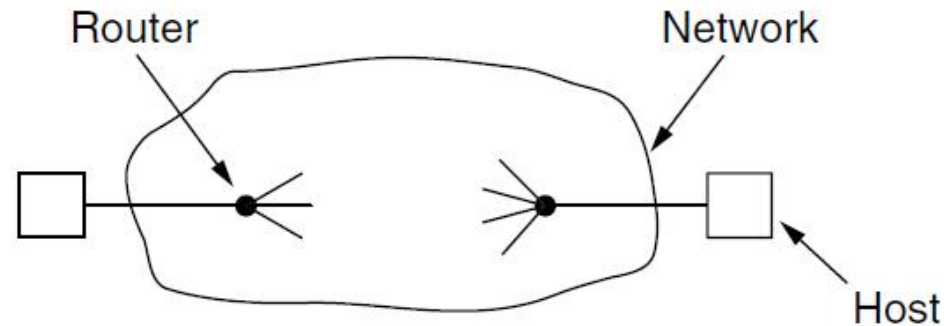
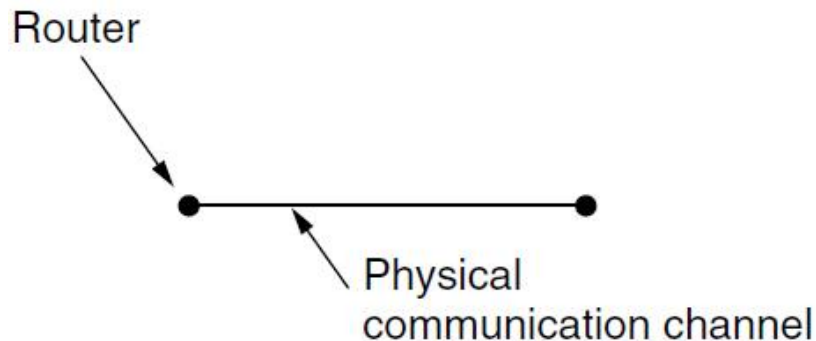


Outline

- 6.1 Function and Services to Transport Layer
- 6.2 Elements of Transport Protocols
 - ◆ 6.2.1 Addressing
 - ◆ 6.2.2 Connection Establishment
 - ◆ 6.2.3 Connection Release
 - ◆ 6.2.4 Flow Control and Buffering
- 6.3 Congestion Control
- 6.4 User Datagram Protocol (UDP)
- 6.5 Transmission Control Protocol (TCP)

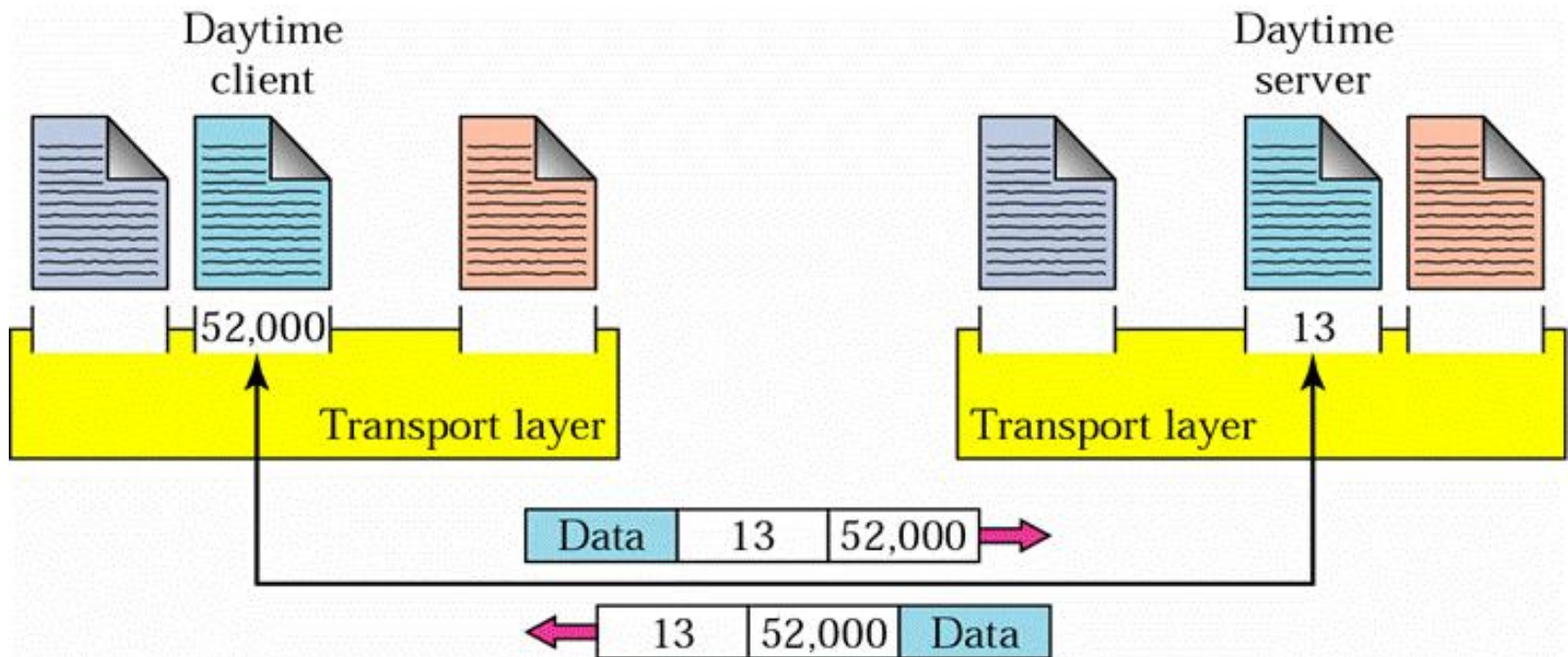
Transport Protocol & Data Link Protocol

- Both have to deal with **error control**, **sequencing(顺序)**, and **flow control**, among other issues.
- **node-to-node delivery** vs. **process-to-process delivery**
- **Environment differences**
 - ◆ Addressing of destinations
 - ◆ Initial connection establishment
 - ◆ Storage capacity in the network
 - ◆ **Difference of degree** (eg. Buffering and flow control)



6.2.1 Addressing: Port Number

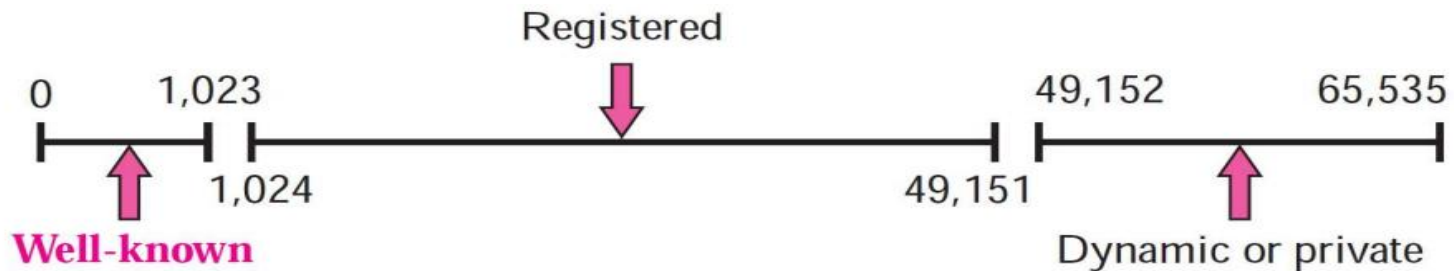
- SAP of the application process
- Deciding which application gets which messages



[Forouzan]

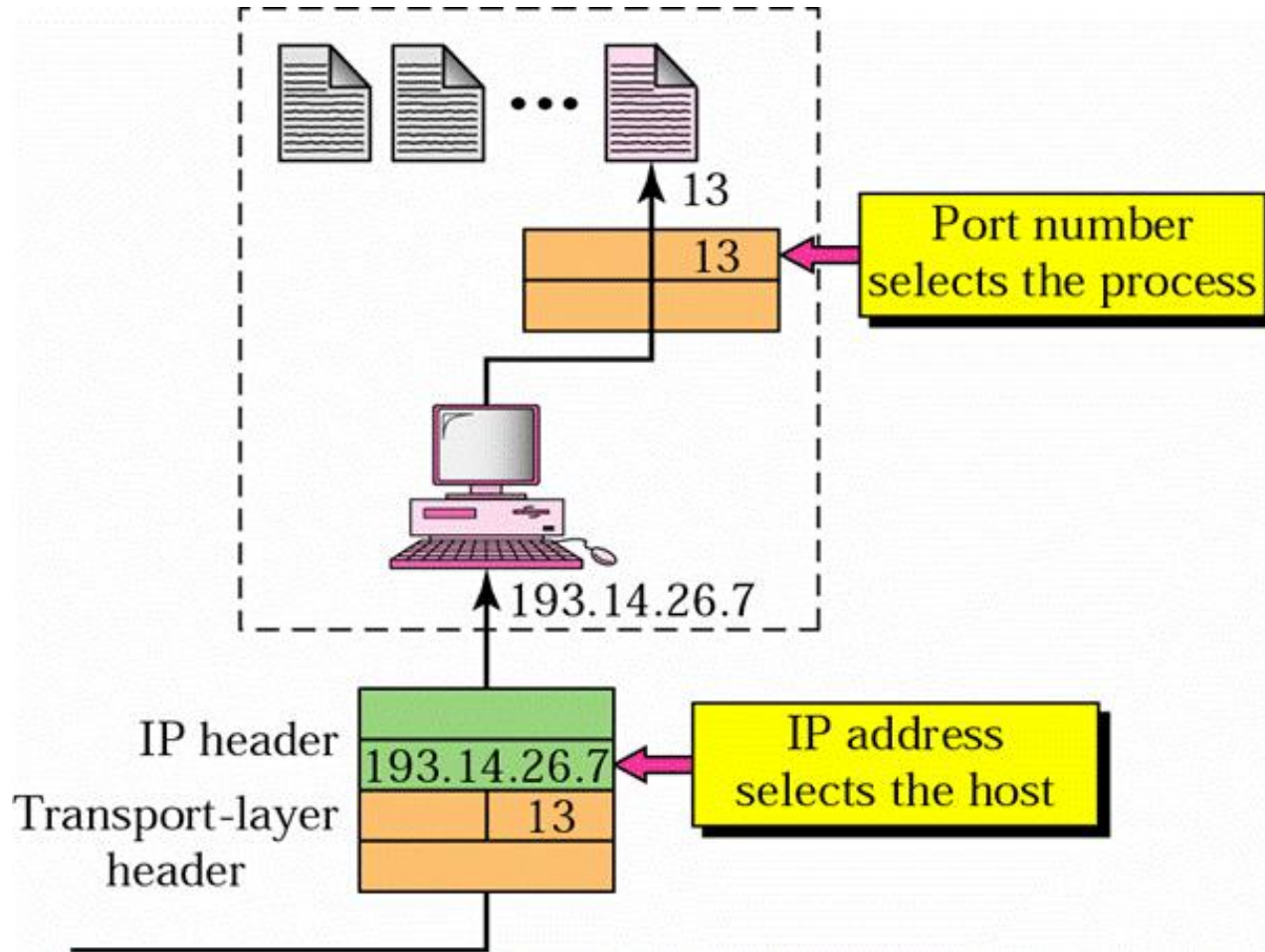
Port Number

- Well-known
- Registered
- Dynamic or private



[Forouzan]

IP Address vs. Port Number



[Forouzan]

Addressing

■ Gets the Server's TSAP

- ◆ Well-known TSAP (适用于少数永久不变的关键服务)

- mail server: 25
- web server: 80 or 443
- ...

- ◆ Portmapper(端口映射器): eg a **name server or directory server**

- 能够根据Client的服务请求将相应的TSAP地址返回给Client

■ Process server (节省资源)

- ◆ If there are many server processes, most of which are rarely used, it is wasteful to have each of them active
- ◆ 进程服务器作为不常用的服务器的代理
- ◆ 仅适用于服务器进程可按需创建的情况

Process Server

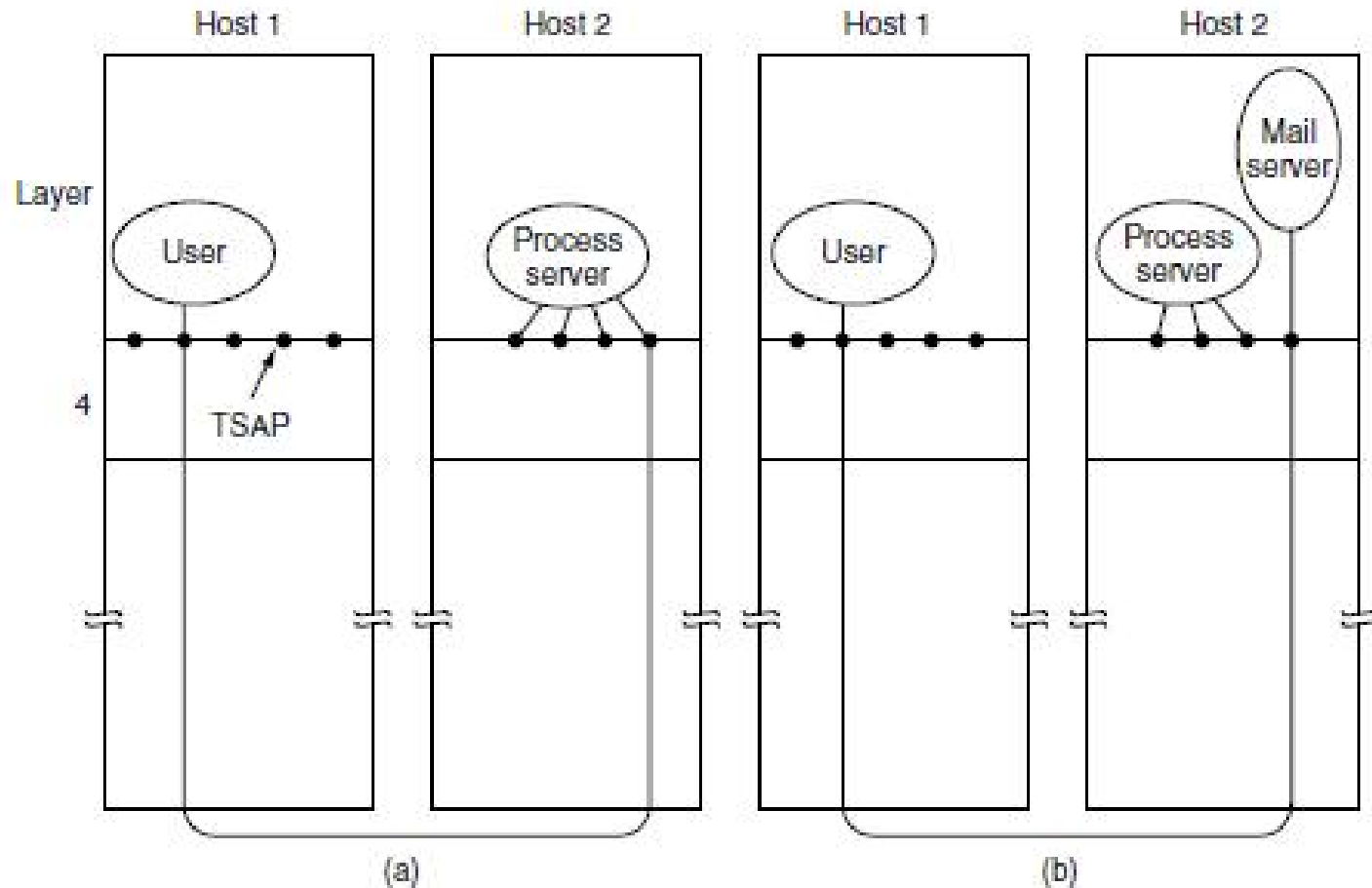
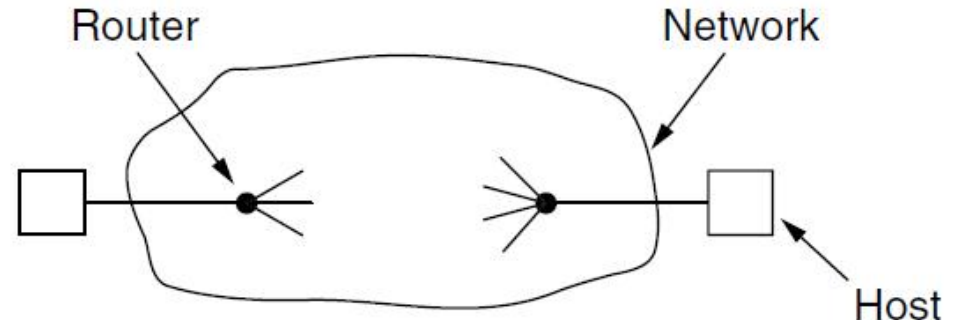
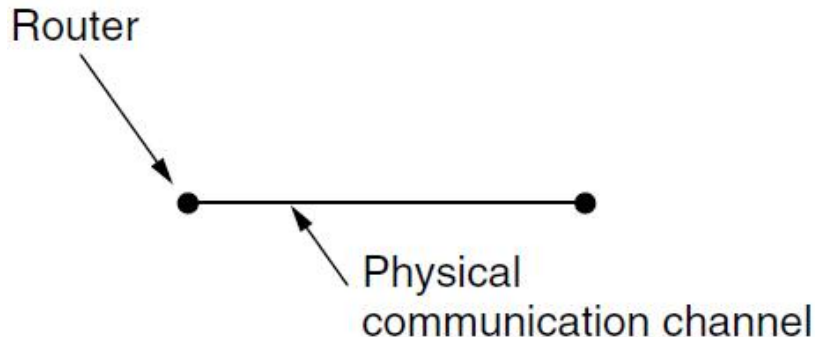


Figure 6-9. How a user process in host 1 establishes a connection with a mail server in host 2 via a process server.

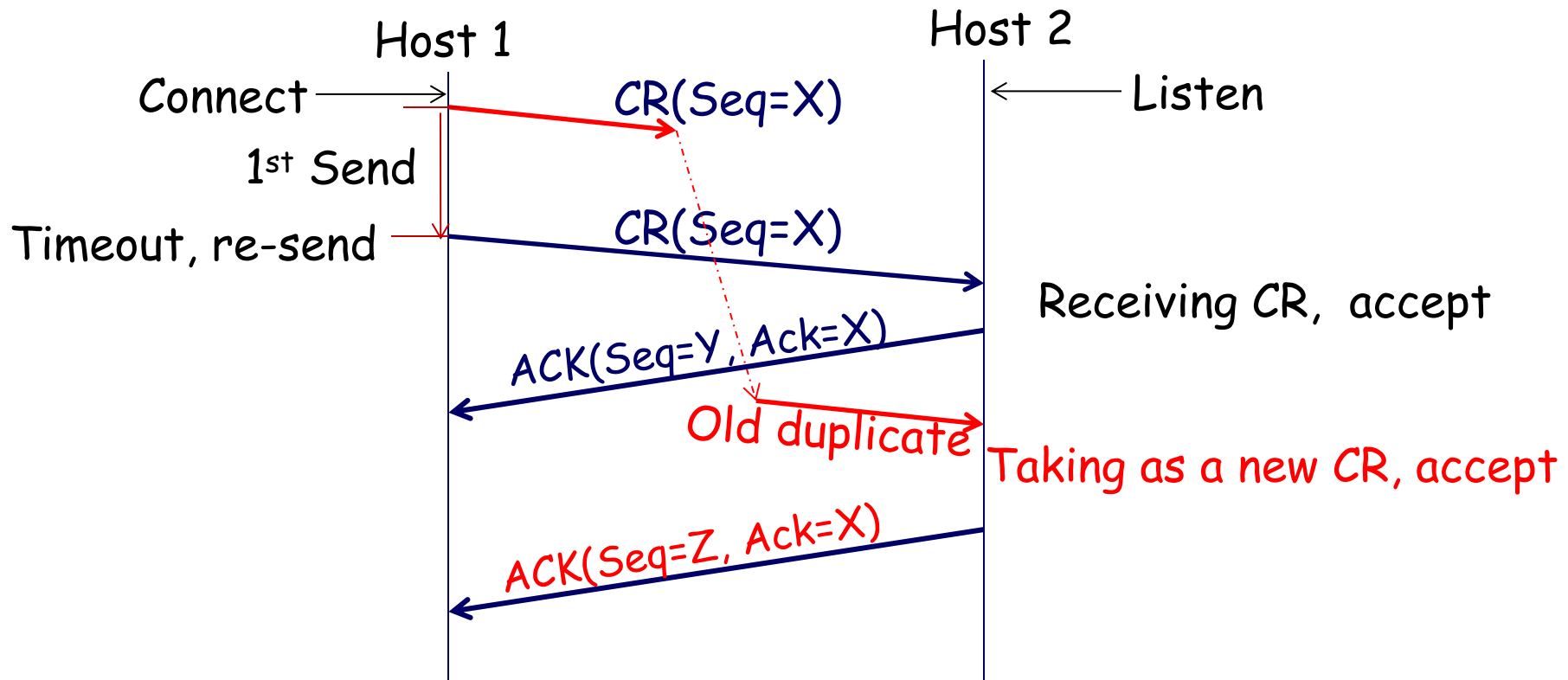
6.2.2 Connection Establishment: Problems

- Establishing a connection sounds easy
 - ◆ 2-way handshake (两次握手)
 - ◆ One sends a *ConnectionRequest* to the destination and wait for a *ConnectionAccepted* reply
- Problems
 - ◆ Network can **lose**, **delay** and **retransmit** packets
 - ◆ **Delayed duplicates**



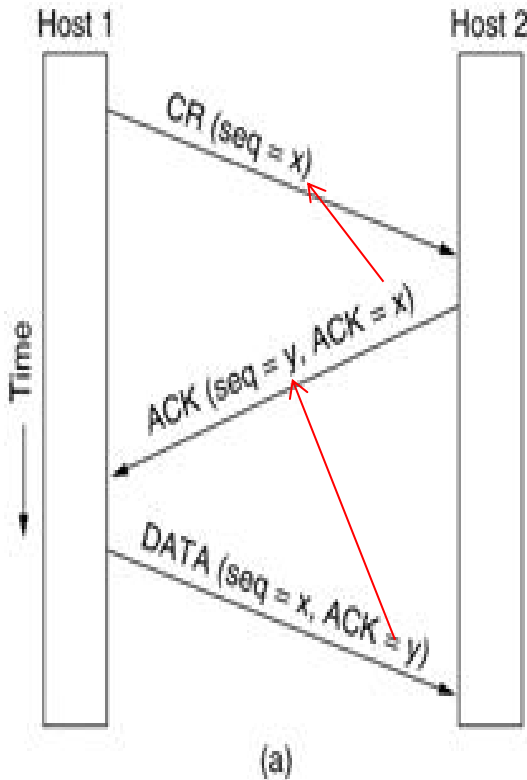
Problem of Delayed Duplicate CR

- Goals: 收发双方能够正确建立一个且仅一个连接
- *Connection Request(CR)* TPDU was delayed and retransmitted, causing Host2 to regard the **duplicate** CR as a **new** one

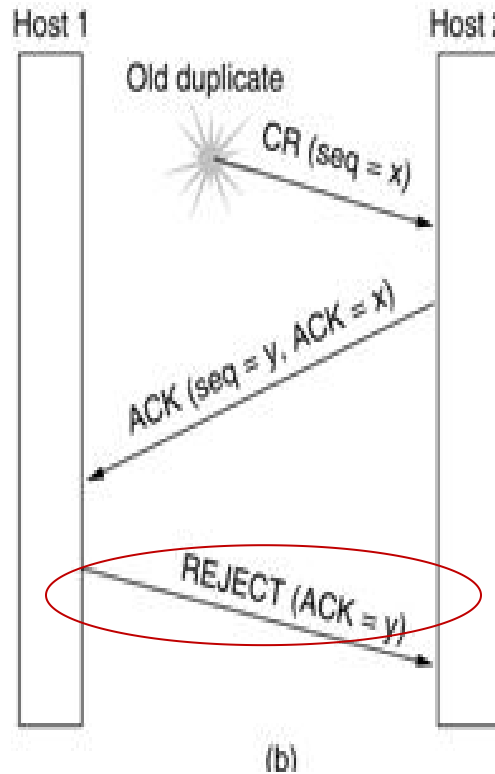


Reject duplicate Connection Request

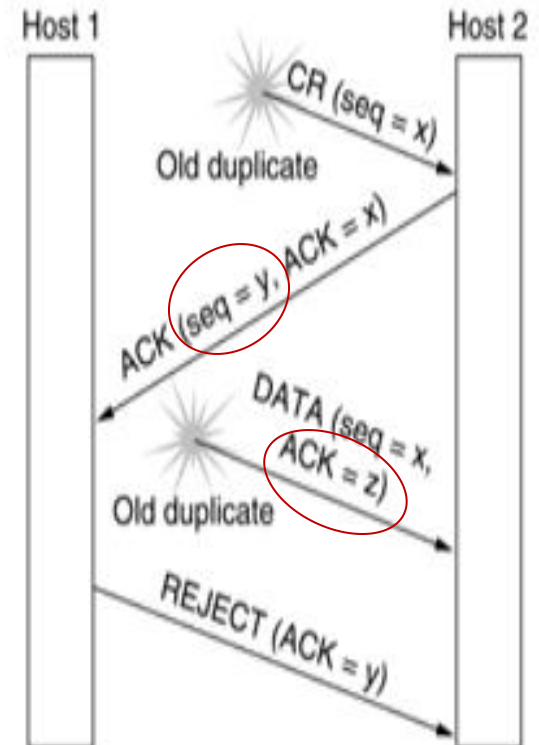
Solutions: 3 Way Handshake(三次握手)



Normal operation



Duplicate Connection Request



Duplicate Connection Request and duplicate ACK

➡ RST flag ➡ TCP Connection Management

Computer Networks, Transport Layer

6.2.3 Connection Release

■ Asymmetric Release (violent—突然)

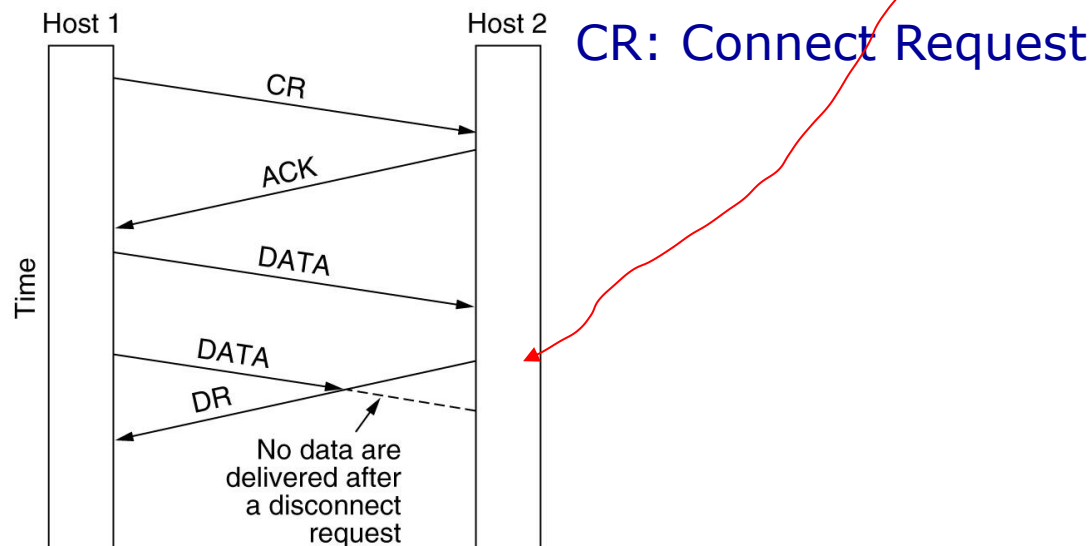
- ◆ when one party hangs up, the connection is broken
- ◆ Asymmetric release is abrupt and **may result in data loss**

■ Symmetric Release (graceful-平滑)

- ◆ Treats connection as two separate unidirectional connections
- ◆ Requires **each one to be released separately**

■ Goals

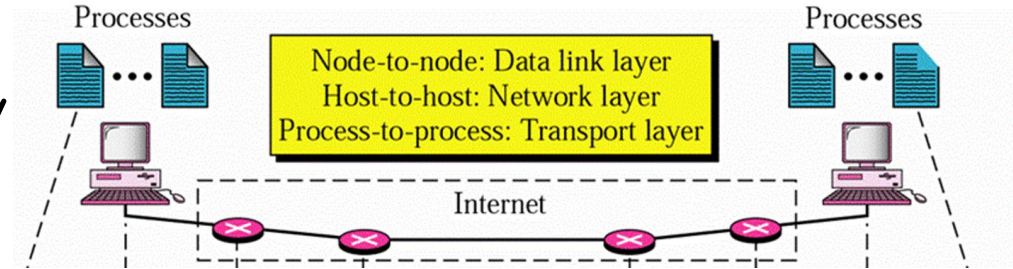
- ◆ 释放过程不丢包
- ◆ 无半开放连接**Half-open connection**(一方资源已经释放, 另一半未释放)



Review

■ Transport Layer

- ◆ process-to-process delivery

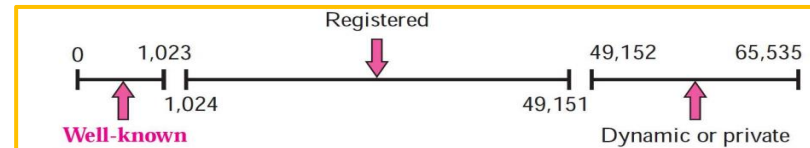


- ◆ Transport Protocol & Data Link Protocol **Environment differences**

■ Elements of Transport Protocols

- ◆ Addressing: **port number**

- Well-known, Registered, Dynamic/private

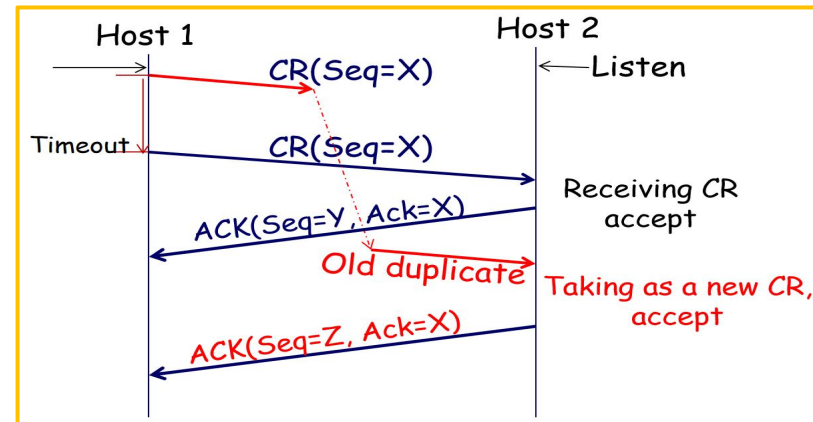


- ◆ Connection Establishment

- Connection: full-duplex
- 3-way handshake **Why?**

- ◆ Connection Release

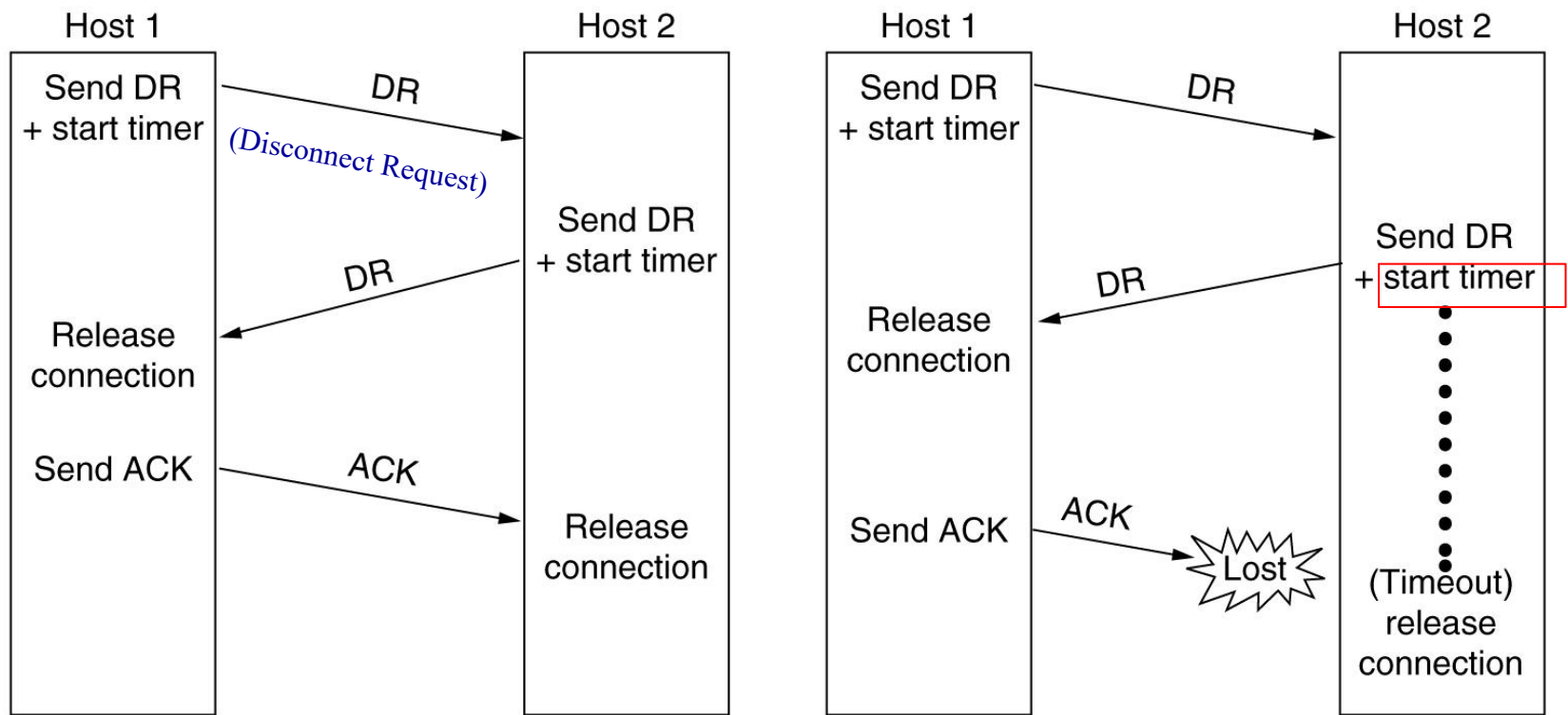
- **Symmetric Release**, each one to be released separately
- 3-way handshake



Connection Release

■ Releasing a connection means

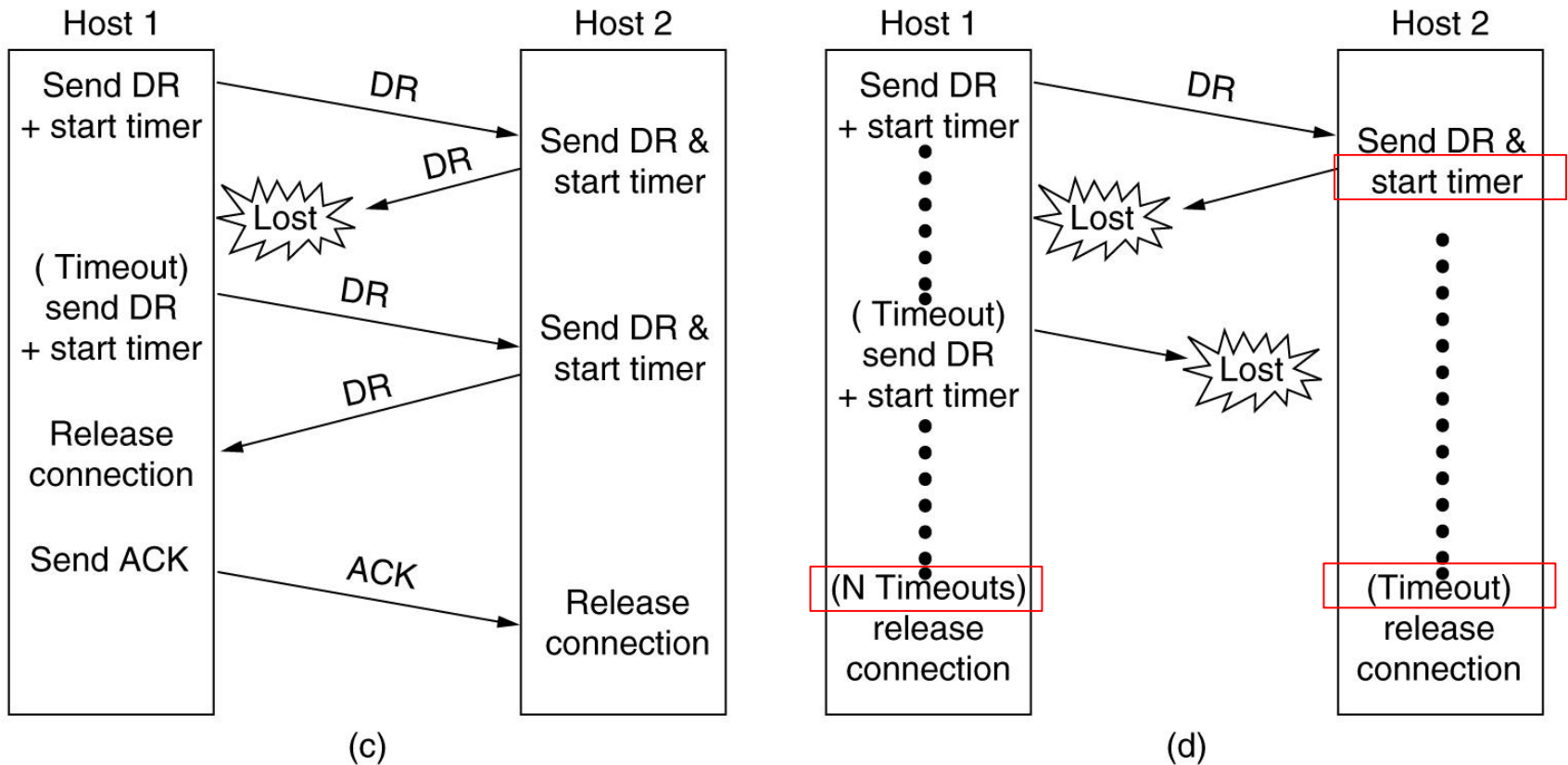
- ◆ The transport entity **removes the information** about the connection from its table of currently open connections and **signals the connection's owner** (the transport user)



(a) Normal case of a three-way handshake.

(b) final ACK lost.

Connection Release



(c) Response lost (d) Response lost and subsequent DRs lost

First DR is lost?

Connection Release

■ Half-open connection

- ◆ Initial DR and N retransmissions are all lost
- ◆ The **sender** will give up and **release the connection**
- ◆ The **receiver is still full active**

■ Killing off half-open connection

- ◆ **Automatically disconnected rule(自动断开连接规则)**: if no segments have arrived for a certain number of seconds, the connection is automatically disconnected
- ◆ **Dummy segment** 哑段(防误判)

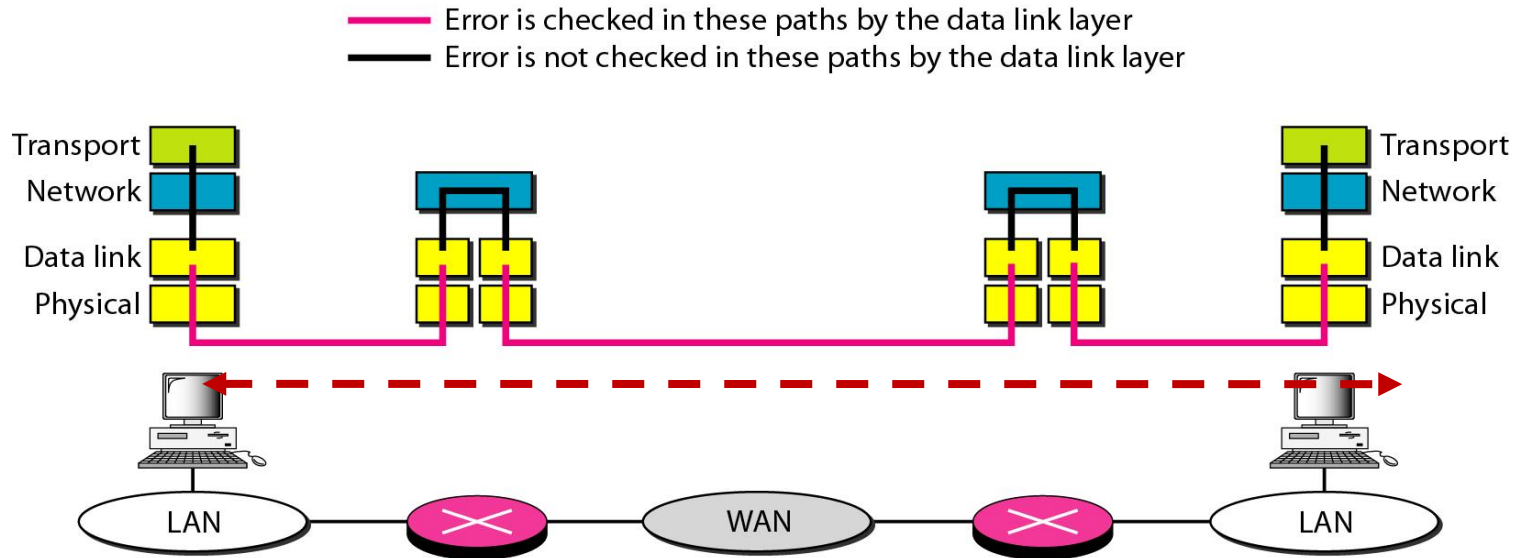
6.2.4 Error Control & Flow Control

■ ARQ(Automatic Repeat reQuest)

- ◆ A segment carries an **error-detecting code**(Checksum)
- ◆ A segment carries a **sequence number** to identify itself and is retransmitted by the sender until it receives an **acknowledgement** of successful receipt from the receiver (**sequence number + timer + ACK**)
- ◆ **Sliding window** protocol is used to support bidirectional data transfer.

■ End-to-end argument(端到端/进程-进程)

(1)Error Control: Transport Layer vs. Data Link Layer



■ difference in environment

- ◆ Reliability at the data link layer is between two **adjacent nodes**
- ◆ Reliability at transport layer ensures **end-to-end**.

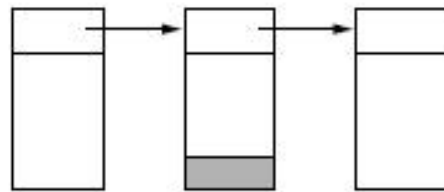
■ difference in degree (retransmissions and sliding window)

- ◆ DL: for wireless links(BD is small and **small window is sufficient**), for wired and optical fiber links(error-rate is low and **retransmissions can be omitted**)
- ◆ TL: a **larger sliding window** is used and the issue of **buffering data is important**

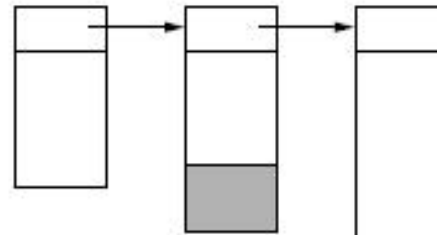
(2) Buffering Trade-off

- The buffers are needed at both the sender and the receiver.
- The best Trade-off depends on type of traffic
 - ◆ For low-bandwidth bursty traffic, such as BBS
 - it is reasonable not to dedicate any buffers, but rather to acquire them dynamically at both ends
 - relying on buffering at the sender if segments must occasionally be discard.
 - ◆ For file transfer and other high-bandwidth traffic
 - the receiver shall does dedicate a full window of buffers, to allow the data to flow at maximum speed.
 - TCP uses this strategy.

How to organize the buffer pool

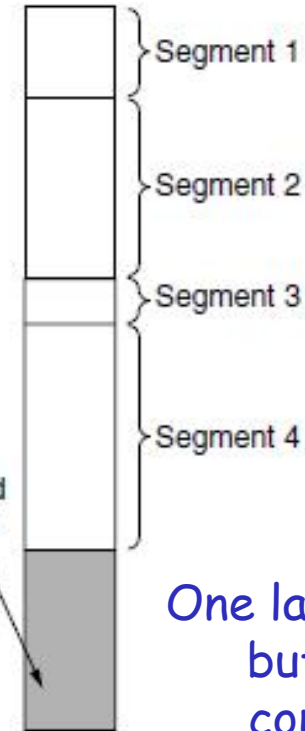


Chained fixed-size buffers



Chained variable-sized buffers

better memory **utilization**,
more **complicated** buffer
management.

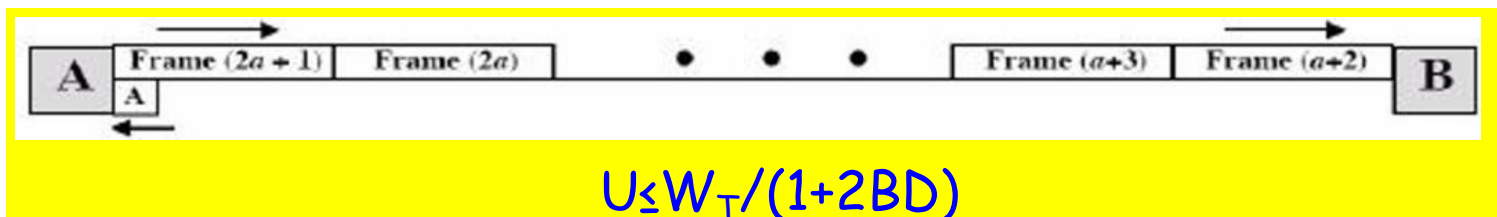


One large circular
buffer per
connection

simple and **does not depend on segment sizes**, but makes good use of memory only when the connections are **heavily loaded**

(3)Flow Control

- Sending window W_T is related to BD



- BD is large and variable, variable-sized sending windows

- Initial buffer negotiating

- Two flow control factor

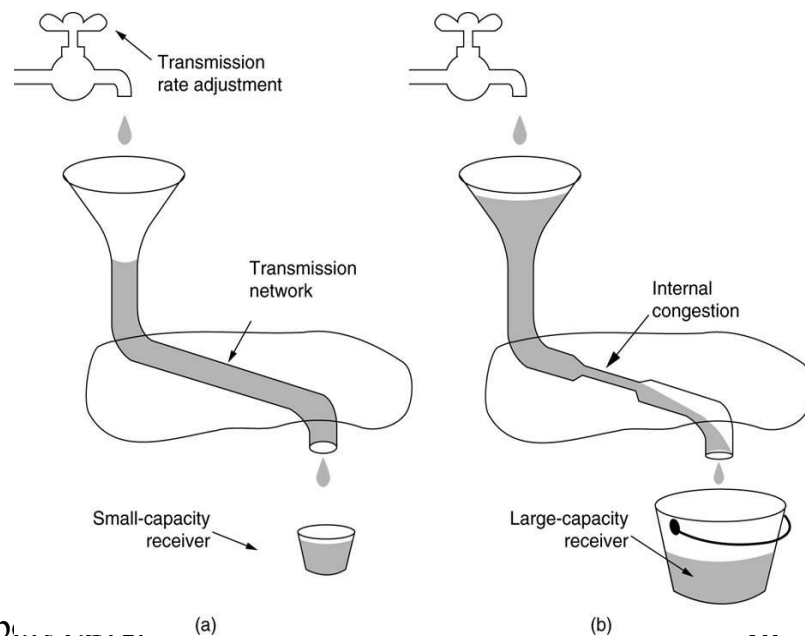
- Buffer space available in the receiver

explicit window indication

(flow control)

- The capacity of the network

sender monitor the network's carrying capacity (congestion control)

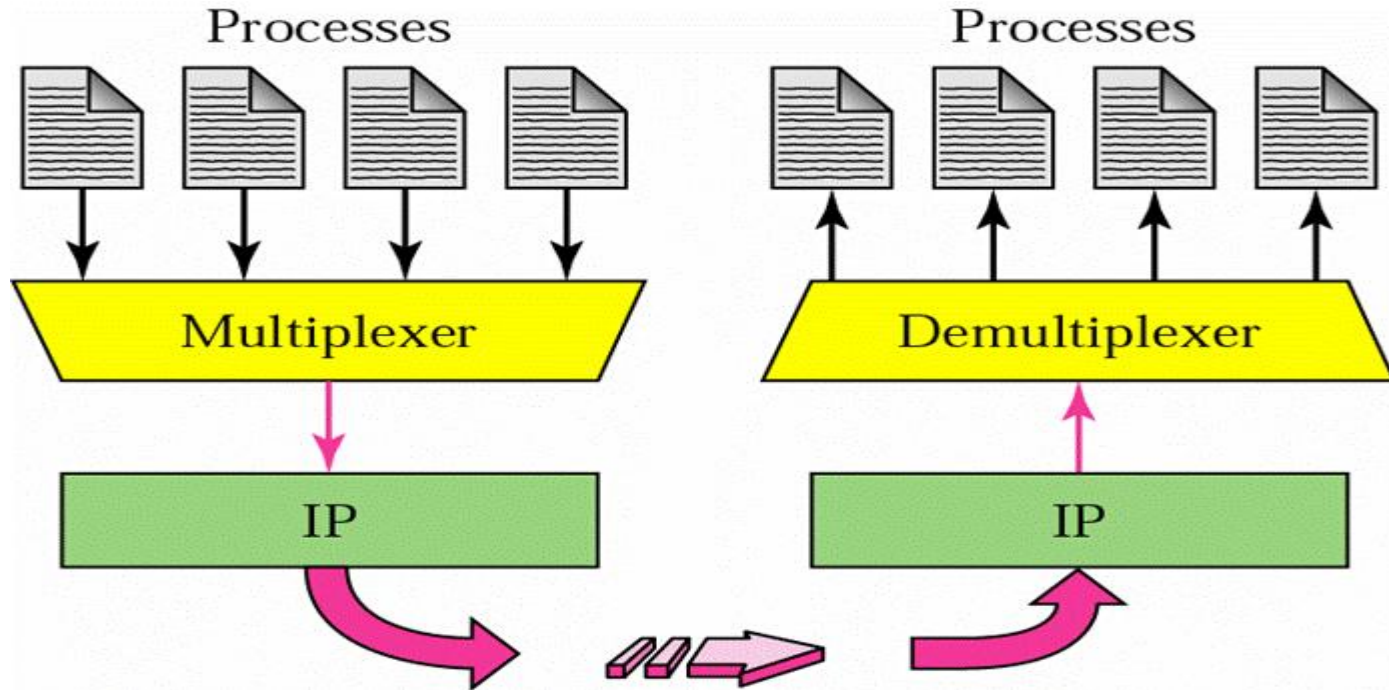


Flow Control: Dynamic Buffer Allocation

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

- The arrows show the direction of transmission
- An ellipsis (...) indicates a lost segment

6.2.5 Multiplexing(复用)



- At sender site, **multiplexing** at transport layer accepts packets from different processes, differentiated them by port numbers.
- At the receiver site, **demultiplexing** process works. Transport layer delivers each message to appropriate process based on port number.

[Forouzan]

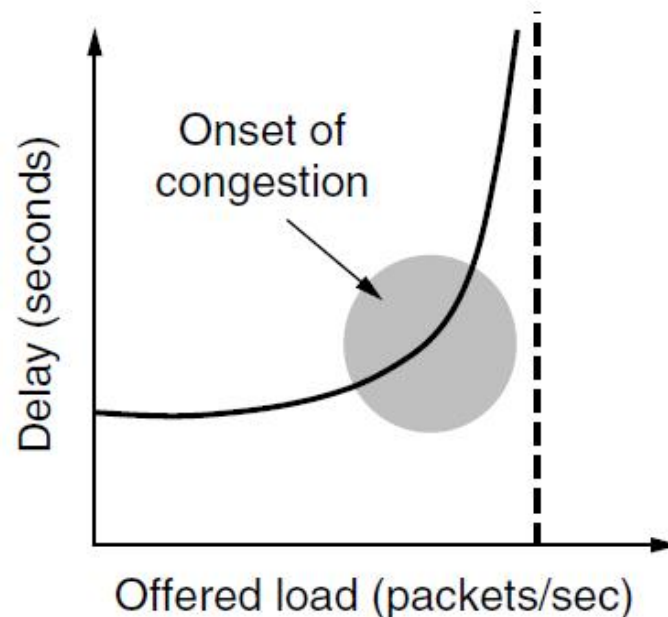
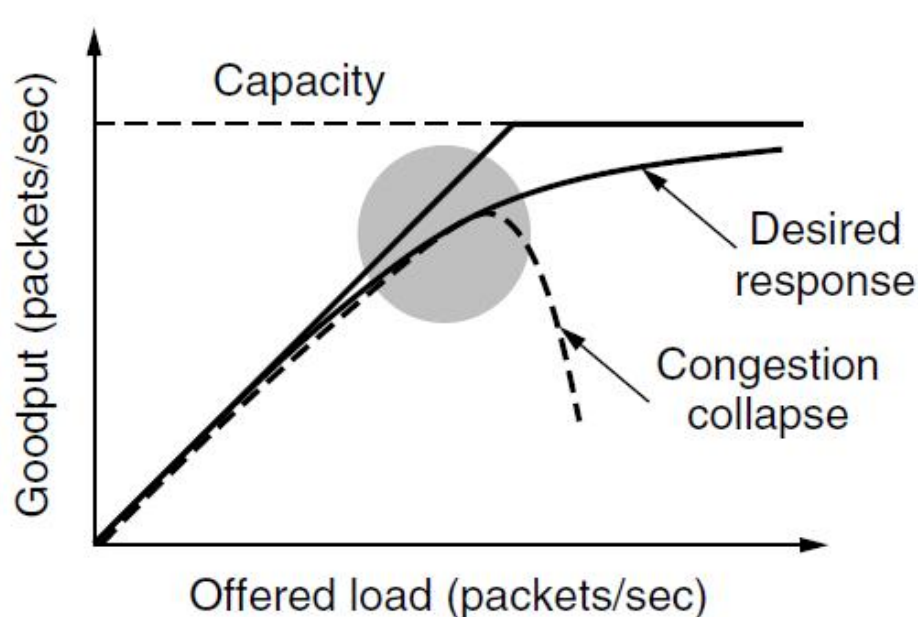
Outline

- 6.1 Function and Services to Transport Layer
- 6.2 Elements of Transport Protocols
 - ◆ 6.2.1 Addressing
 - ◆ 6.2.2 Connection Establishment
 - ◆ 6.2.3 Connection Release
 - ◆ 6.2.4 Flow Control and Buffering
- 6.3 Congestion Control
- 6.4 User Datagram Protocol (UDP)
- 6.5 Transmission Control Protocol (TCP)

(1) Congestion Control

- Responsibility of the network and transport layers
 - ◆ Congestion occurs at routers, so it is **detected** at the **network layer** (long delays, lost packets)
 - ◆ The only effective way is for the **transport protocols** to **send packets** into the network more **slowly**
- Internet relies **heavily** on the transport layer for congestion control (TCP)

Efficiency(效率) and Power(功率)



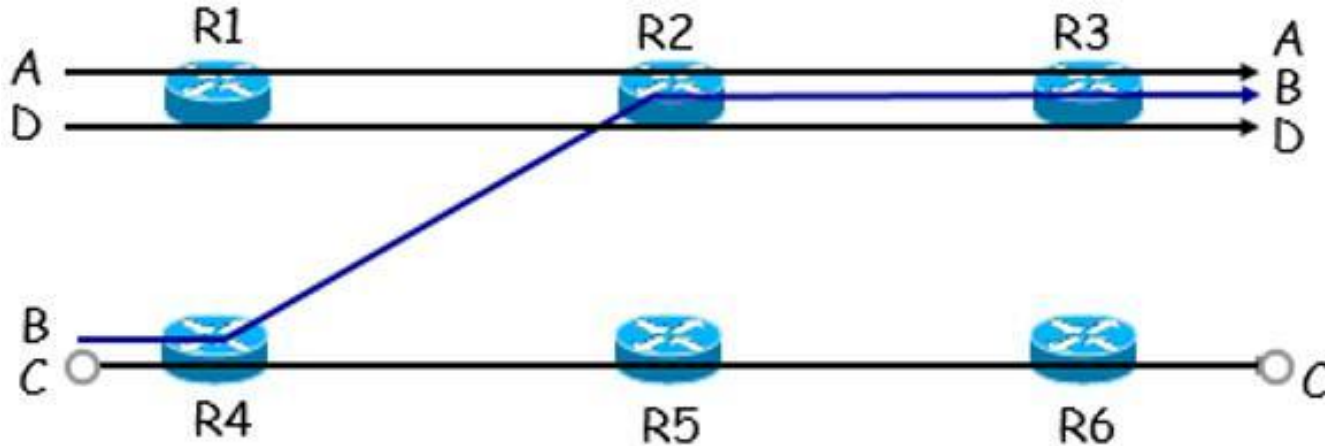
$$power = \frac{load}{delay}$$

- The load with the **highest power** represents an **efficient load**(即最大吞吐量) for the transport entity to place on the network

(2) How to divide bandwidth between transport senders?

■ Goal of Congestion Control Algorithm in transport layer

- ◆ **Avoid congestion** (拥塞)
- ◆ Find a good allocation of bandwidth to use **all** the available bandwidth (Efficiency效率)
- ◆ Be fair across competing transport entities (Fairness公平)
- ◆ Quickly track changes in traffic (Convergence收敛)

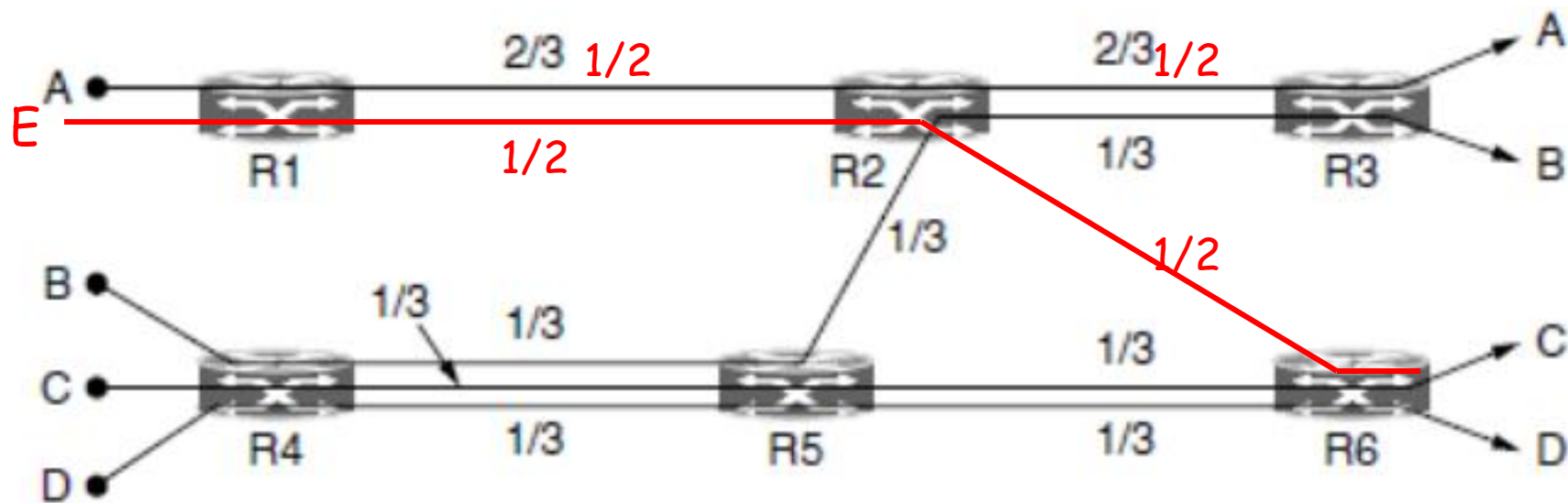


How to divide bandwidth between transport senders fairly?

■ Max-Min Fairness(最大最小公平性)

- ◆ If and only if the allocation is feasible and an attempt to **increase the allocation** of any participant necessarily **results in the decrease** in the allocation of some other participant with an equal or smaller allocation.
- ◆ 含义：资源分配向量的**最小分量的值最大**，防止任何网络流被‘饿死’，同时在一定程度上尽可能增加每个流的速率。
- ◆ 原则：一个信道上，在**满足最小需求**的前提下，各流尽量**均分带宽**；如果增加任何一个参与者（分量）的带宽，将导致其它的具有相同或者更少带宽的参与者（分量）的带宽下降，此时即满足了最大最小公平性。

Example of Max-Min Fairness



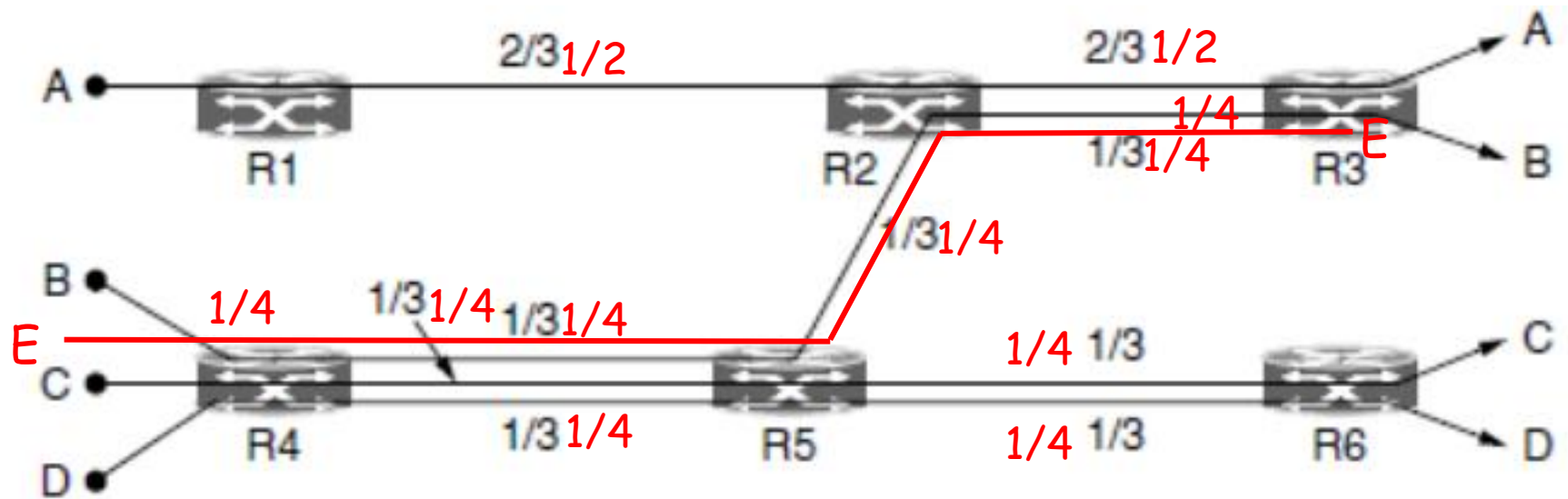
A-D: 4 flows

Each link has the same capacity, 1 unit(单位1)

如果增加B的带宽，将导致与B具有相同带宽的C和D的带宽减少，因此当前分配已满足**Max-Min Fairness**

问题：假如加入一个新的流E，路径E→R1→R2→R6,请问5个流的带宽如何变化？

Example of Max-Min Fairness



A-D: 4 flows

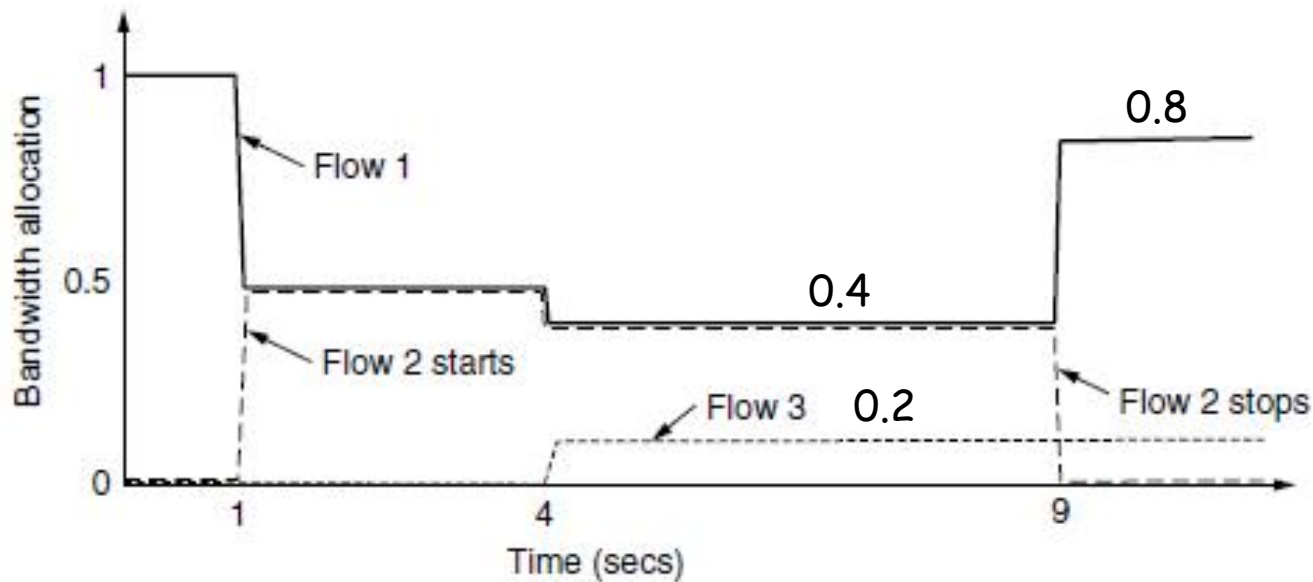
Each link has the same capacity, 1 unit(单位1)

如果增加B的带宽，将导致与B具有相同带宽的C和D的带宽减少，因此当前分配已满足**Max-Min Fairness**

问题：假如加入一个新的流E，路径E→R4→R5→R2→R3, 请问5个流的带宽如何变化？

Convergence(收敛)

- A good congestion control algorithm should rapidly converge to the ideal operating point(a fair and efficient allocation of bandwidth), and it should track that point as it changes over time. (尽快达到理想分配)



At all times, the total allocated bandwidth is approximately 100%, so that the network is fully used, and competing flows get equal treatment

(3) Congestion Control in Internet

■ 三步走

1. Congestion detection: **Router**根据队列长度或者线路利用率判断
2. Congestion notification: 将拥塞情况通知源主机
 显式拥塞通知: TCP with ECN
 或 **隐式拥塞通知**: (RED) 源主机(TCP)自己根据丢包情况判断
3. **Source** reduces traffic: 源主机采取措施 (TCP)
 在出现拥塞时, **降低发送速率**;
 在无拥塞时, **增加发送速率**;

Signals of some congestion control protocols

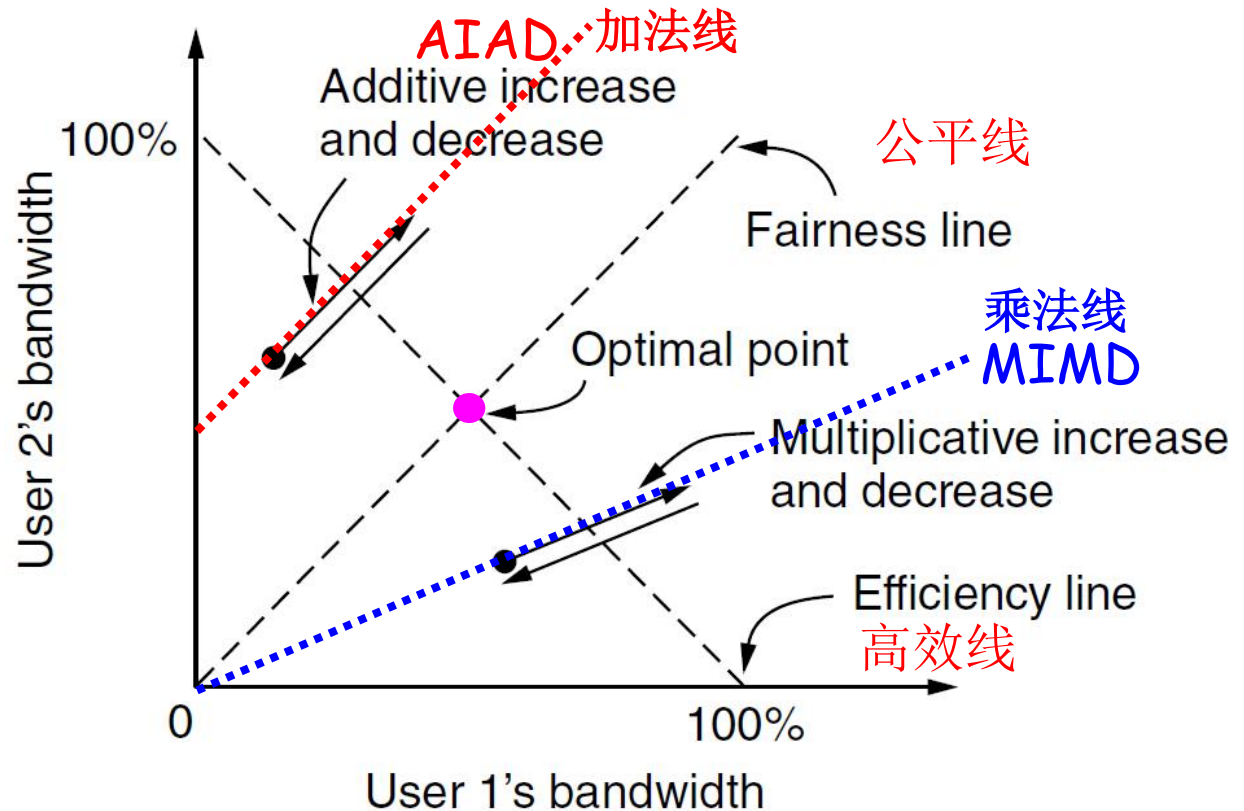
Protocol	Signal	Explicit?	Precise?
XCP	Rate to use	Yes	Yes
TCP with ECN	Congestion warning	Yes	No
FAST TCP	End-to-end delay	No	Yes
Compound TCP	Packet loss & end-to-end delay	No	Yes
CUBIC TCP	Packet loss	No	No
TCP	Packet loss	No	No

- How to regulate the Sending Rate: depends on the form of the feedback returned by the network
 - ◆ If an explicit and precise signal is given
 - ◆ Use the signal to adjust rate to the new operating point
 - ◆ In the other cases (control law)
 - ◆ No congestion signal, senders should increase their rates;
 - ◆ When a congestion signal is given, senders should decrease their rates

(4)Control Law: AIMD(加法增乘法减)

- Motive: to arrive at the **efficient and fair** operating point

Assumed two connections competing for the bandwidth of a single link.

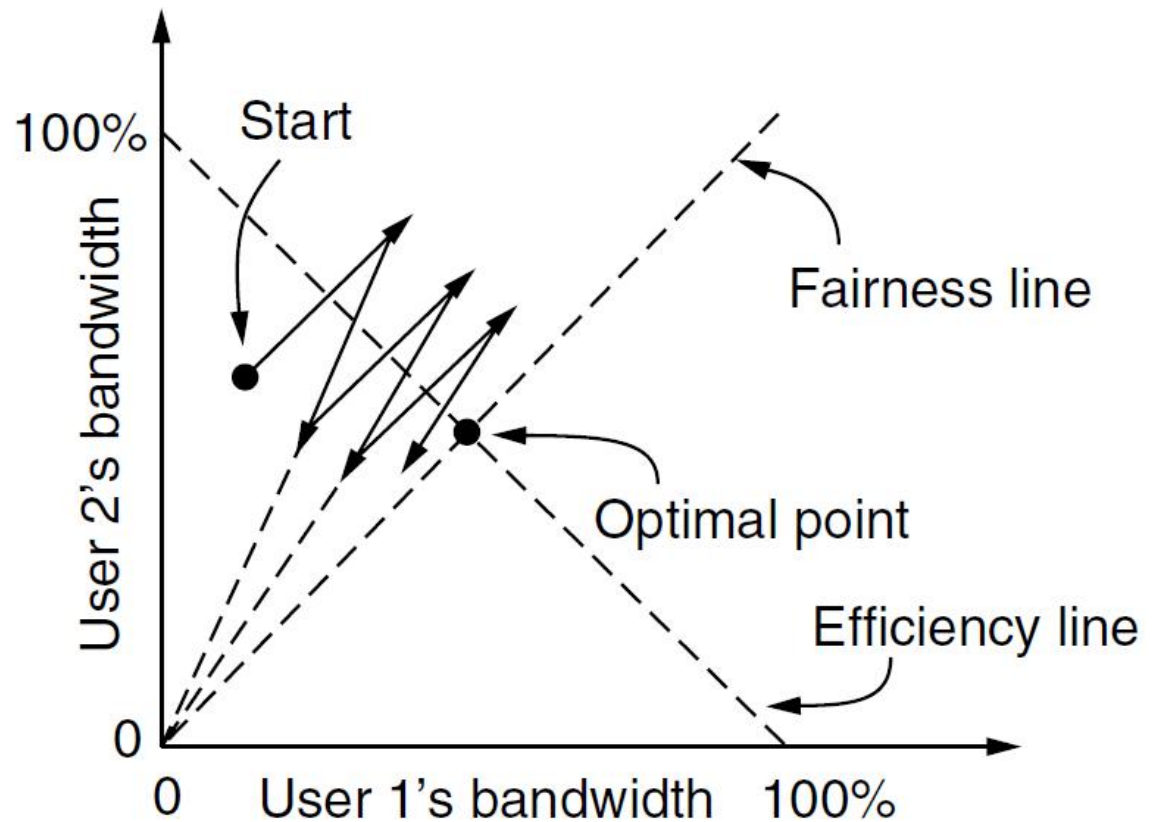


AIAD, MIMD or MIAD are hard to converge to the optimal point(which is both fair and efficient)!

Control Law: AIMD

- Additive Increase Multiplicative Decrease(AIMD): AIMD does converge to the optimal point that is efficient, fair, and convergence.

AIMD可以收敛于最优点



AIMD is used by TCP

■ Control laws used by TCP

◆ AIMD

- ◆ Stability argument: It is easy to drive the network into congestion and difficult to recover, so the **increase policy** should be **gentle** and the **decrease policy** should be **aggressive**.

■ Not adjusting the rate directly, a strategy used in practice is to **adjust the size of a sliding window**

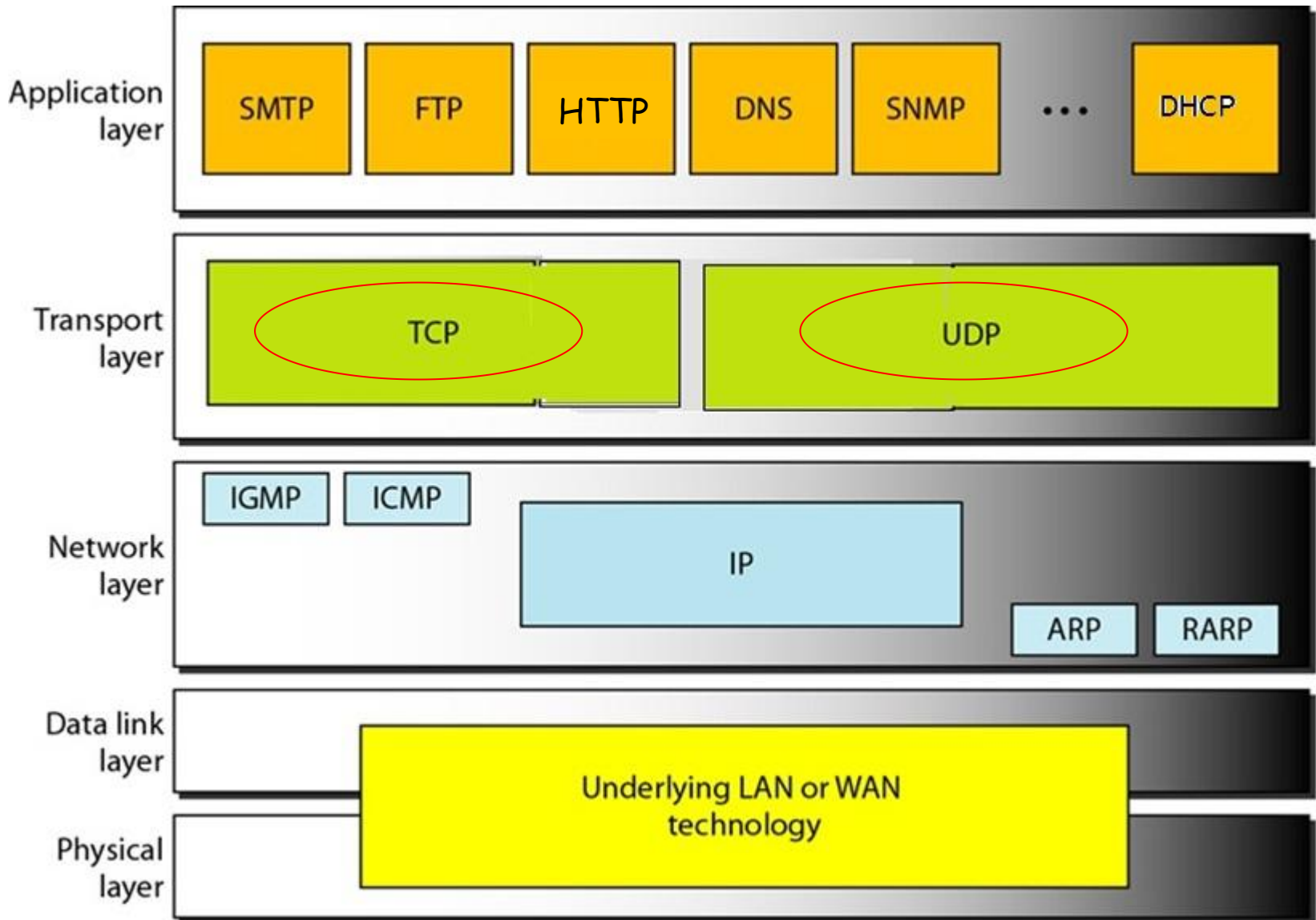
■ TCP-friendly congestion control

- ◆ TCP is the dominant form of congestion control
- ◆ New transport protocol should **complete fairly with TCP**
- ◆ TCP and non-TCP transport protocols can be friendly mixed with no ill effects

Outline

- 6.1 Function and Services to Transport Layer
- 6.2 Elements of Transport Protocols
 - ◆ 6.2.1 Addressing
 - ◆ 6.2.2 Connection Establishment
 - ◆ 6.2.3 Connection Release
 - ◆ 6.2.4 Flow Control and Buffering
- 6.3 Congestion Control
- 6.4 User Datagram Protocol (UDP)
- 6.5 Transmission Control Protocol (TCP)

Transport Layer in TCP/IP Model



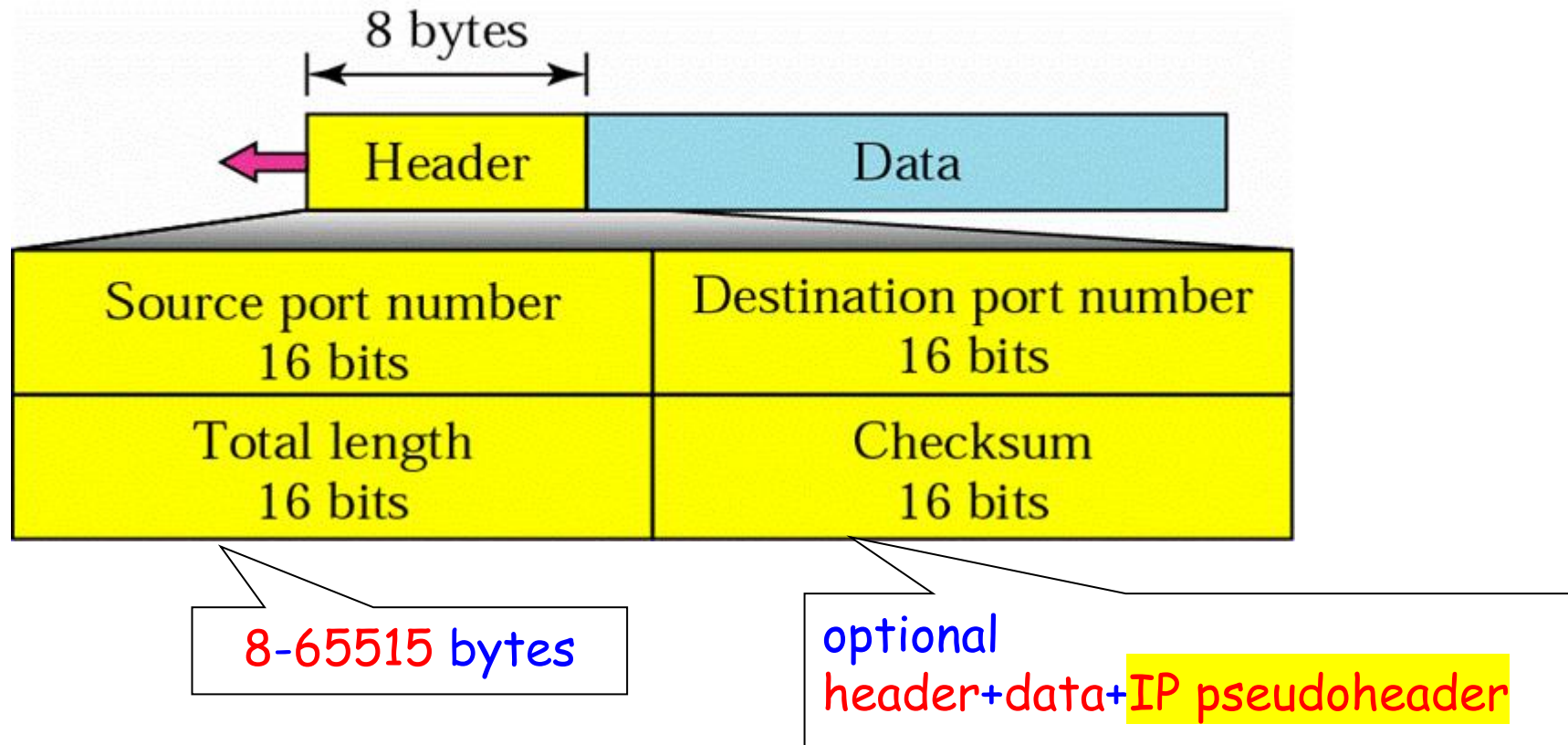
(1) UDP: User Datagram Protocol

- Connectionless
- Unreliable transfer
 - ◆ No acknowledgements to indicate delivery of data
 - ◆ No mechanism to error control
 - No mechanism to detect missing or mis-sequenced messages
 - No mechanism for automatic retransmission
 - ◆ No mechanism for flow control
 - ◆ No mechanism for congestion control
 - ◆ Same best-effort service model as IP
- Provides multiplexing/demultiplexing over IP
- Can be used to send broadcast datagrams(IPv4) or multicast datagrams(IPv4 and IPv6), in addition to unicast and anycast datagrams

(2) UDP Applications

- **DNS** (Domain Name System, 53)
- **DHCP** (Dynamic Host Configuration Protocol, 67/68)
- **RIP** (Routing Information Protocol)
(要求知道上述三种协议的传输层采用UDP协议)
- RTP (Real-time Transport Protocol)
- TFTP (Trivial File Transfer Protocol)
- RPC (Remote Procedure Call)
- NFS (Net File System)
- P2P (Peer-to-Peer, DHT, 'Kademlia')

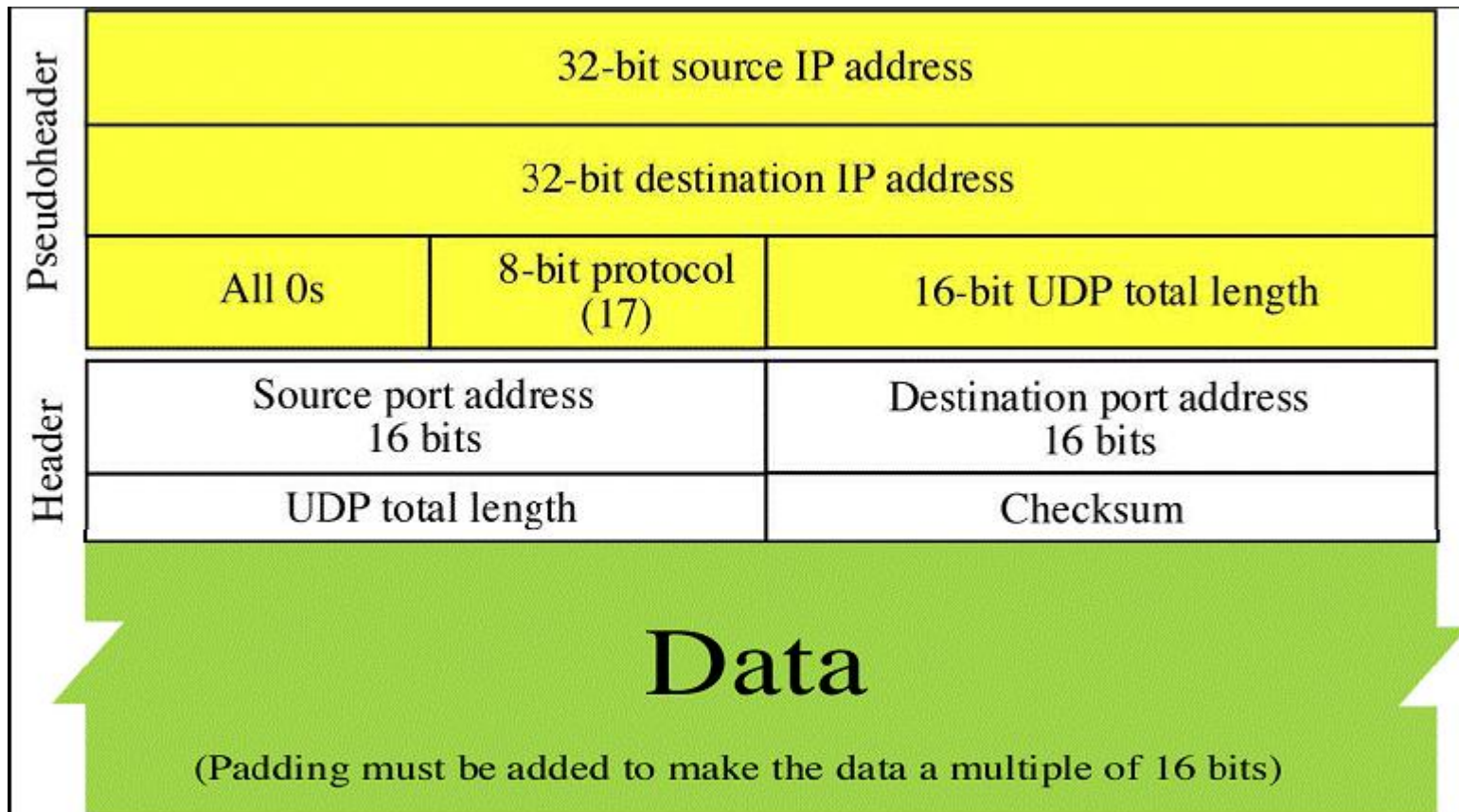
(3) UDP Datagram Format



UDP preserves **message boundaries** (保留消息边界) between the sender and the receiver.

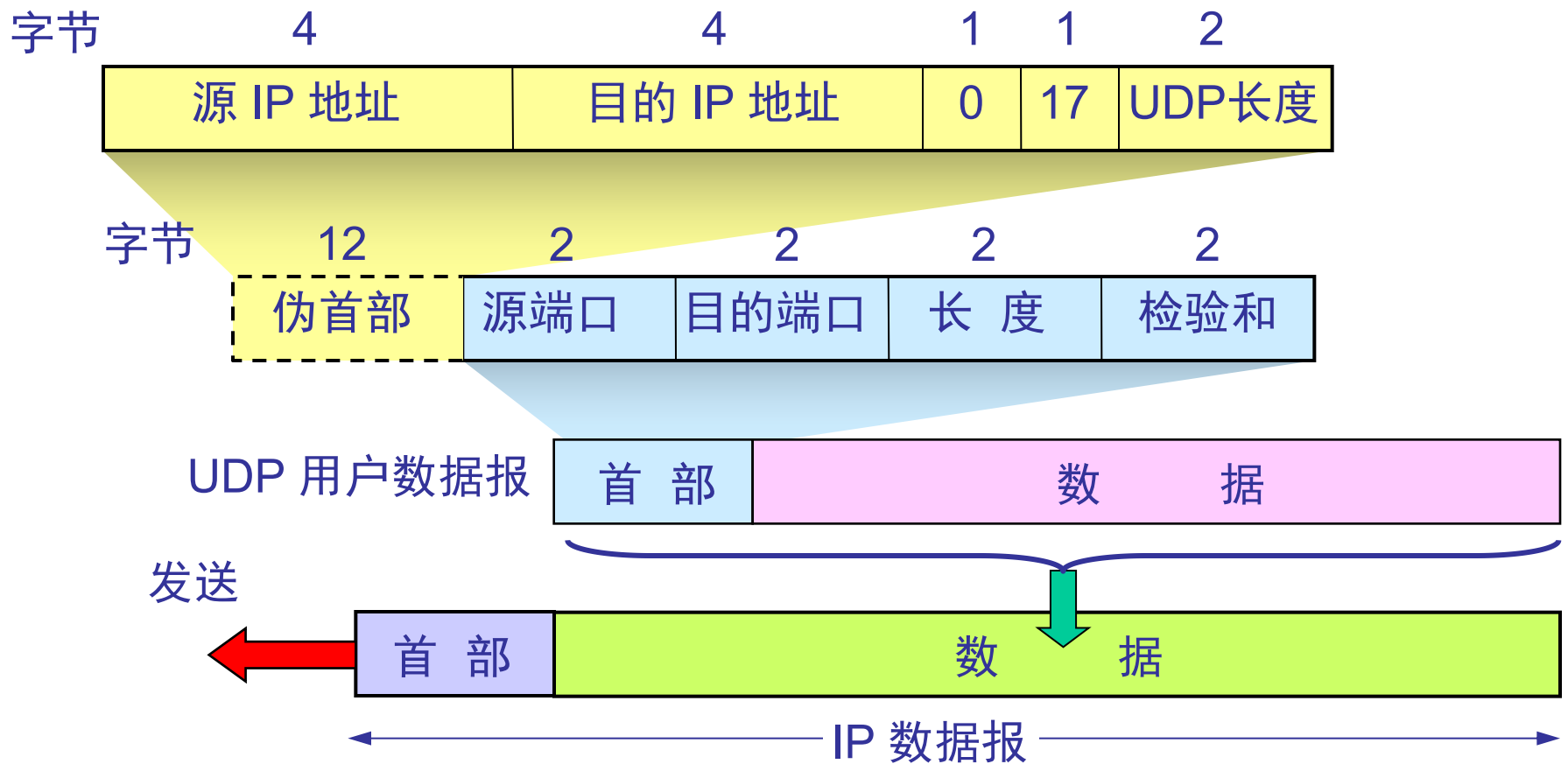
[Forouzan]

Pseudoheader(伪报头)

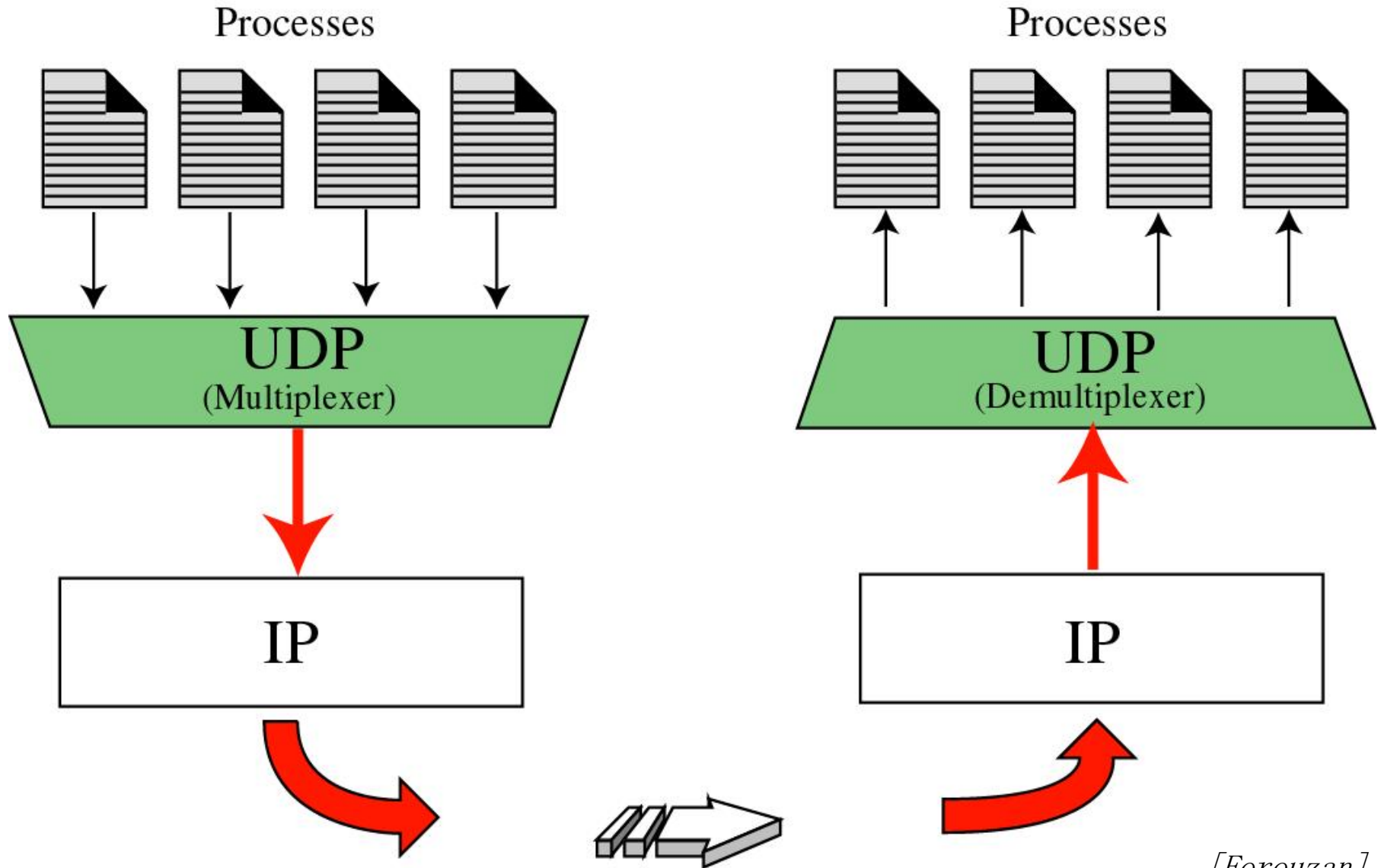


- A pseudo-header is created and computed for checksum, but not transferred.

UDP Checksum



(4) Multiplexing and demultiplexing



Outline

- 6.1 Function and Services to Transport Layer
- 6.2 Elements of Transport Protocols
 - ◆ 6.2.1 Addressing
 - ◆ 6.2.2 Connection Establishment
 - ◆ 6.2.3 Connection Release
 - ◆ 6.2.4 Flow Control and Buffering
- 6.3 Congestion Control
- 6.4 User Datagram Protocol (UDP)
- 6.5 Transmission Control Protocol (TCP)

6.5 TCP: Transmission Control Protocol

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

(1) Features of TCP

- TCP was designed to provide a connection-oriented reliable end-to-end byte stream over an unreliable internetwork
- Connection-oriented: TCP connection
 - ◆ Full-duplex
 - ◆ End-to-End: TCP supports **only unicast connections**, not support multicasting or broadcasting
 - ◆ Socket (four-tuple) (**source port + destination port**)
 - Each connection is identified by **a pair of sockets** eq.(socket1,socket2)
 - **socket address**: **IP address+Port** (16-bit, TSAP)
 - ◆ Well-known ports
 - Port number below 1024 is reserved for standard services
 - RFC1700
 - 'inetd' process (Internet daemon)(同时关联到多个端口上)

[Forouzan]

Well-known Ports of TCP

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing
23	Telnet	Remote login

Review(1)

■ Elements of Transport Protocols

◆ Congestion Control

- Goal: **Avoid congestion, Efficiency, Fairness, Convergence**
- Divide bandwidth between senders: **Max-Min Fairness(会计算)**
- Control Law: **AIMD**

■ UDP

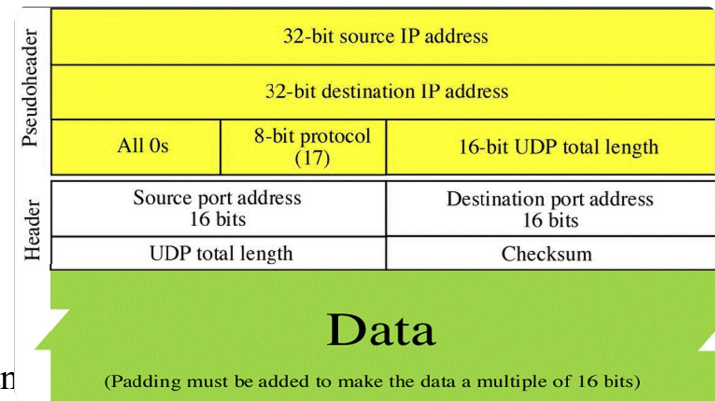
best-effort

preserves message boundaries

- ◆ Services: Connectionless unreliable message(datagram) stream
- ◆ Communication modes: **unicast, multicast, broadcast** (and **anycast**)
- ◆ PDU: **8 bytes** header
 - source port number
 - destination port number
 - checksum (optional)

□ header+data+**IP pseudoheader**

} **multiplexing/demultiplexing** over IP



Review(2)

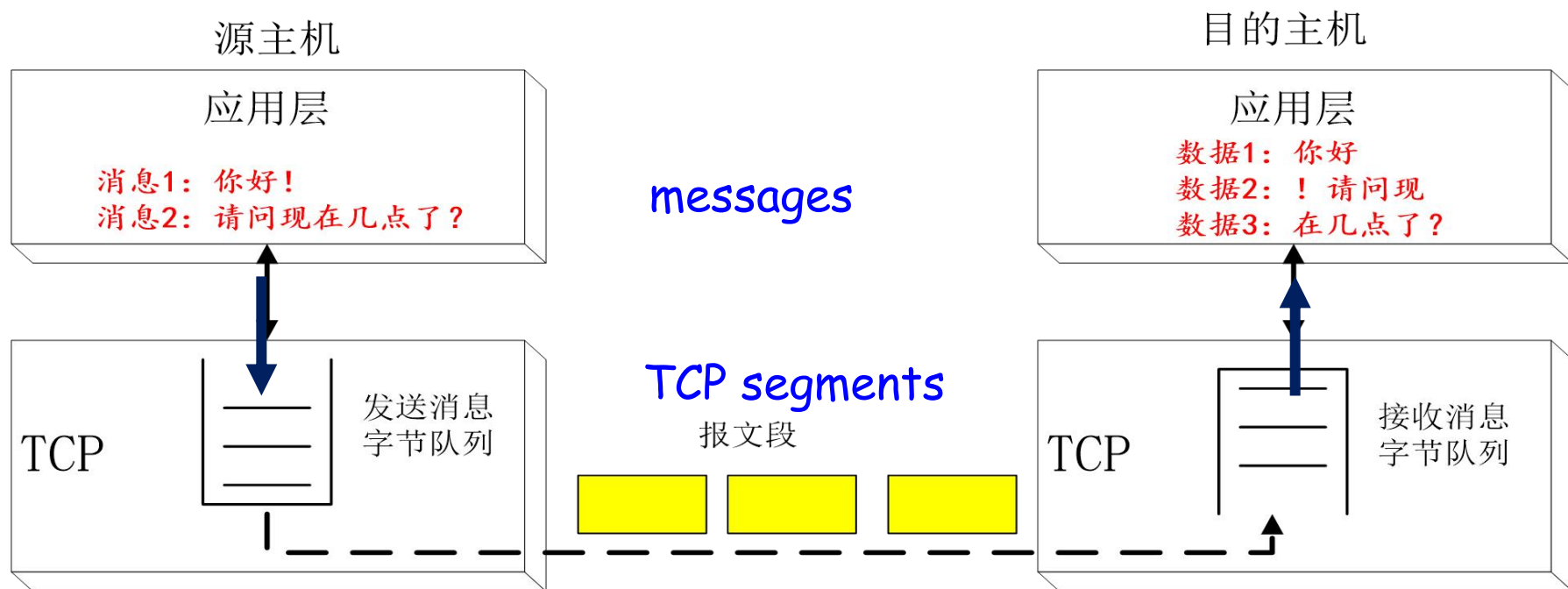
■ TCP

- ◆ Services: connection-oriented reliable end-to-end byte stream
- ◆ TCP connection
 - full-duplex unicast
- ◆ Reliable delivery: ARQ
 - Checksums + Retransmission + Acknowledgement
 - Sequence numbers, Ack number
 - Sliding Window Flow control
- ◆ End-to-End: socket address(IP address+Port number)
- ◆ Byte stream oriented

Features of TCP

- Reliable delivery (ARQ) (Checksums, sequence number, acknowledgement number, window size)
 - ◆ Checksums + Retransmission + Acknowledgement(cumulative ACK)
 - ◆ Sequence numbers + timer detect missing, or mis-sequenced data
 - ◆ Sliding Window Flow control prevents flooding of receiver
- Congestion control: to share network capacity among users
- Byte stream oriented: Message boundaries are not preserved end to end(不能保留消息边界)
 - ◆ TCP报文段内携带多少字节数据由TCP协议决定，与上层消息长度无关
 - ◆ The only thing that is guaranteed is that all data sent will be received without any error and in the correct order. (TCP仅保证交给上层的数据正确、按序) 
 - ◆ TCP连接上每个数据字节均被编号，即拥有自己的唯一的序号。Why?
 - ◆ Q&A: 是否有数据长度限制？接收方用户如何区分不同消息？

TCP不能保证上层应用消息的边界



TCP报文段内携带多少字节数据，是TCP自己决定的，跟上层消息的长度无关



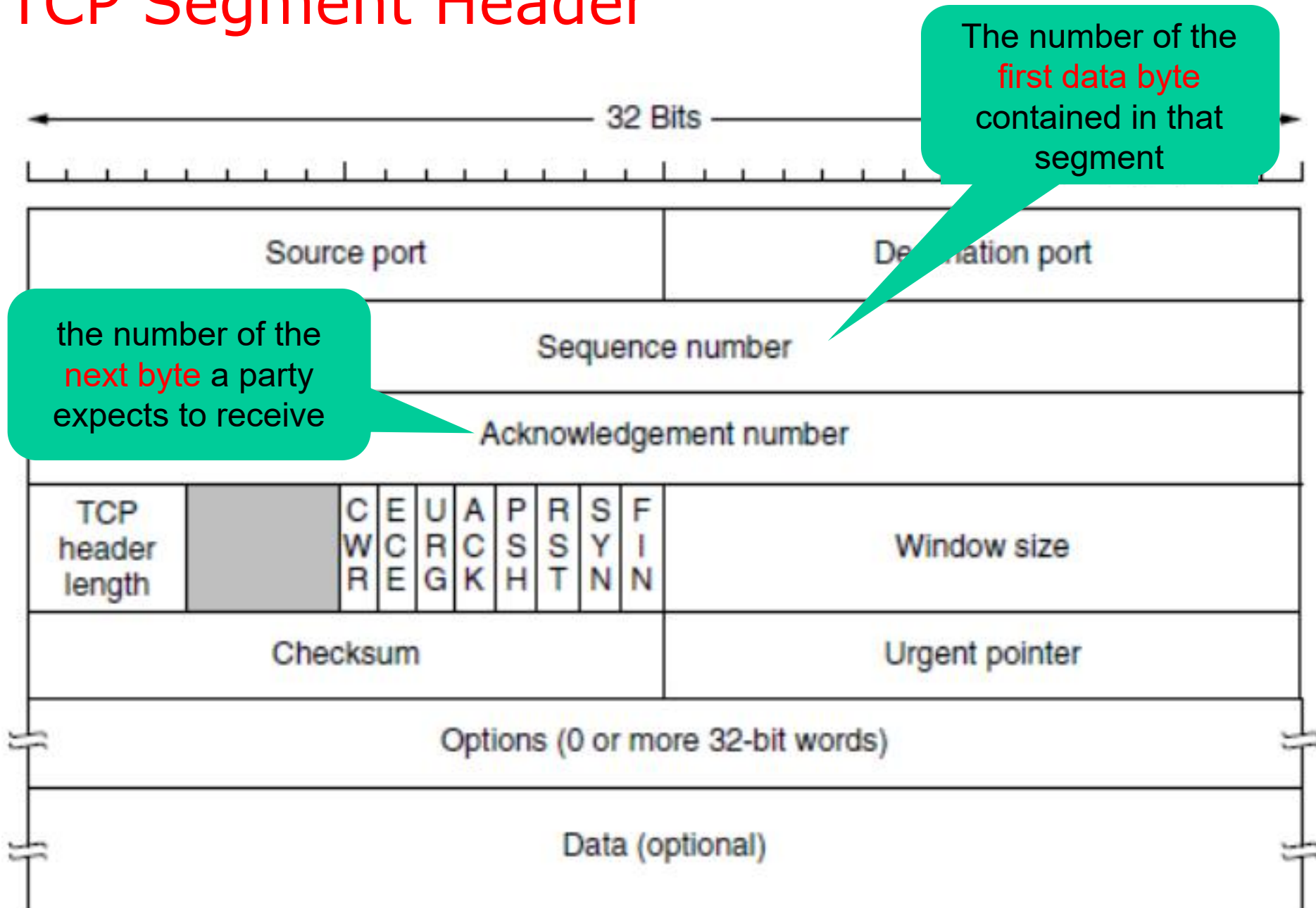
More on TCP Segment(报文段)

- TCP segment=(20 or more bytes) head+(n bytes) data
- Segments without any data are legal and are commonly used for ACK and control messages
- **MSS** (Maximum Segment Size): Maximum data length in a segment(payload length)
 - ◆ Limited by IP datagram length (65515 Bytes)
 - ◆ Limited by MTU of each data link (generally 1500 Bytes—Ethernet payload size), so router may fragment a segment
- Byte-oriented sequence number and ACK number
 - ◆ TCP段中的序号是该段第一个字节的序号
 - ◆ ACK段中的确认序号是下一个期待的字节的序号

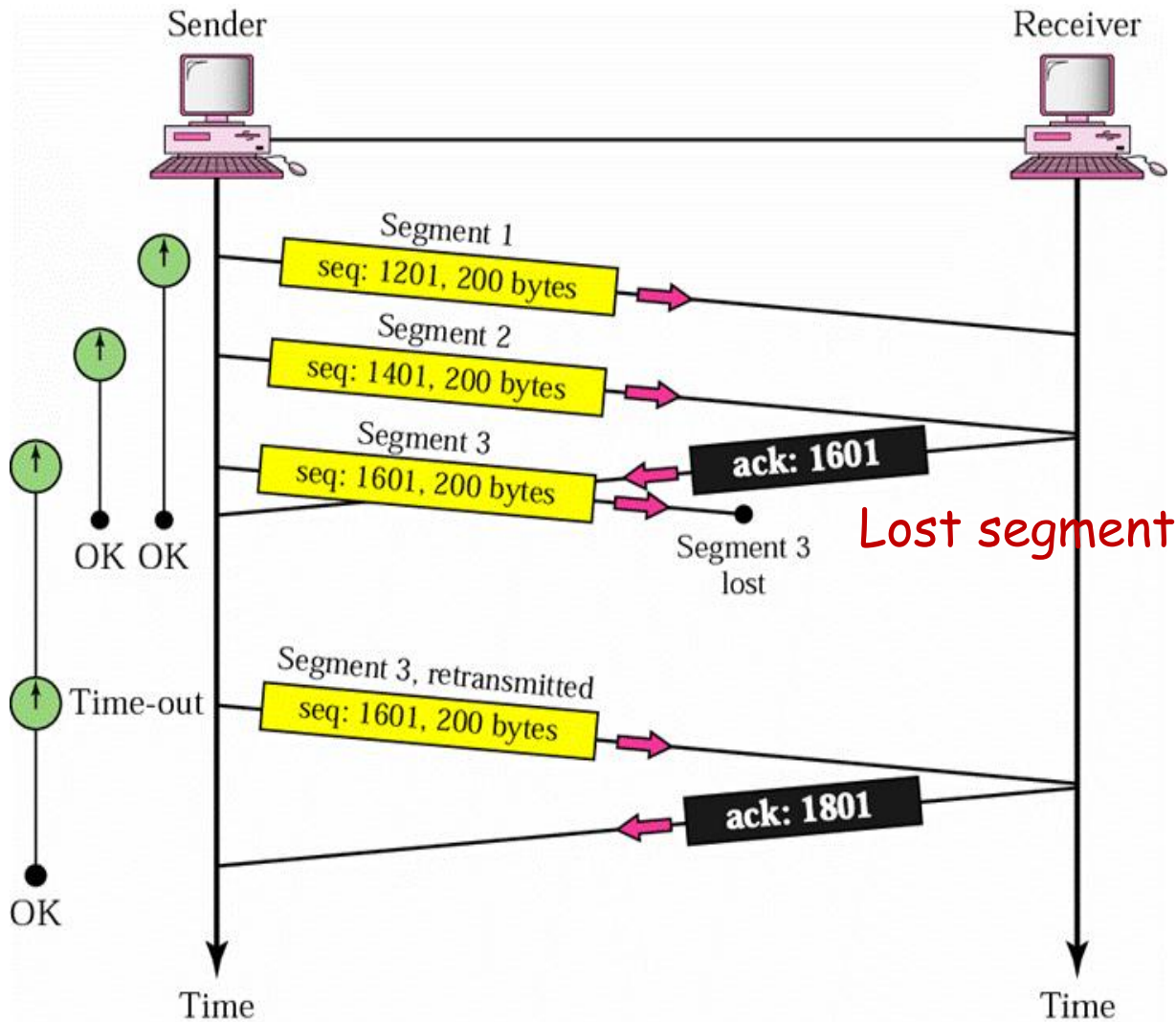
6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Header
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

TCP Segment Header



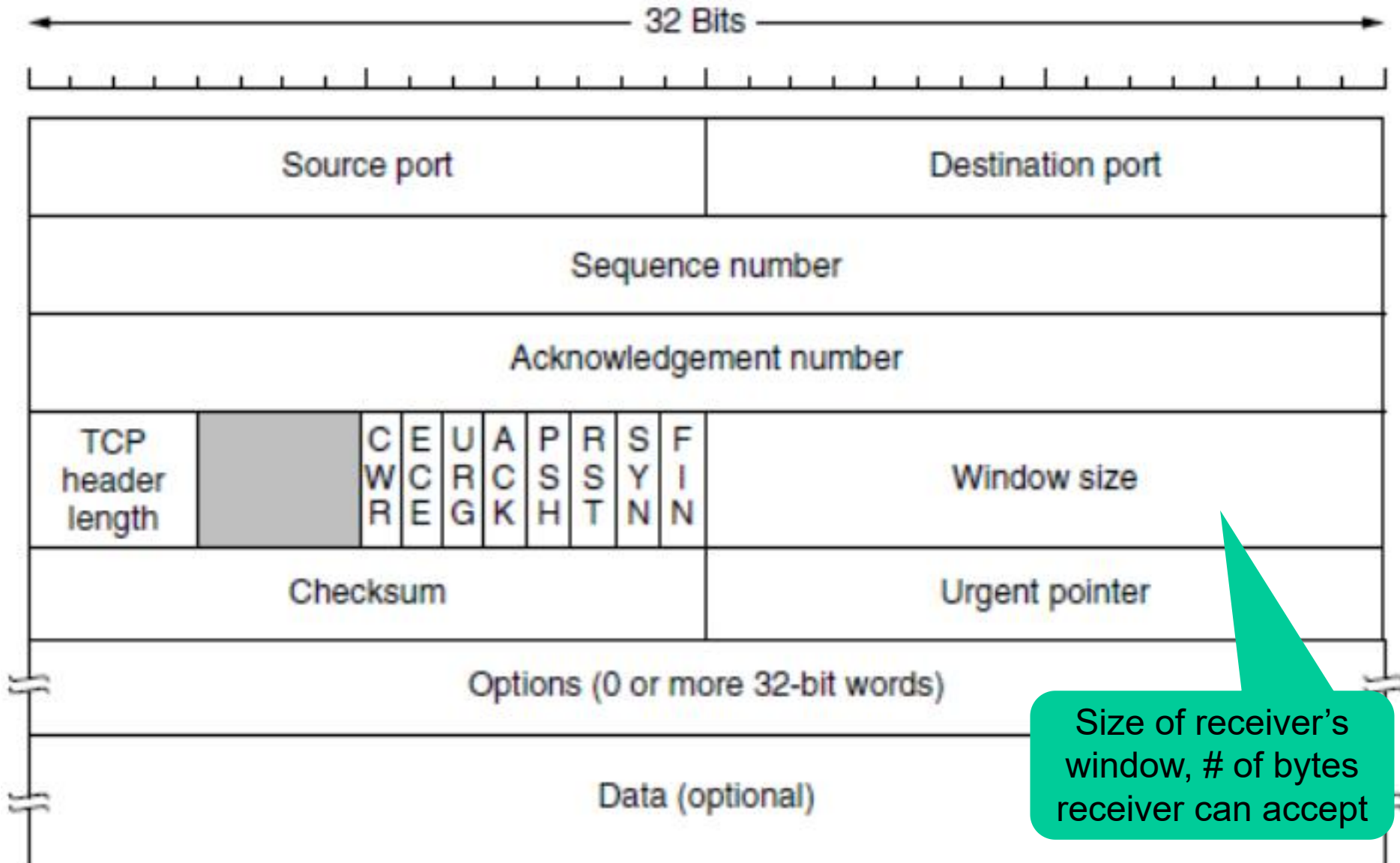
Example of Seq. No. , ACK No



Lost segment

[Forouzan]

TCP Segment Header



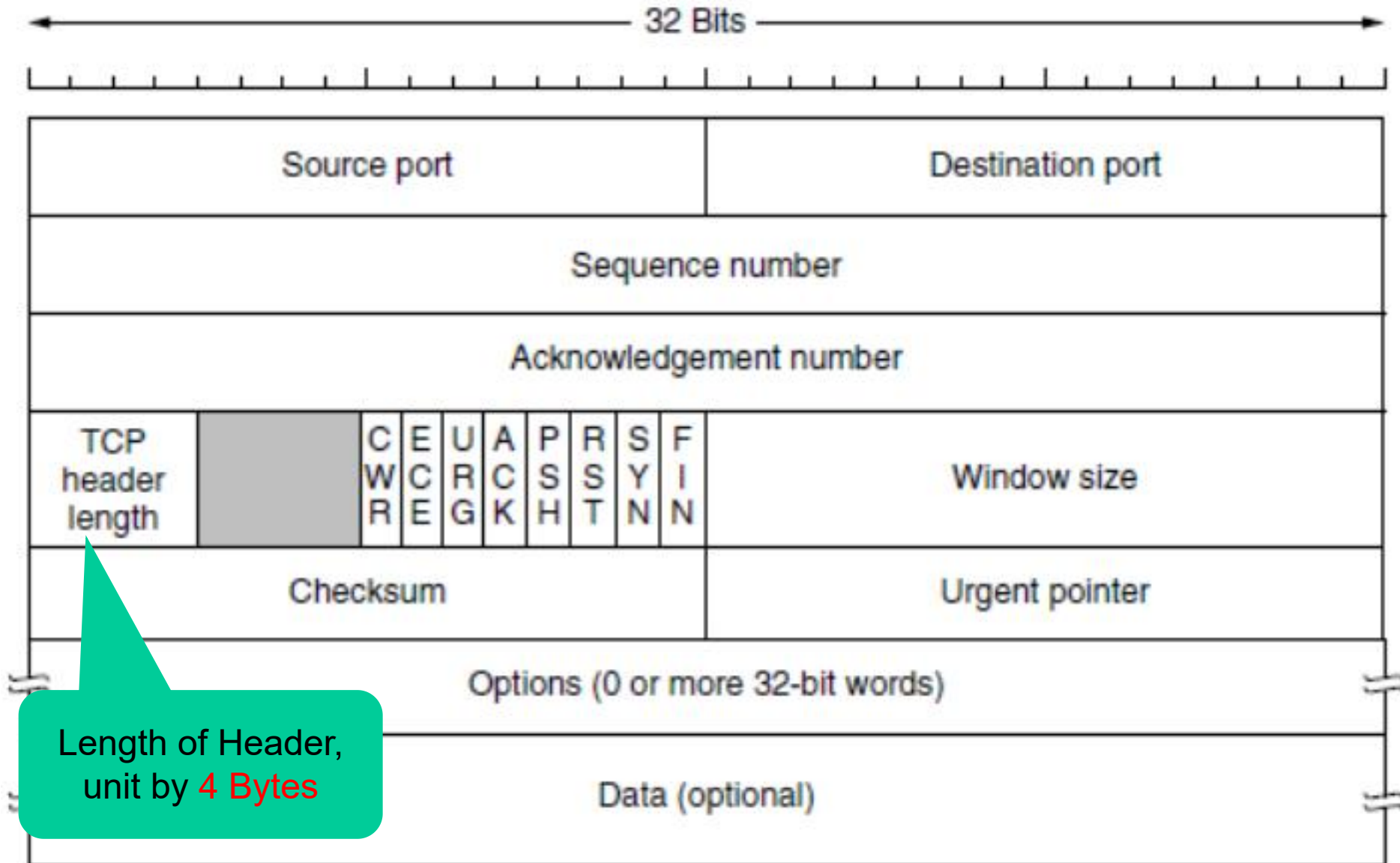
Window size and ack. No.

eg: 某接收端发送了一个段:

Acknowledgement number=701 Window size=1000

Means: 701号之前的字节已收到发送方还可发送1000字节(701-1700)的数据

TCP Segment Header

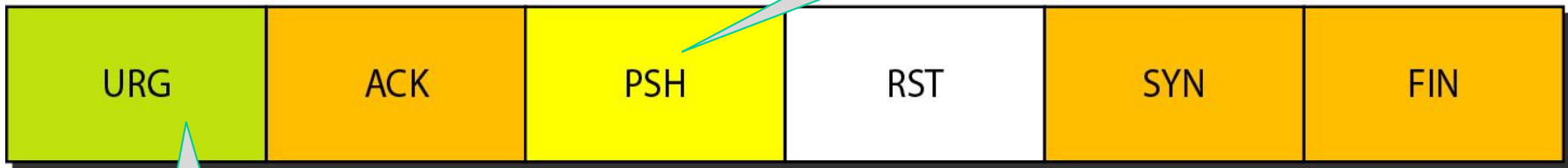


Flags Field

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection

Not to delay the transmission or delivery, eg. Interactive game

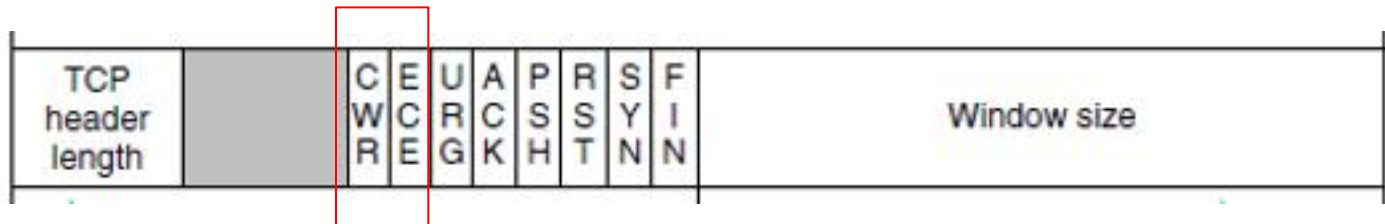
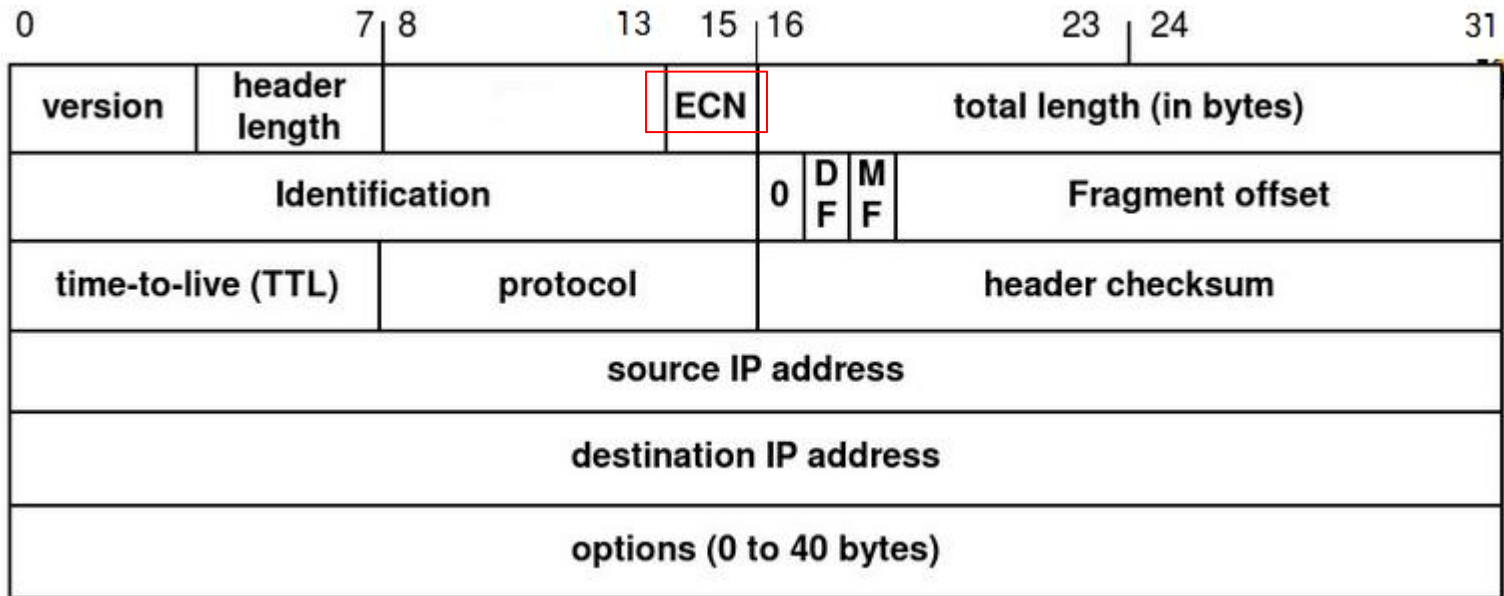


To be treated first and delivered to application layer ASAP, eg. Interrupt signal

Flag	Description
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection. → Reject
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

注：仅当ACK=1时，window size才有效

ECN and ECE/CWR



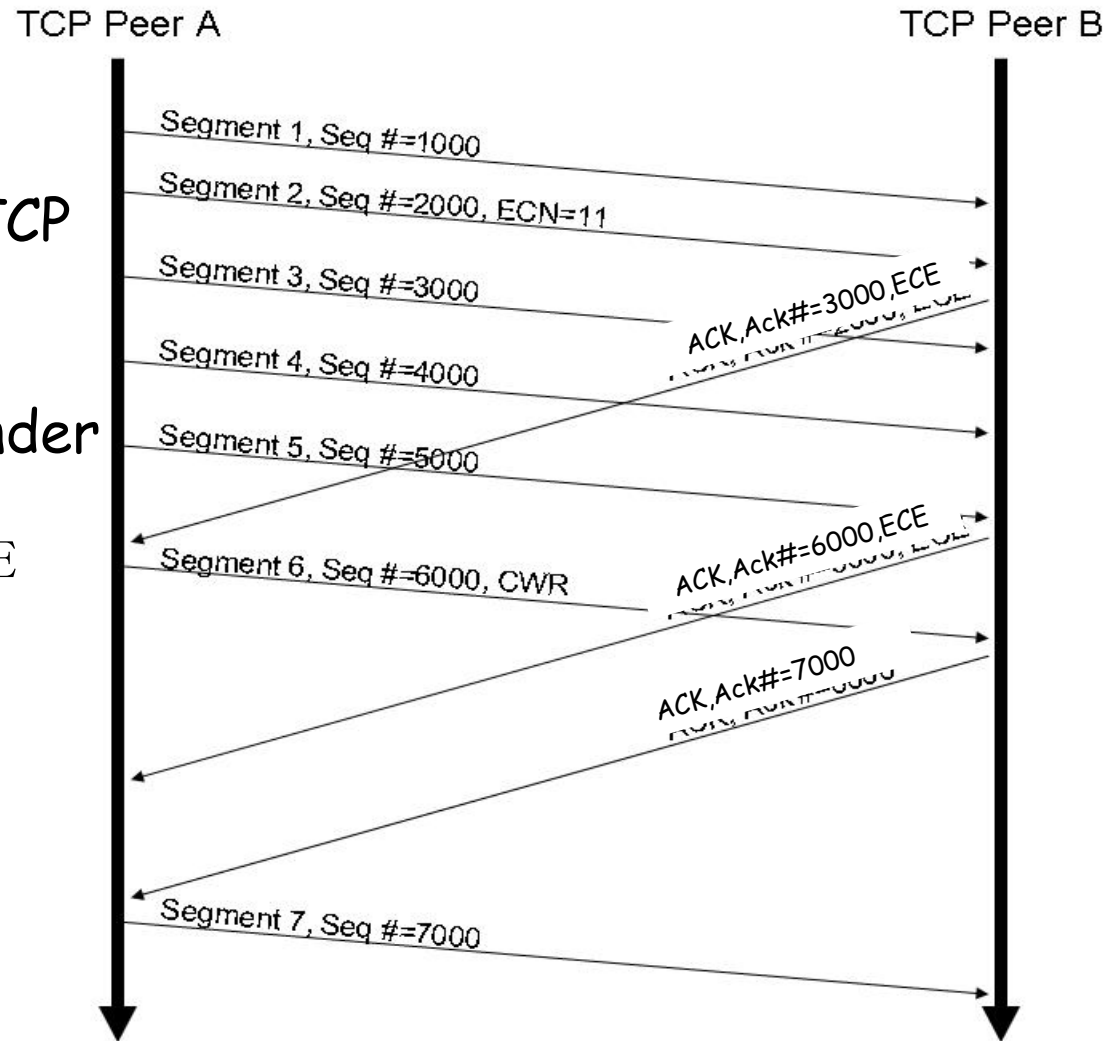
Example of using ECE and CWR

■ RFC 3168

ECN is set by router;
ECE(**ECN-E**cho) is set by TCP receiver
CWR(**C**ongestion **W**indow **R**educed) is set by TCP sender

收到CWR之后，接收主机清除ECE

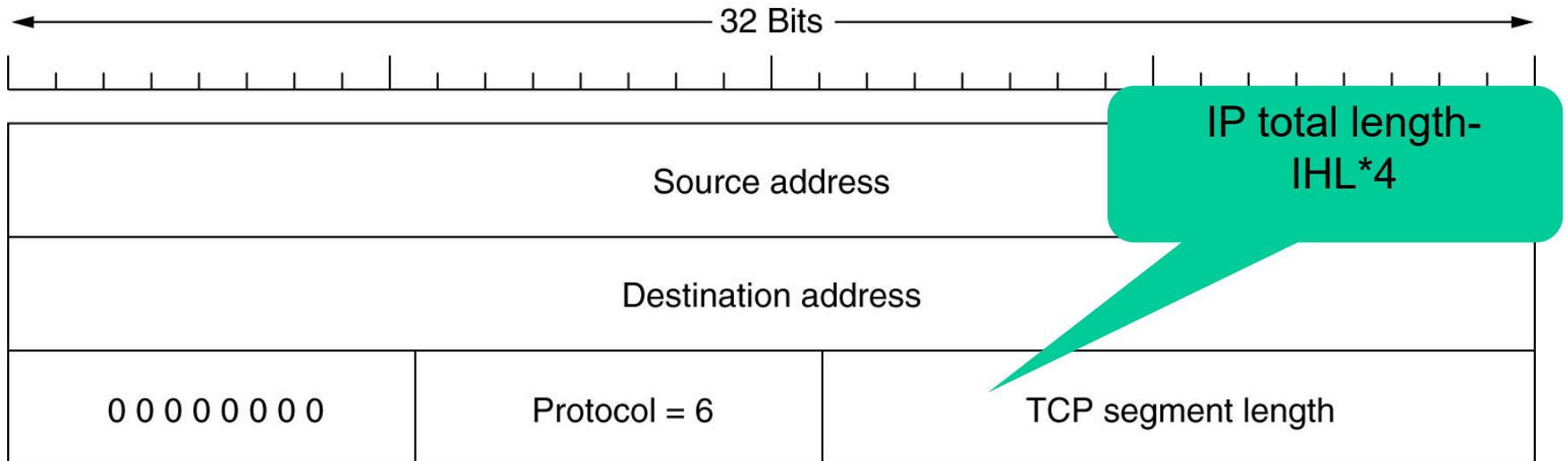
显式拥塞通知



The TCP checksum

■ TCP Checksum

- ◆ Provides **extra reliability**
- ◆ It checksums the **header**, the **data**, and the conceptual **pseudo-header**
- ◆ Pseudo-header: helps detect mis-delivered packets
- ◆ Checksum is mandatory(**TCP中的校验和是必须支持的**)



The **pseudoheader** included in the TCP checksum

Checksum summary

- IP、UDP和TCP使用相同的校验码算法(checksum, 16bit)
- IPv4: 只对IP包头校验
- IPv6: 无校验码
- IP之上的协议采用伪报头 (Pseudoheader)
 - ◆ Pseudoheader includes
 - source IP
 - destination IP
 - protocol
 - segment length
 - ◆ UDP: 可选, 伪报头+UDP头+数据 进行校验
 - ◆ TCP: 必选, 伪报头+TCP头+数据 进行校验

Options in TCP Header

■ MSS(Maximum Segment Size)

- ◆ Allow hosts to specify the MSS it is willing to accept
- ◆ **Negotiated** during **connection setup**
- ◆ MSS need not be the same in the two directions
- ◆ 作用：防止IP分片、适配链路MTU
 - Ethernet: default MSS=1460B (1500-20-20)

■ Window scale

- ◆ Allows the sender and receiver to negotiate a **window scale factor(0-14)** at the start of a connection
- ◆ shift the window size field up to **14 bits to the left**, allowing the **window size up to 2^{30} bytes**
- ◆ 作用：提升高速链路效率(吞吐量)

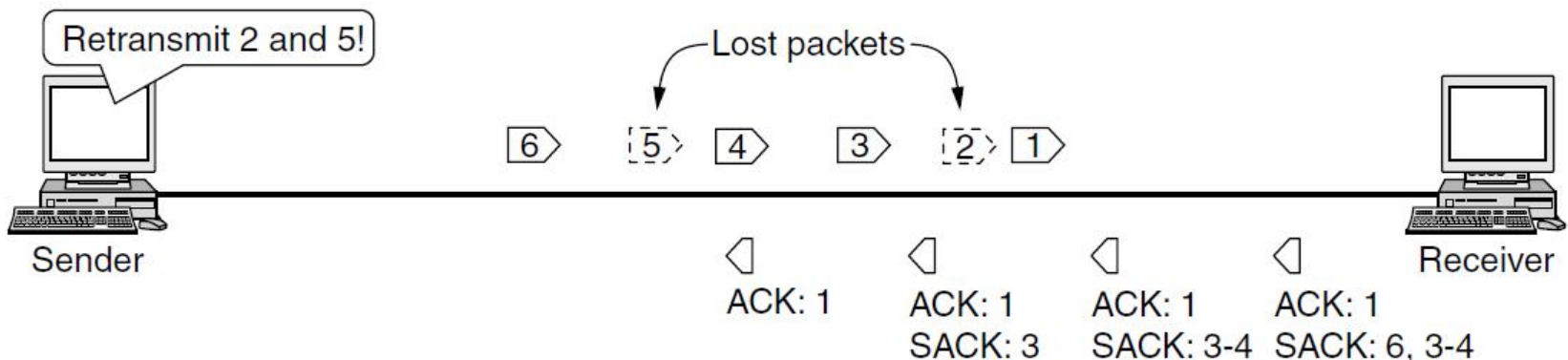
■ Timestamp

- ◆ Used to compute **round-trip time** samples
- ◆ **PAWS**(Protection Against Wrapped Sequence numbers)
 - Sequence numbers + Timestamp

TCP option: SACK

■ SACK (Selective ACK)

- ◆ Allow a receiver to tell a sender the **ranges** of sequence numbers that it has received.
- ◆ It **supplements the Acknowledgement number** and is used after a packet has been lost but the subsequence(or duplicate) data has arrived
- ◆ With SACK, the sender is **explicitly aware** of what data the receiver has and hence determine what data should be retransmitted
- ◆ Defined in RFC 2108 and RFC 2883



6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

TCP Connection Establishment

■ Client

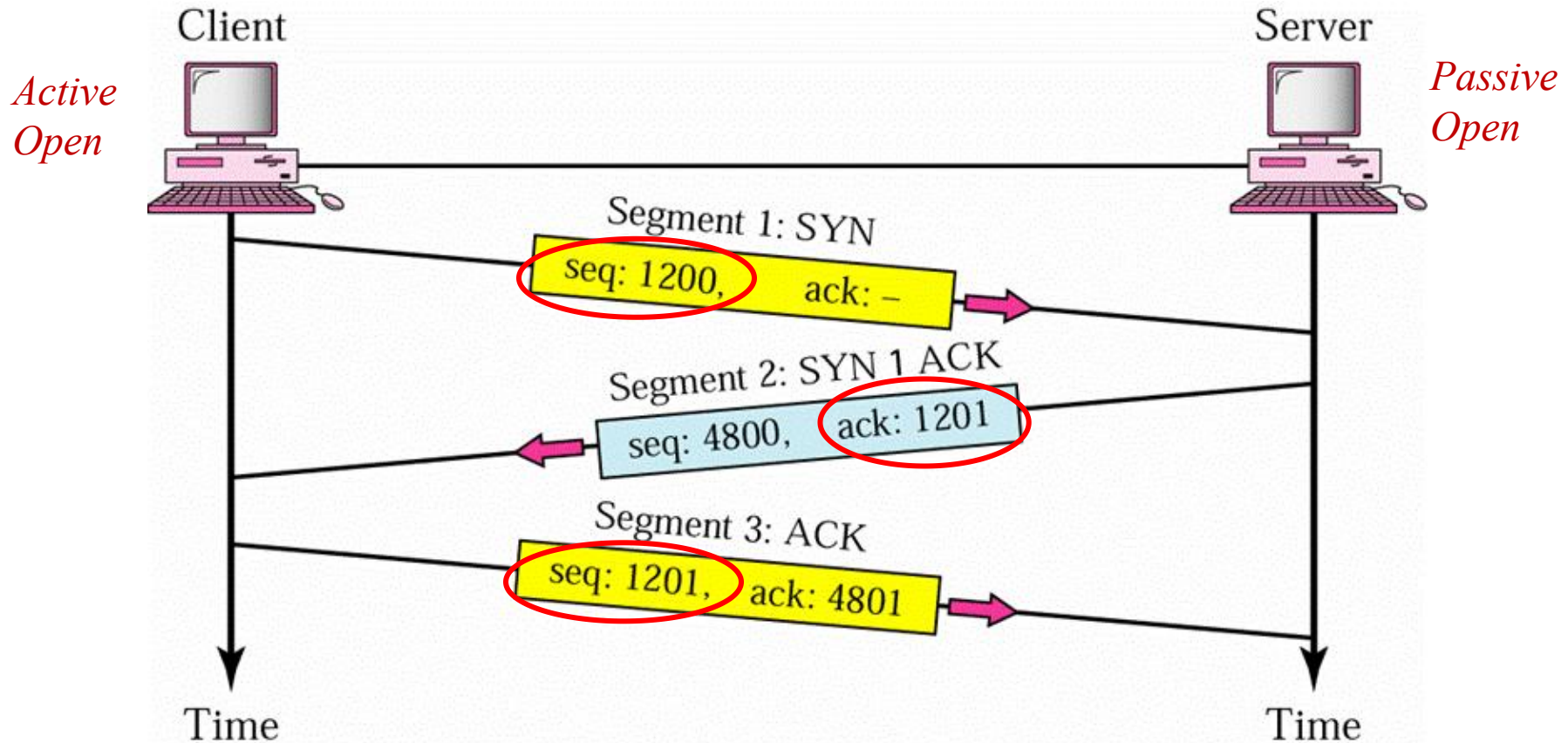
- ◆ Executes a **CONNECT** primitive, specifying the **IP address** and **port**
- ◆ sends a TCP segment with the **SYN bit on (SYN=1)** and **ACK bit off (ACK=0)** and waits for a response

■ Server

- ◆ When client's SYN segment arrives, checks to see if there is a process that has done a **LISTEN** on the port
 - If not, it sends a reply with the **RST bit** on to reject the connection
 - If some process is listening to the port, that process is given the incoming TCP segment,

3-step Connection Establishment

Agreed on **starting sequence number** for each side and solved the problem of **delayed duplicate**



A **SYN** segment cannot carry data, but consuming **1** sequence number.

A **SYN+ACK** segment cannot carry data, but consuming **1** sequence number.

Initial sequence number

■ Initial sequence number

- ◆ A clock-based scheme is used, with a clock tick every 4 μ sec

■ Security (不要求)

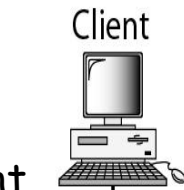
- ◆ A vulnerability with three-way handshakes is that the listening process must remember its sequence number as soon it responds with its own SYN segment---SYN flooding attack
- ◆ SYN Cookie—one way to defend against this attack

6.5 TCP

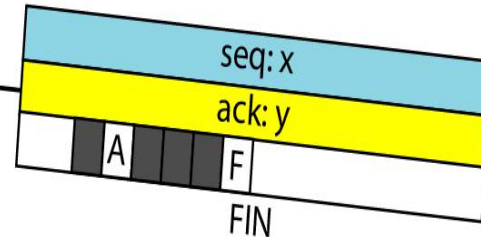
- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

Connection Termination: 3-way handshaking

Receiving **close command** from Application, Client TCP, sends a **FIN** segment to initiate active close.



Active close

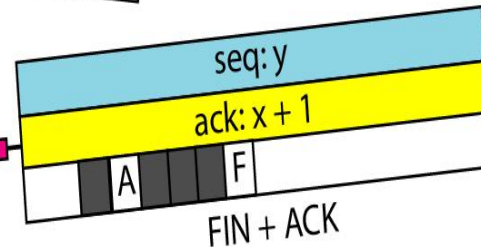


A: ACK flag
F: FIN flag

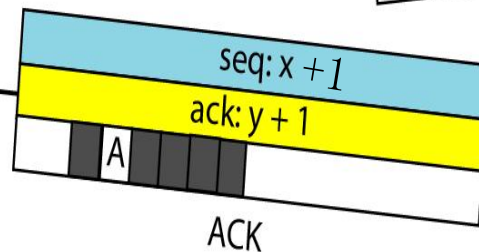


The Server TCP informs its process and sends back a **FIN + ACK**, to confirm previous **FIN** and announce closing of its return path

Passive close



The Client TCP sends last **ACK** segment, to confirm the receipt of the FIN from server

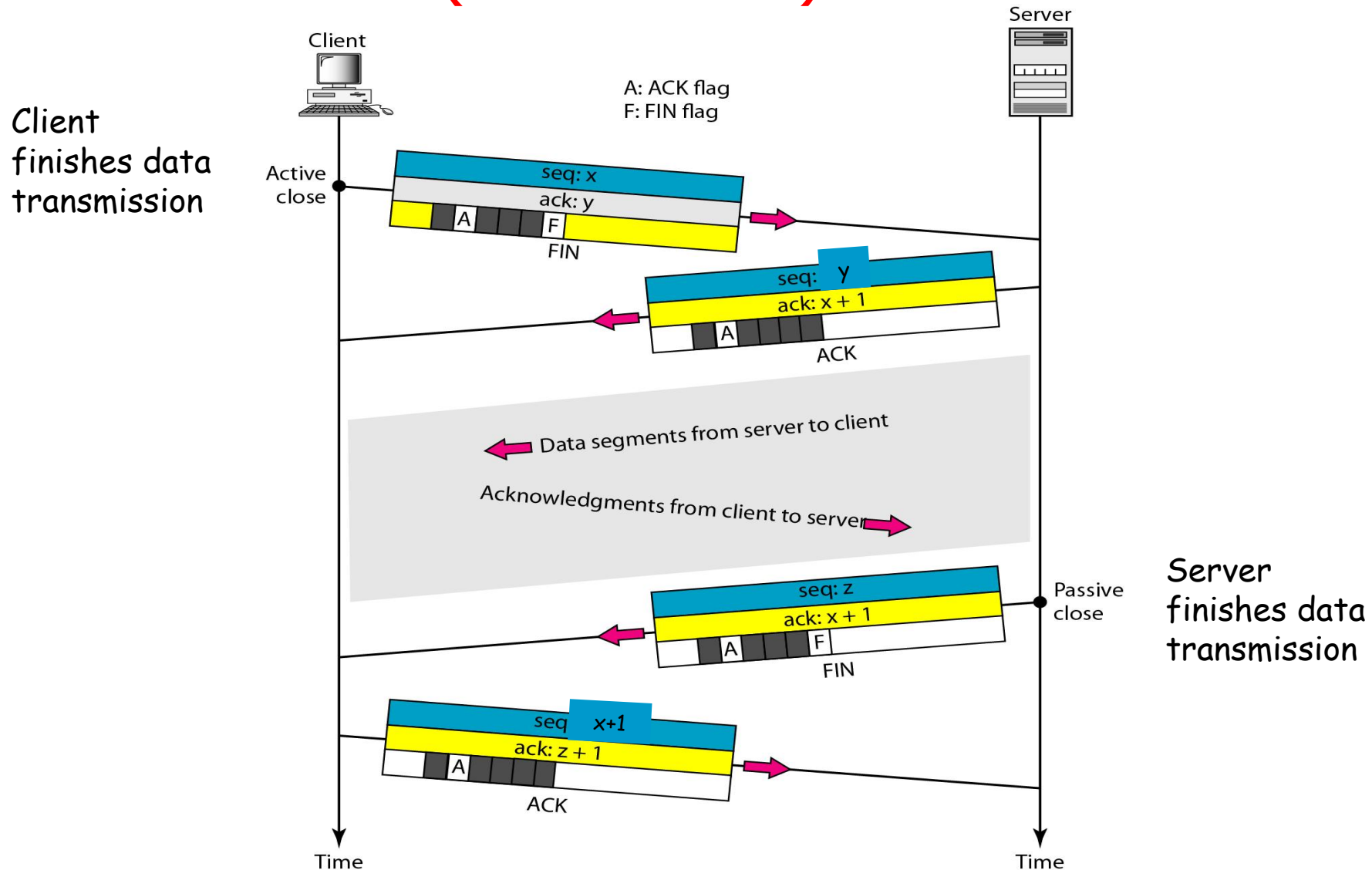


Time

Time

A **FIN**, **FIN+ACK** segment cannot carry data, but consuming **1** sequence number.

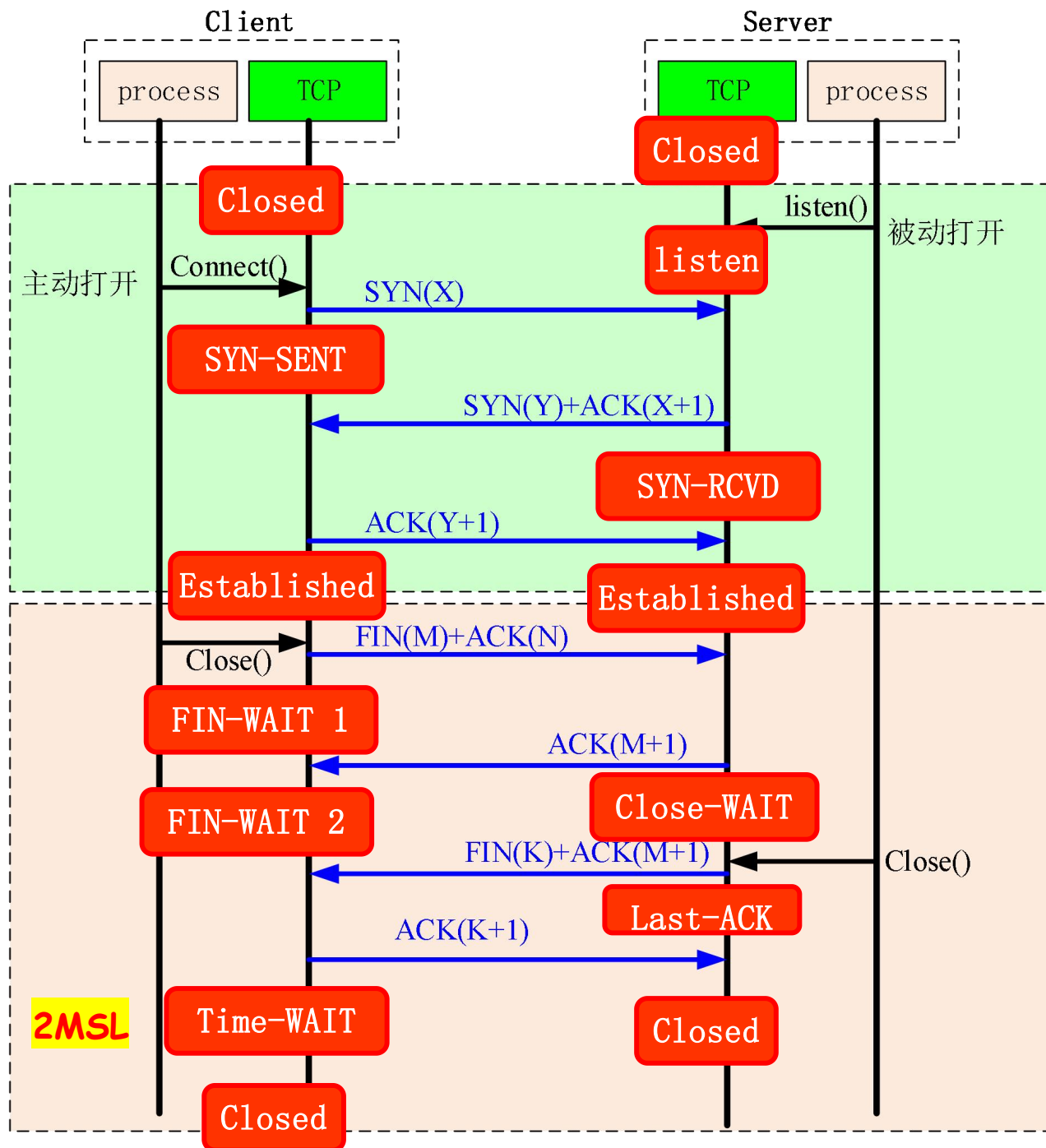
4-step Connection Release: Half-close(半关闭连接)



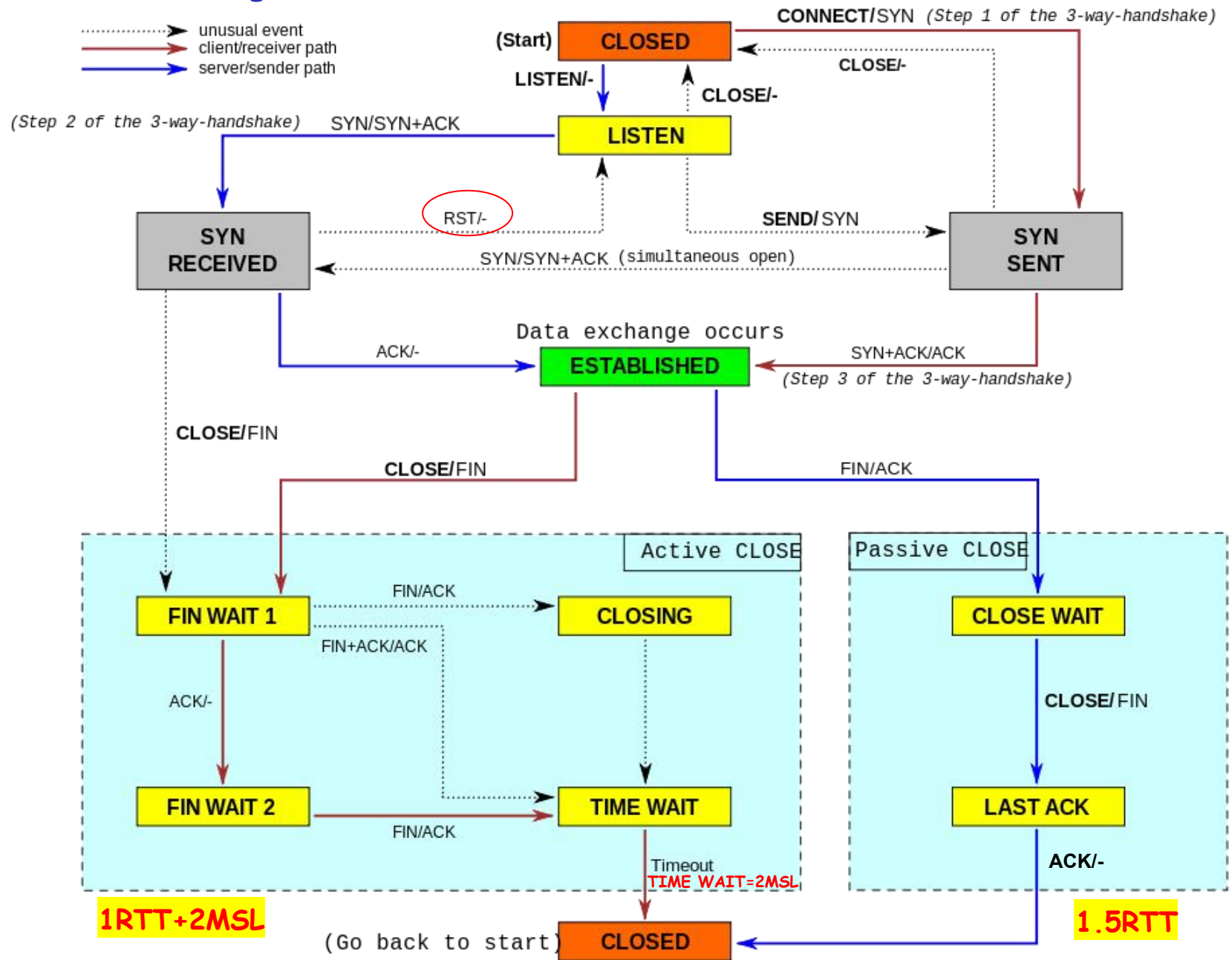
A **FIN** segment cannot carry data, but consuming **1** sequence number.

6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control



TCP connection management finite state machine.



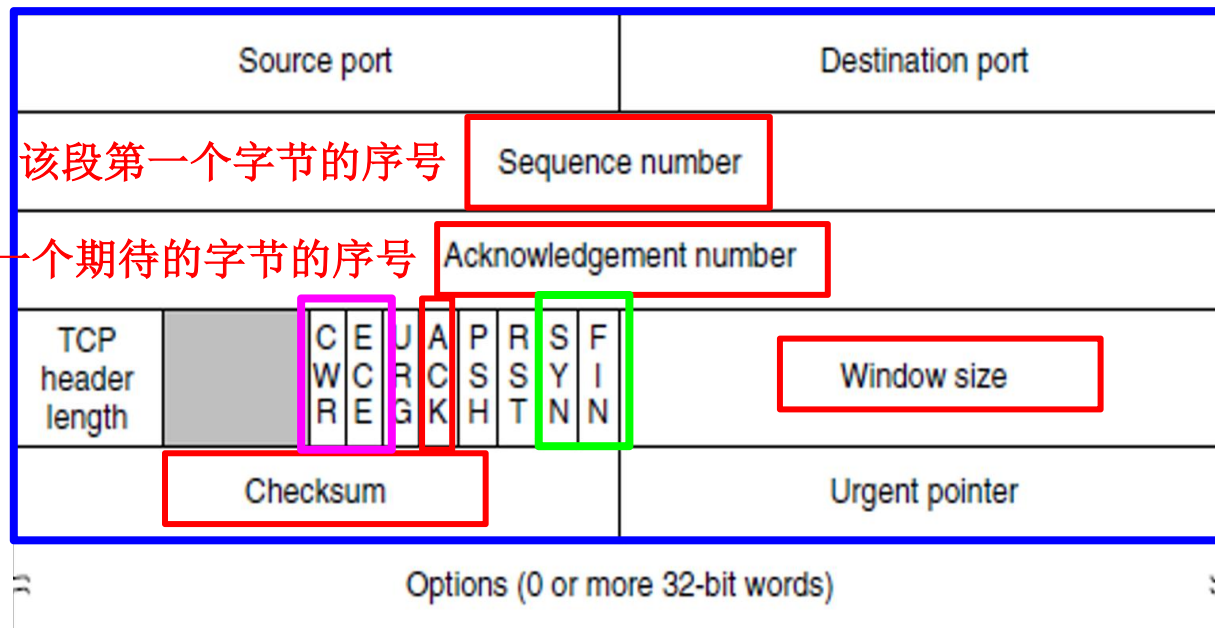
TCP Review (1)

■ Services: connection-oriented reliable end-to-end byte stream

■ TCP Segment

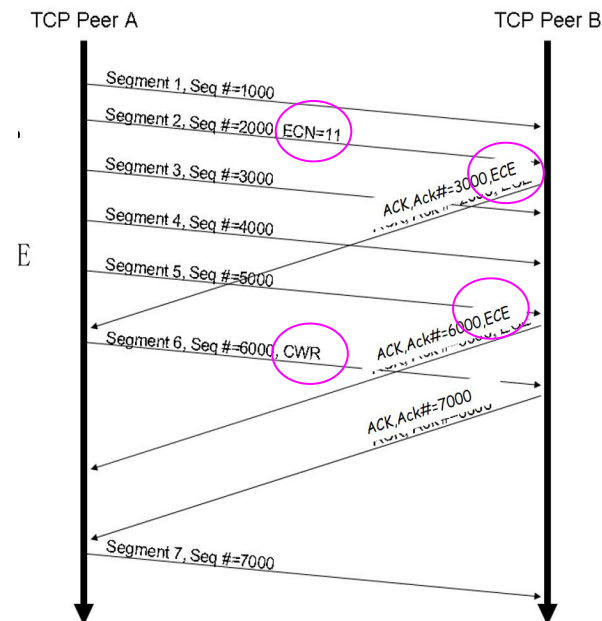
- ◆ Header: 20 or more bytes head
- ◆ Data: n bytes (segments without any data are legal)
- ◆ MSS: Maximum data(payload) length in a segment

■ TCP Segment Header



◆ Checksum (psaudoheader+header+data)

Explicit Congestion Notification



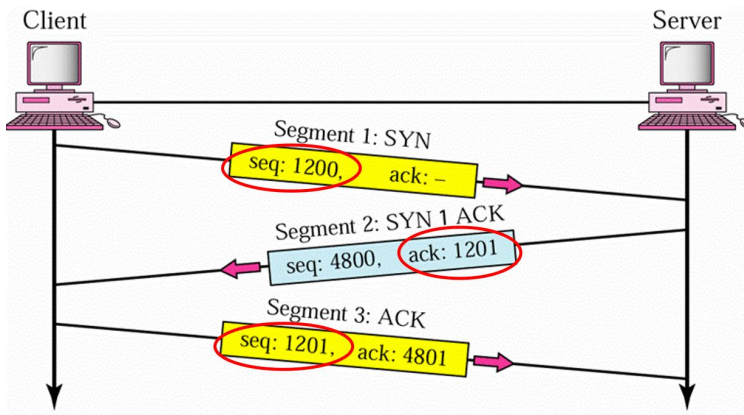
TCP Review(2)

Options in TCP Segment Header

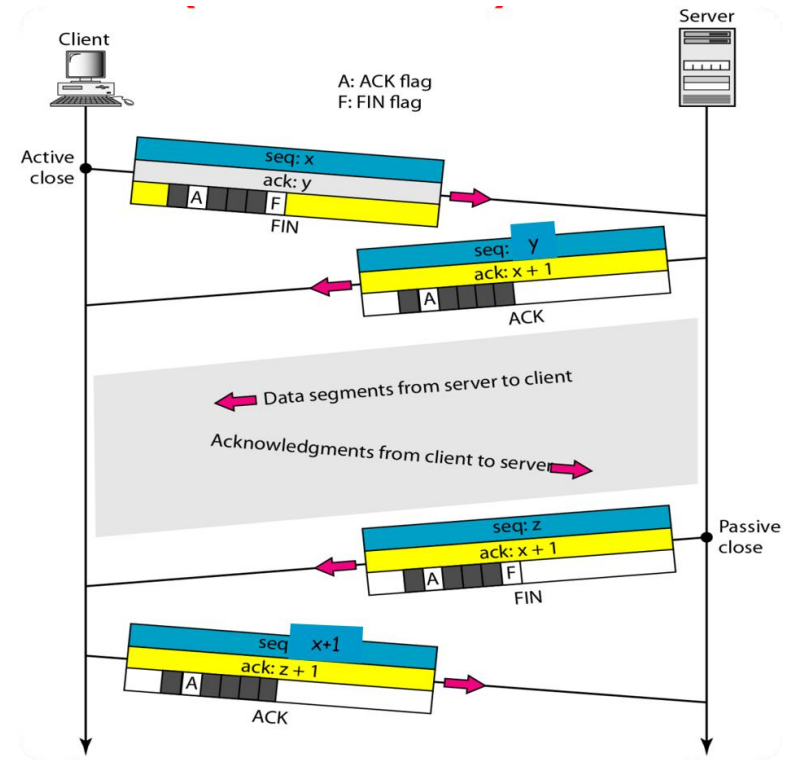
- ◆ **MSS**: 避免IP分片
- ◆ **Window scale**: shift the window size field up to 14 bits to the left, allowing the window size up to 230 bytes(提升吞吐量)
- ◆ **SACK**: ranges of sequence numbers that the receiver has received

Connection Establishment

Connection Release



SYN, SYN+ACK, ACK



FIN, ACK, FIN, ACK

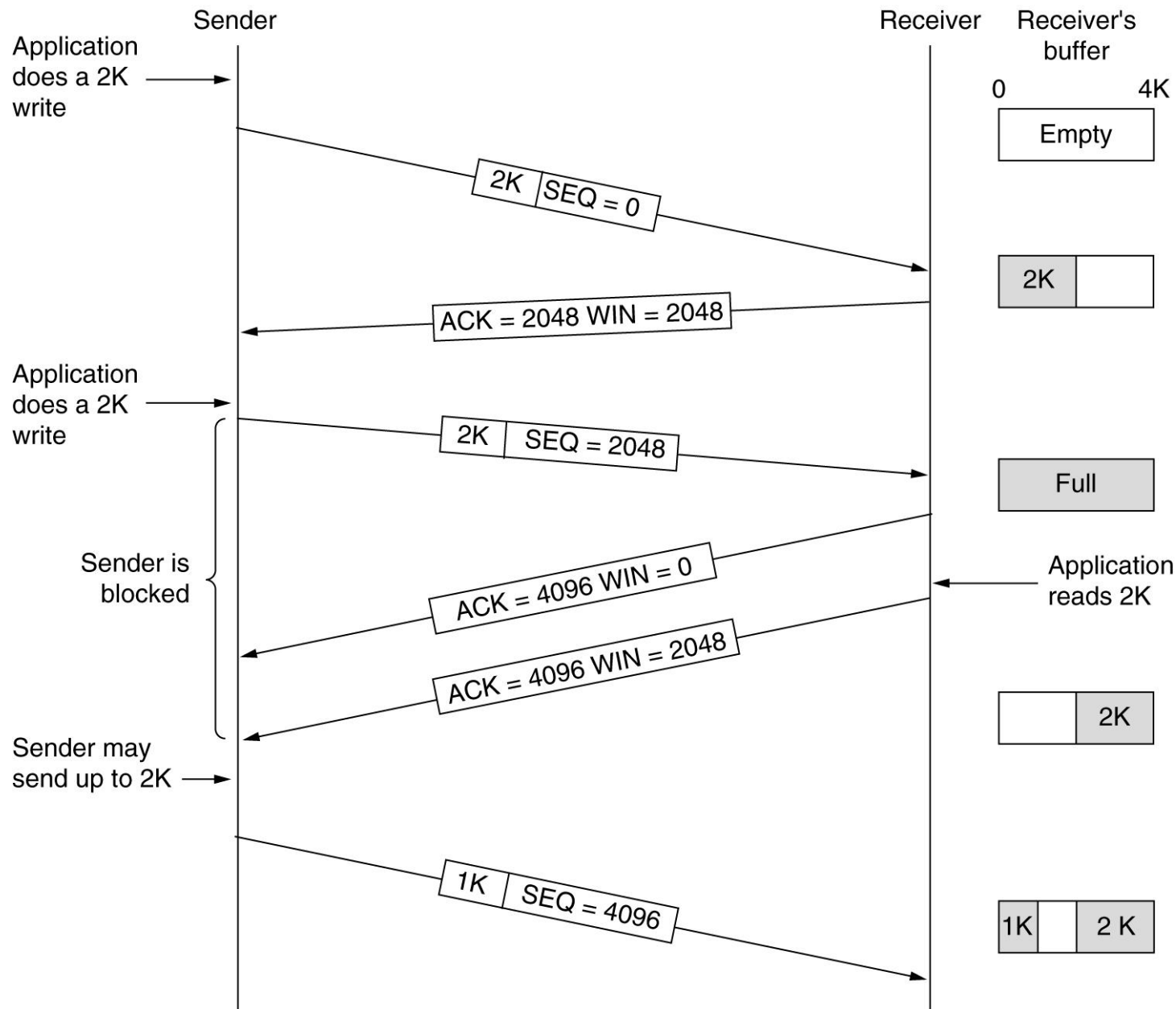
6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

(1)Flow Control: Sliding Window

- Acknowledgement and buffer allocation are separated 
- Differences with Data Link Layer Flow Control:
 - SW in TCP is **byte oriented**, but SW in DL is **frame oriented**
 - TCP SW is **variable size** (acknowledgement number + Window Size) , but DL SW is **fixed size**
- When the window announced by receiver is 0
 - Sender may not normally send segments
 - **Urgent data** may be sent
 - **Window probe**: Sender may re-send a 1-byte segment to probe ACK & window update to **prevent deadlock**

Window management in TCP



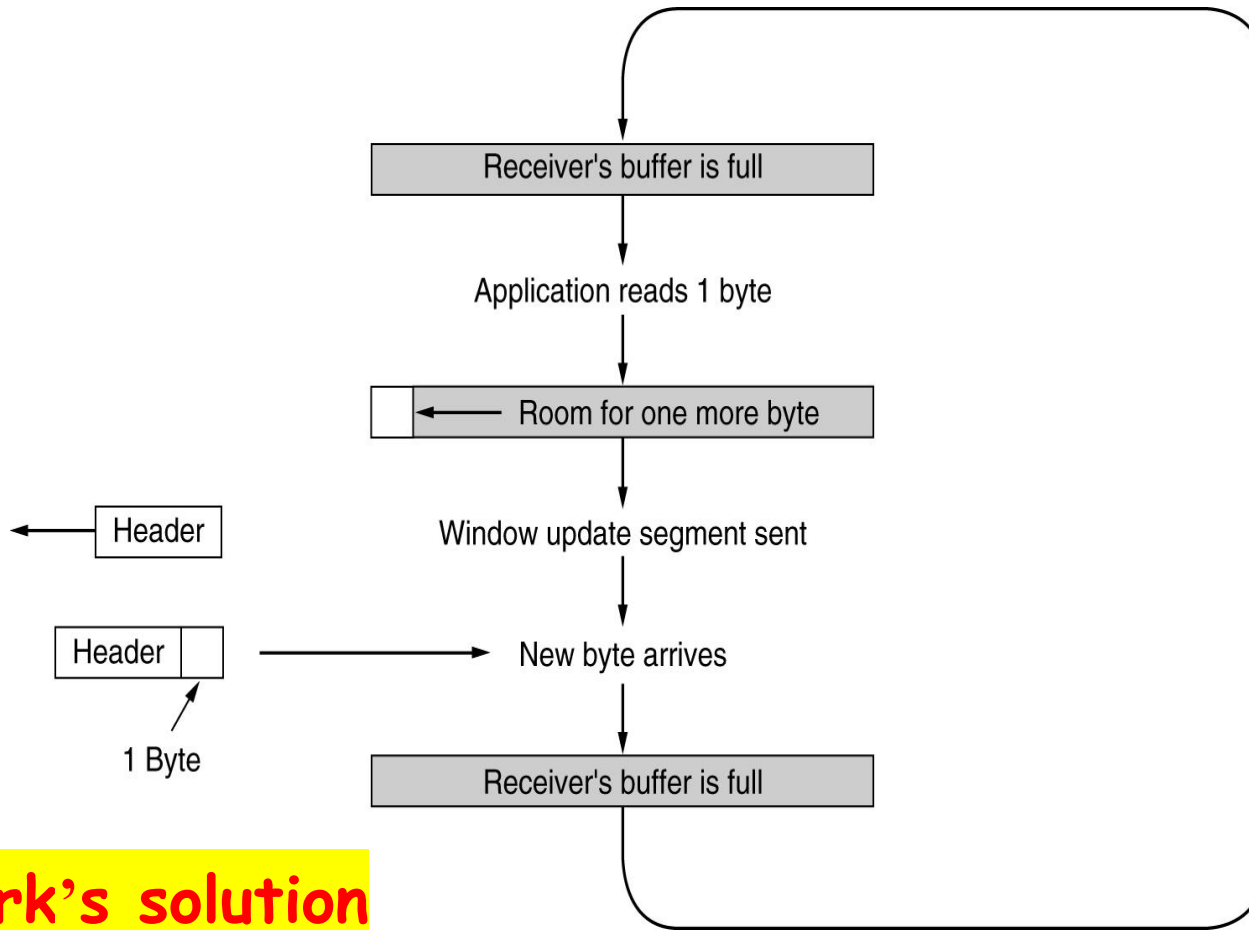
(2)Features of TCP's Sliding Windows

- **Delayed sending:** senders are not required to transmit data as soon as they come in from the application.
- **Dynamic Window Size**
 - ◆ The sender window size is totally **controlled by the receiver window value**. However, the actual window size can be smaller if there is congestion in the network.
- The receiver can send an acknowledgment at any time.
 - ◆ cumulative acknowledgment
 - ◆ **SACK**
 - ◆ **delayed acknowledgment (delay 500ms)**

(3)Nagle's algorithm

- **Problem:** **Small packets problem** (Small packets are wasteful)
 - ◆ Overhead: TCP head + IP head=20+20 bytes
 - ◆ When used in TELNET connection, 1 byte data carried with 40 bytes overhead
- **Improvement:** **Sending larger segments(避免发小包)**
 - ◆ Sender: **Nagle algorithm**
 - Send the first piece and buffer all the rest
 - Do not send until either of the following conditions exist:
 - ▣ Acknowledgement to last segment has been received
 - ▣ Buffered data fills half the window or a maximum segment(MSS) eg.1460Byte
 - ◆ Receiver: Delay acknowledgement and window update, reduce the load placed by the receiver.
- **TCP Option**
 - ◆ TCP_NODELAY: Disable Nagle's algorithm & using PUSH bit
 - ◆ Linux TCP_CORK: Disable Nagle's algorithm, wait 200ms to accumulate data for sending

Problem 2: Silly Window Syndrome



■ Clark's solution

The **receiver** should not send a **window update** until it can handle the **MSS** or its **buffer is half empty**, whichever is smaller

小结：TCP提升传输效率

- TCP发送小包会降低网络的传输效率
- 场景1：发送应用进程1次发送一个字节数据
- 解决方案：Nagle's Algorithm发送端避免发送小包
 - ◆ TCP连接上最多只有一个未被确认的小包，TCP发送方缓存后续的数据直到收到TCP确认或者积累了足够的数据(MSS)
- 场景2：接收应用进程1次接收处理一个字节数据(低能/糊涂窗口综合症)
- 解决方案：Clark's Algorithm禁止接收端TCP发送只有一个字节的窗口更新段
 - ◆ 接收端TCP有一定数量缓存空间之后再通告给对方(can handle MSS or buffer is half empty)
- 两种方法配合使用

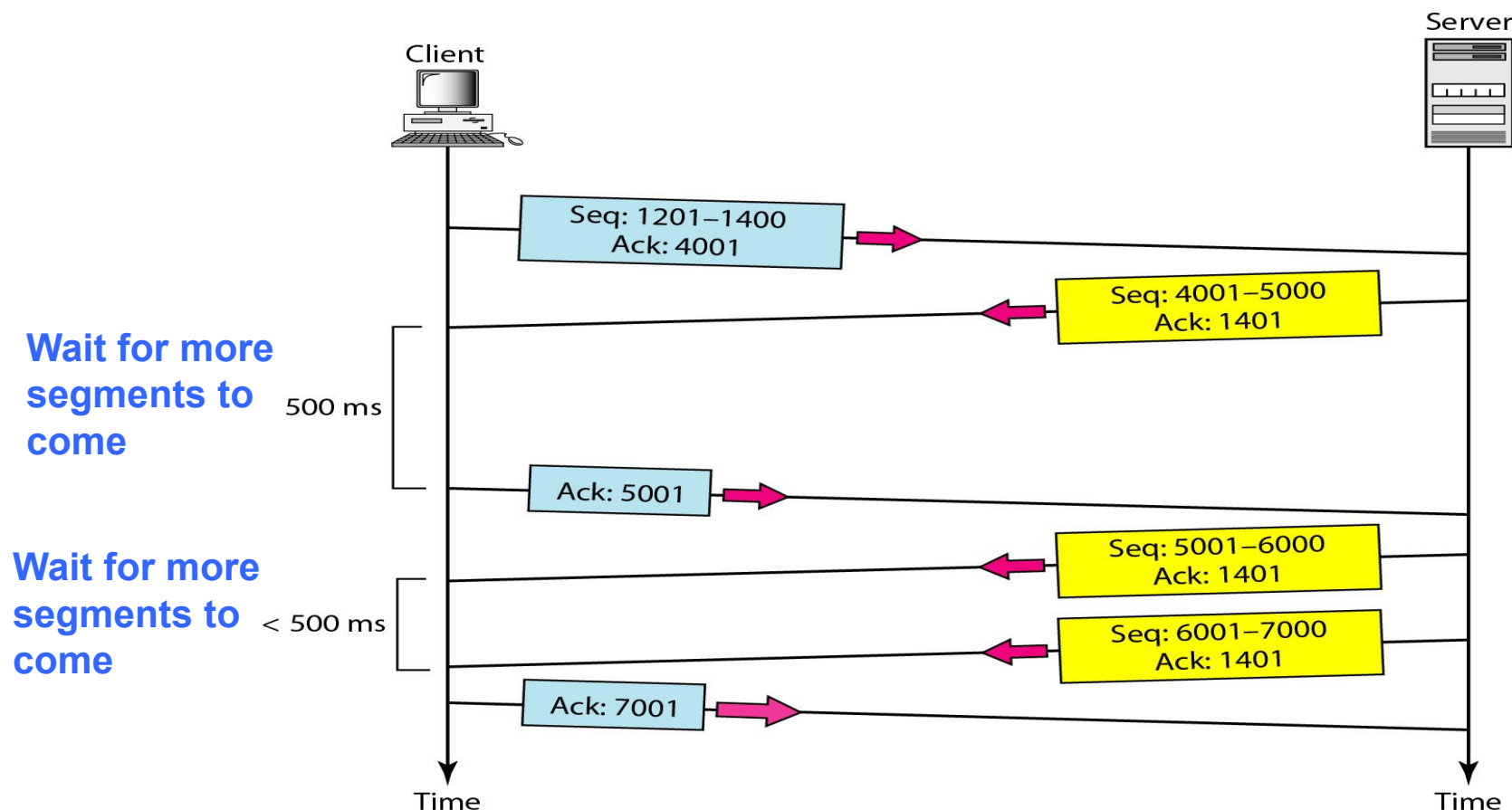
6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (*補*) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

TCP Error Control

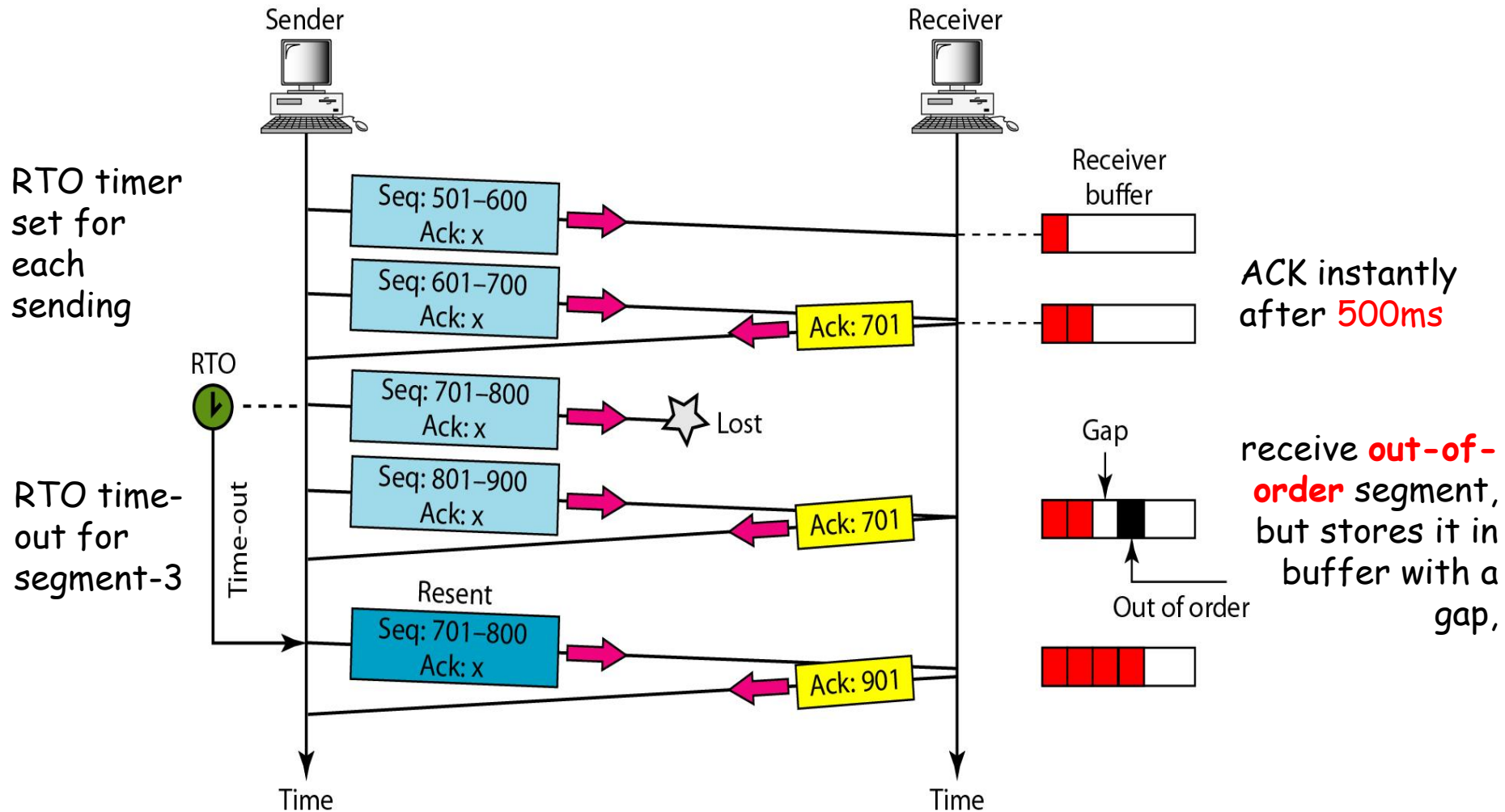
- TCP provides **reliable transmission**
 - ◆ Error control includes mechanism for detecting corrupted segments, lost segments, out-of-order segments and duplicated segments.
- Error checking and correction using **ARQ**
 - ◆ **Checksum**: to check for corrupted segment and discard it.
 - ◆ **Acknowledgement (ACK)**: to confirm the receipt of segments.
 - ◆ **Time-out** : Timer set for retransmission of segments, referred as RTO(Retransmission Time-Out) timer.
- The heart of error control mechanism is **Retransmission**.
- No retransmission and timer set for an ACK segment

Normal Operation

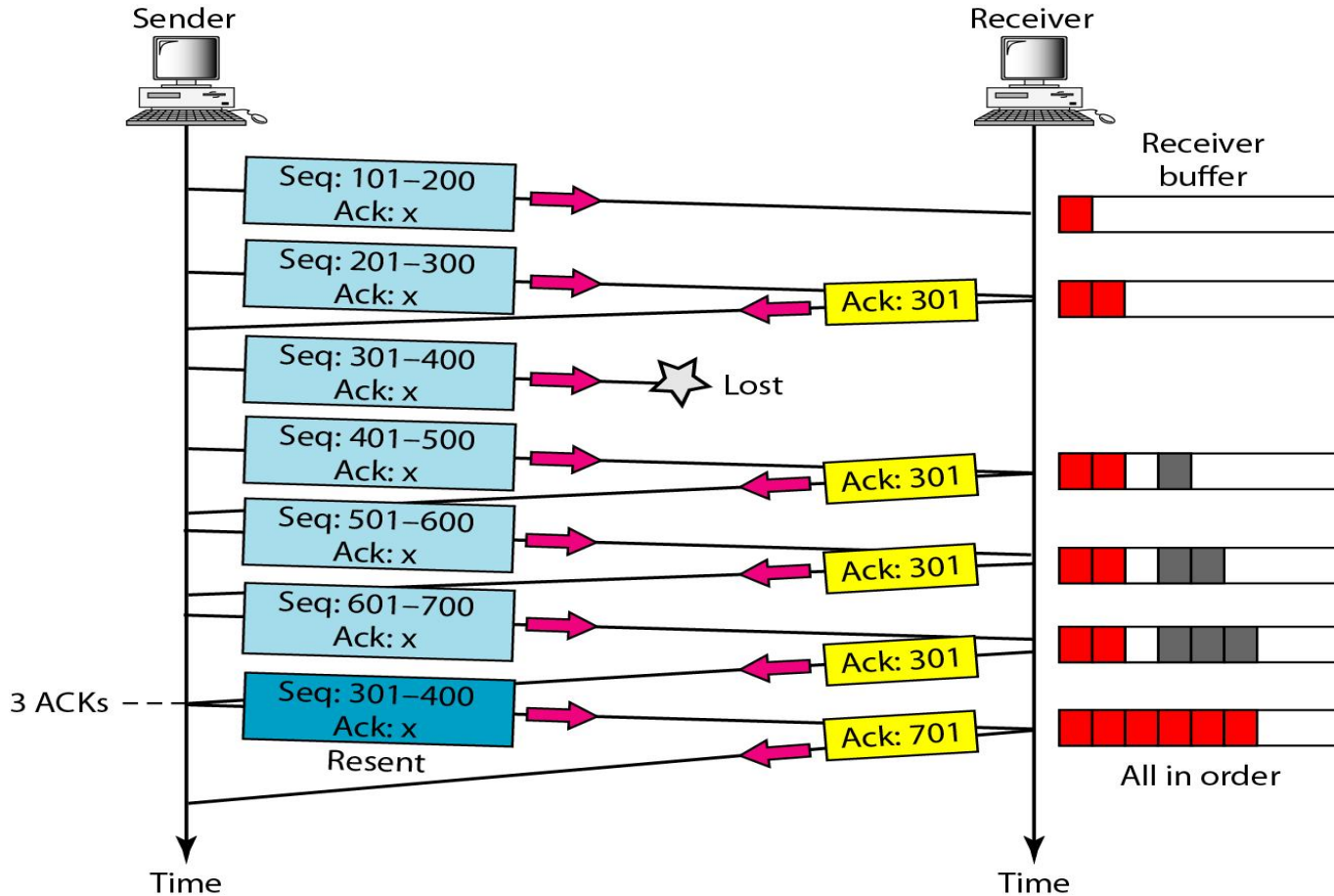


A slight delay waiting of **500 ms** before sending off the ACK, to see any more segments arrive so that ACK can cover for this range of time(**Cumulative ACK**). (e.g. ACK: 6001 is skipped).

Lost Segment



Fast Retransmission(快速重传)

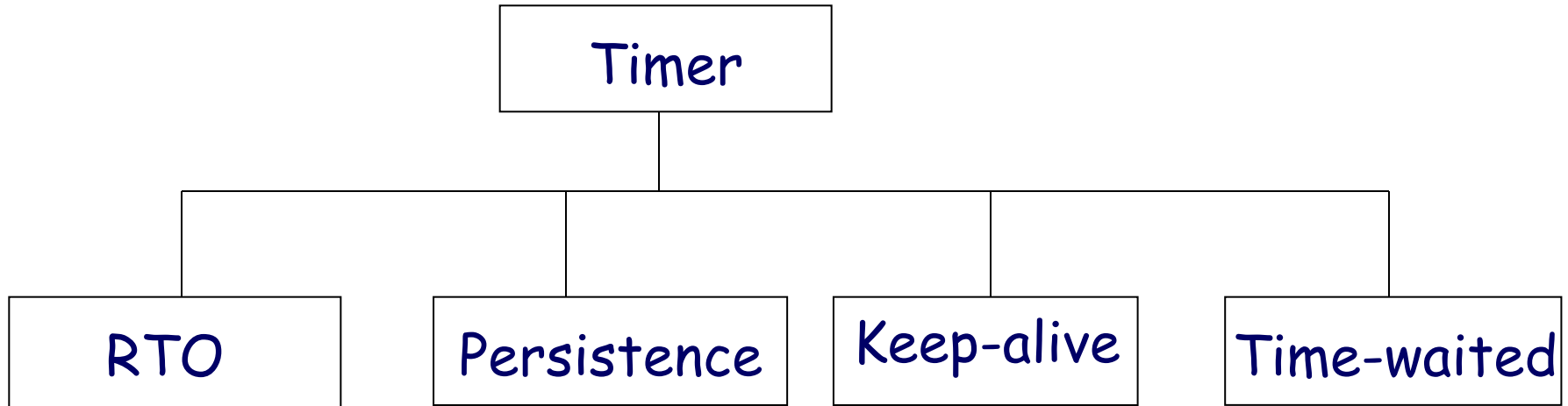


FAST re-transmission: When 3 duplicate ACKs are returned, the sender resends the lost segments immediately without waiting for RTO.

6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

TCP Timer Management



TCP Timers

■ Persistence(持续/坚持) timer

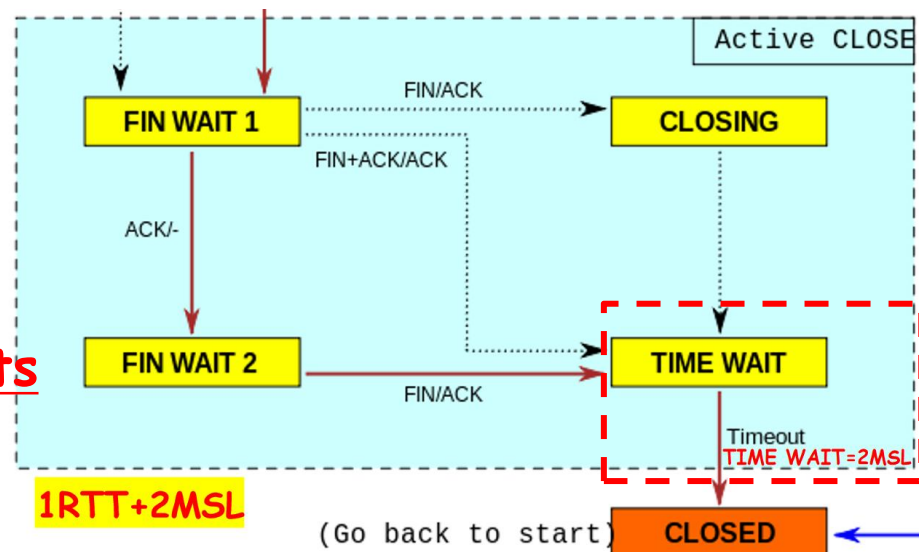
- ◆ It is designed to prevent the 0-window deadlock
- ◆ When **timer goes off**, the sender transmits a probe(dummy) segment to the receiver. The response to the **probe gives the window size**

■ Keepalive(保活) timer

- ◆ It is designed to prevent the half-open connection
- ◆ When a connection has been idle for a long time, the keepalive timer may go off to cause one side to check whether the other side is still there
- ◆ This feature is controversial

■ Time-waited timer

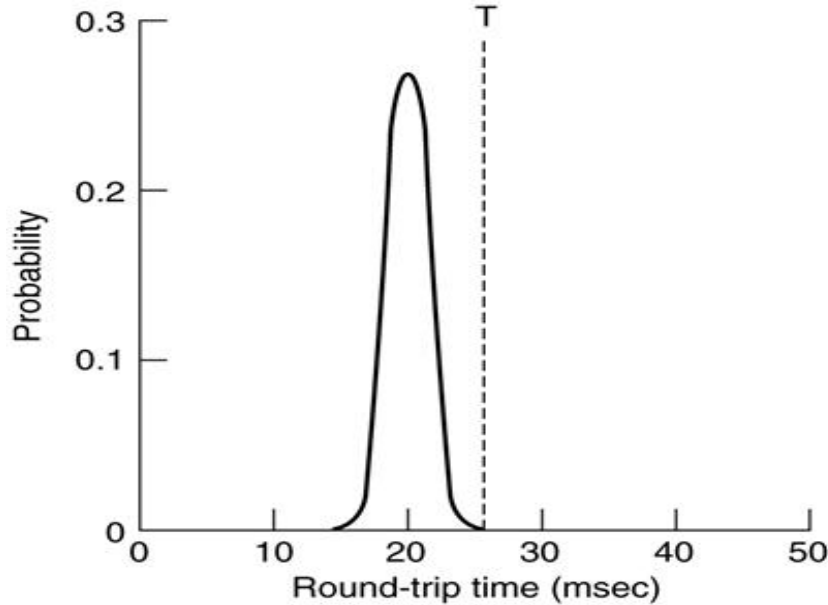
- ◆ 2MSL(Maximum Segment Lifetime)
- ◆ To make sure that when a connection is closed, all packets created by it have died off



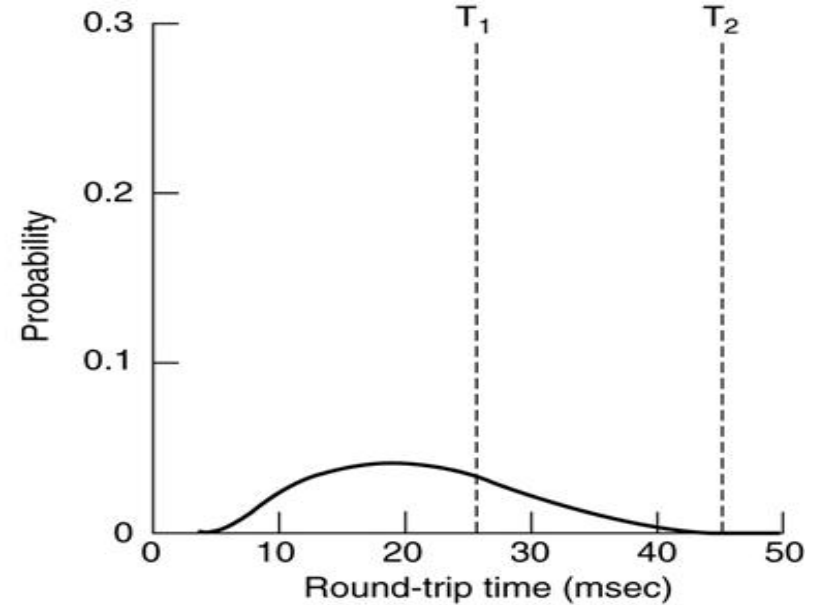
RTT Estimation: Data Link Layer vs. TCP

- How long should the timeout be?
- It is much more difficult in the TL than in DL

◆ Probability density of Round Trip Time (RTT)



(a)
Data Link Layer



(b)
TCP

- Too short (T_1): duplicate packets, which may make congestion worse
- Too long (T_2): low throughput, inefficiency

Retransmission Timer (Textbook 6.5.10)

- Solution: use a **dynamic algorithm** that constantly adapts the timeout interval, based on continuous measurements of network performance

- **Jacobson algorithm**

- ◆ **RTO**(重传计时器) is sensitive to in **RTT-variance** (**RTTVAR**) as well as **SRTT**

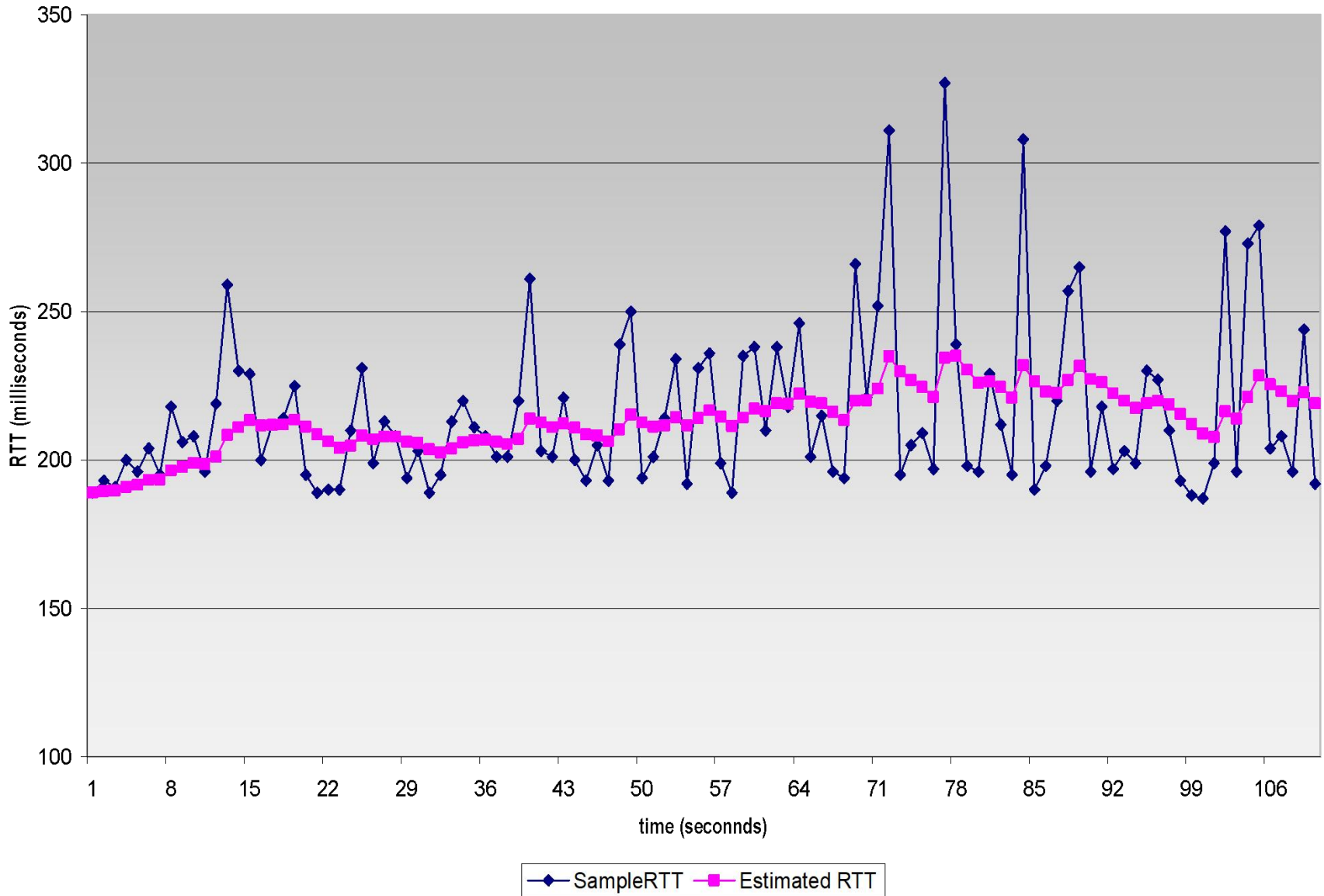
- ◆ **SRTT**: Smoothed RTT(Round-Trip Time)

EWMA Exponentially Weighted Moving Average(指数加权移动平均)

$$\text{SRTT} = \alpha \times \text{SRTT} + (1 - \alpha) \times (\text{new sample}) \quad (\text{会计算})$$

Sample: AckRcvdTime – SendPacketTime, $0 \leq \alpha < 1$, usually 7/8

Example RTT estimation:



Retransmission Timeout: RTO

- ◆ *RTTVAR: The variance in round-trip times*

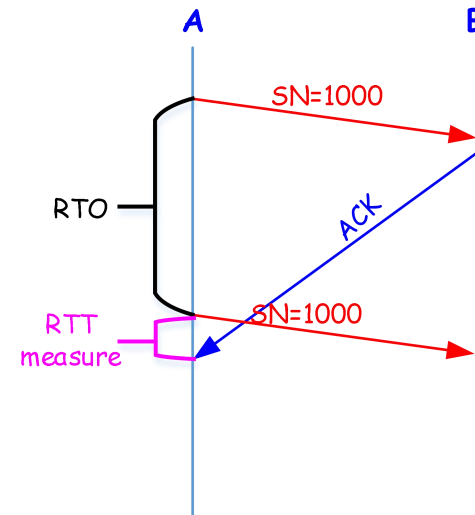
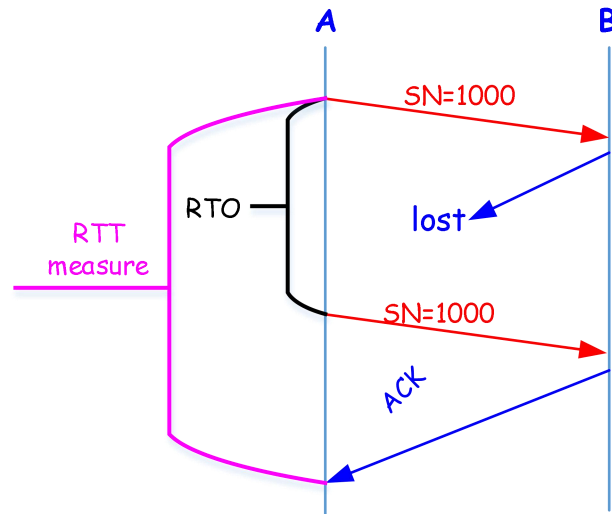
$$RTTVAR = \beta \times RTTVAR + (1-\beta) \times |SRTT-Sample|$$

$0 \leq \beta < 1$, usually $3/4$

- ◆ $RTO = SRTT + 4 \times RTTVAR$

Retransmission Timeout: RTO

- Question: For retransmission, how to calculate Sample and RTO? $Sample = AckRcvdTime - SendPacketTime$



◆ Karn's algorithm

- do not update estimates on any segments that have been retransmitted.
- RTO is doubled on each successive retransmission

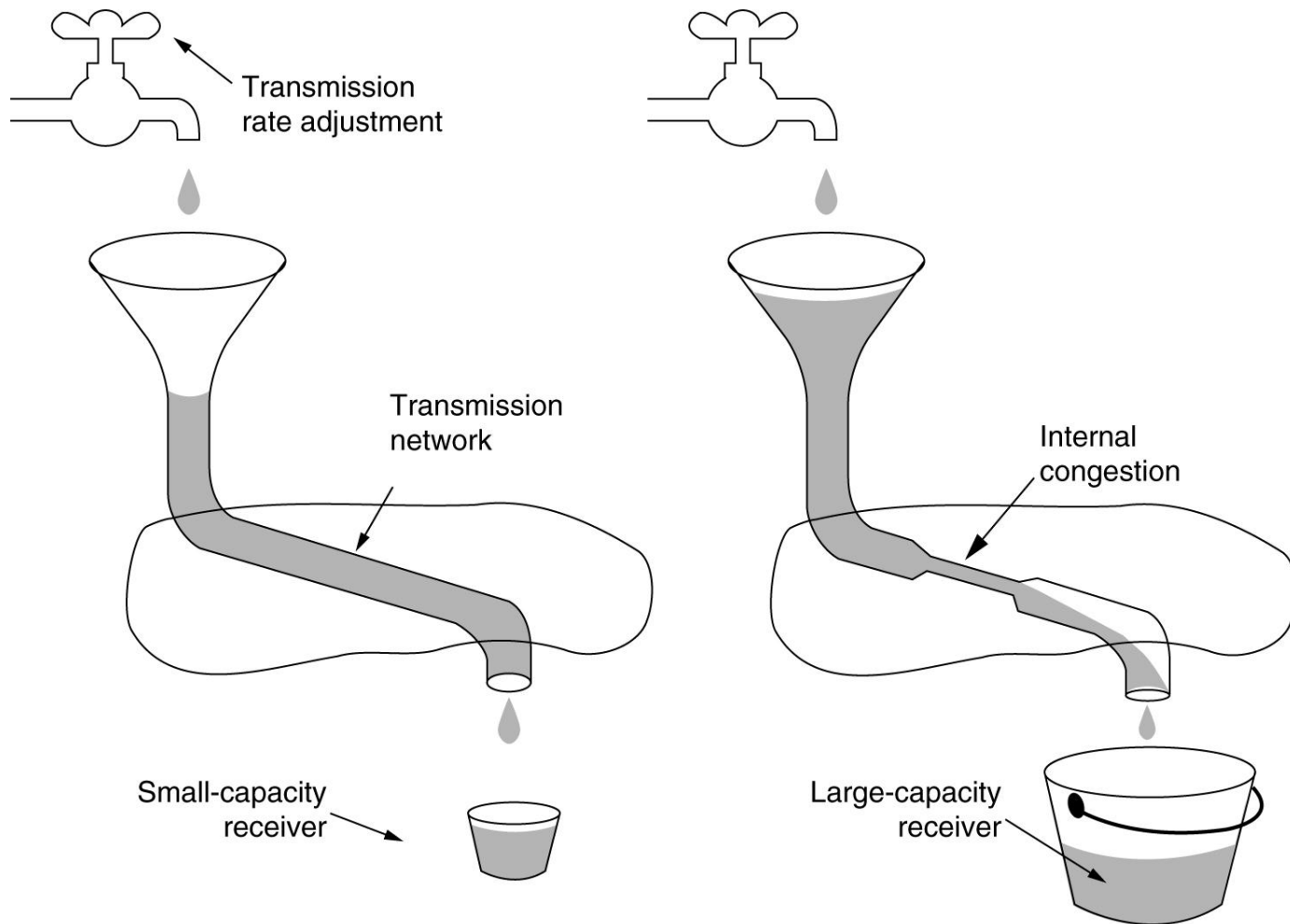
6.5 TCP

- 6.5.1-3 TCP Features
- 6.5.4 TCP Segment Format
- 6.5.5 Connection Establishment
- 6.5.6 Connection Release
- 6.5.7 Connection Modeling
- 6.5.8 Flow Control
- (補) Error Control
- 6.5.9 TCP Timer Management
- 6.5.10 Congestion Control

(1) Congestion Control in Internet

- **Router** (network layer) : detects congestion when queues grow large and tries to manage it, if only by dropping packets.
- **Source host** (transport layer, TCP): receives congestion feedback from the network layer and slow down the rate of traffic that it is sending into the network.

Network capacity and Receiver capacity



- (a) A fast network feeding a low capacity receiver
- (b) A slow network feeding a high-capacity receiver

(2) Congestion Window

- Congestion Window (*cwnd*) is a variable held by the TCP sender for each connection.
 - ◆ Denotes **how many bytes network is able to absorb**
- Van Jacobson's Solution
 - To approximate an **AIMD** congestion window
 - ◆ TCP maintains a congestion window(cwnd) and a flow control window(rwnd), both are tracked dynamically, the sender window is the smaller of them. $\min(cwnd, rwnd)$
 - ◆ Using **packet loss** as a congestion signal
 - Depends on transmission errors being relatively rare
 - This is why 802.11 includes its own retransmission mechanism
 - ◆ A good timer is needed to detect packet loss accurately
 - The timer includes estimates of the **mean and variation in RTT**

(3) Problems of Congestion Control

■ How to detect congestions

- ◆ **Packet dropping** is the best sign of congestion
- ◆ How to detect packet drops? **ACKs**
 - TCP uses ACKs to inform receipt of data
 - ACK indicates last contiguous byte received
- ◆ **No ACK after time-out implies packet drop**

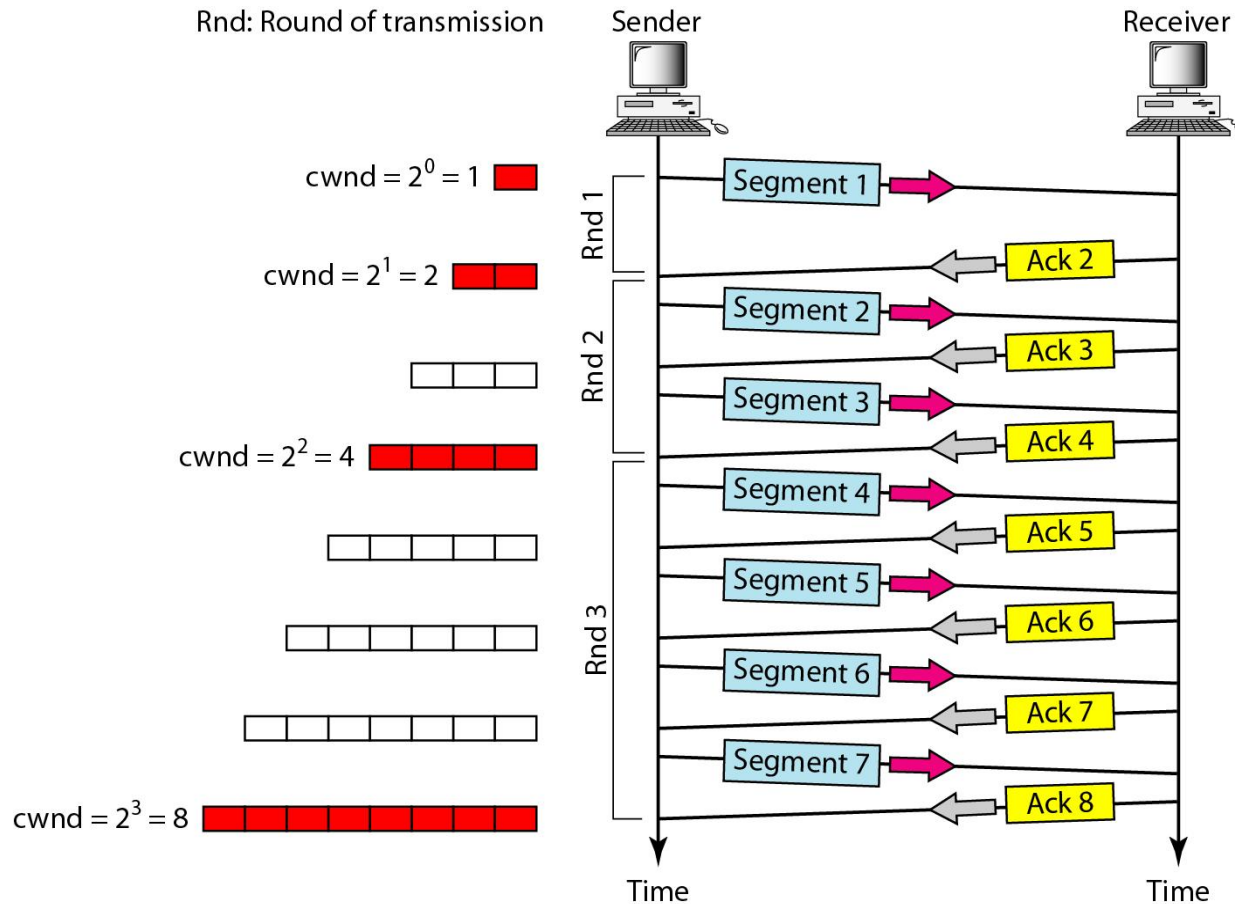
■ How to adjust sending rate (AIMD)

- ◆ Upon receipt of ACK, increasing rate
- ◆ Upon detection of packet loss, decreasing rate

TCP Congestion Control Algorithms

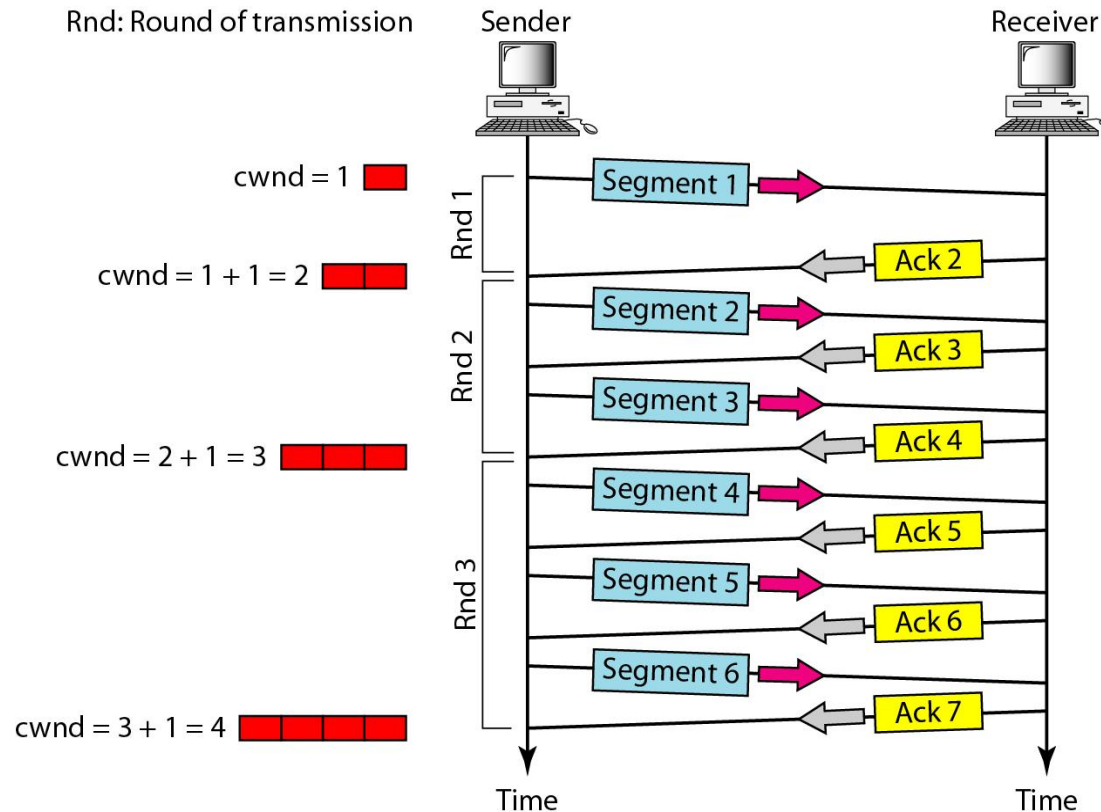
- For simplicity, *number of segments* is used instead of number of bytes
- **Slow-Start**: initial rate is slow ($cwnd=1$)
 - ◆ $cwnd+1$ upon receipt of ACK (**Exponential growth**)
 - ◆ **Exponential growth** fills the network “pipe” in no time
- When to leave Slow Start and enter AI?
 - ◆ A Threshold is necessary: *ssthresh*
- **Additive Increase**: A more gentle adjustment algorithm-
-AIMD
 - ◆ Additive Increase, Multiplicative Decrease
 - ◆ **Gentle increase, Drastic decrease**
 - On receipt of ACK, $cwnd += 1/cwnd$
 - On detection of packet loss (time out), $cwnd = 1$

Slow Start: Exponential Increase



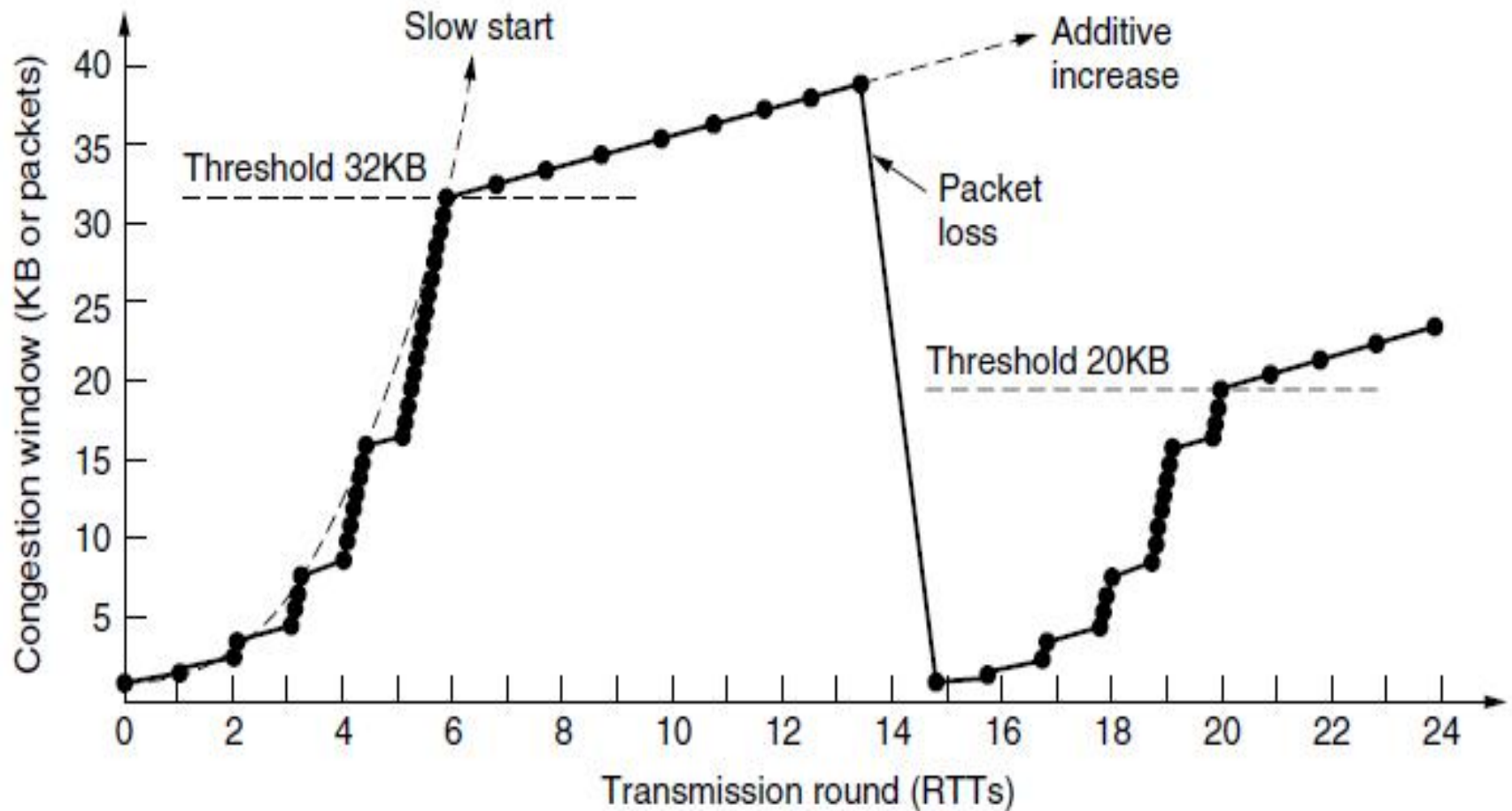
In the slow-start algorithm, the size of the congestion window increases **exponentially** until it reaches a threshold.

Congestion Avoidance: Additive Increase



In the congestion avoidance algorithm, the size of the congestion window **increases additively** until congestion is detected.

Slow Start and AIMD Example (Figure 6-46)



Fast Retransmission and Fast Recovery

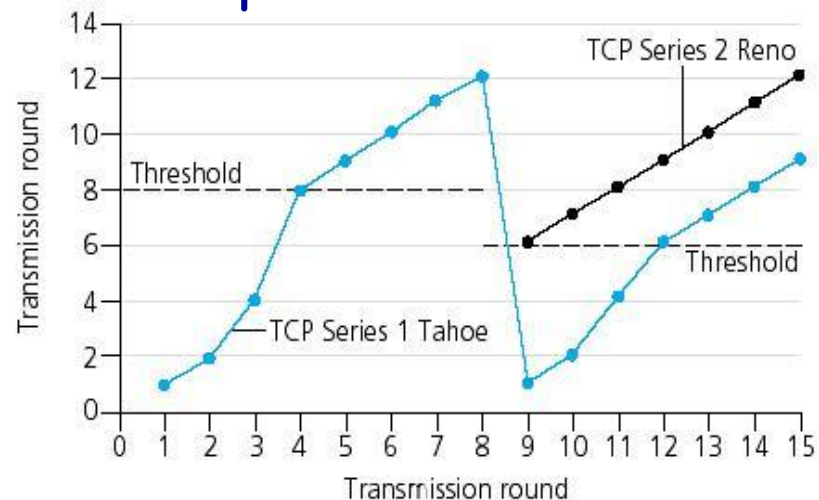
■ Fast Retransmission

- ◆ **Duplicate ACKs**: As packets beyond the lost packet arrive at the receiver, they trigger ACKs that bear the same ACK number.
- ◆ **Fast retransmission**: Three duplicate ACKs imply that a packet has been lost, the lost packet can be retransmitted right away before the RTO fires.
- ◆ **TCP Tahoe** treats 3 duplicate ACKS the same as a timeout, then a new slow start phase starts.

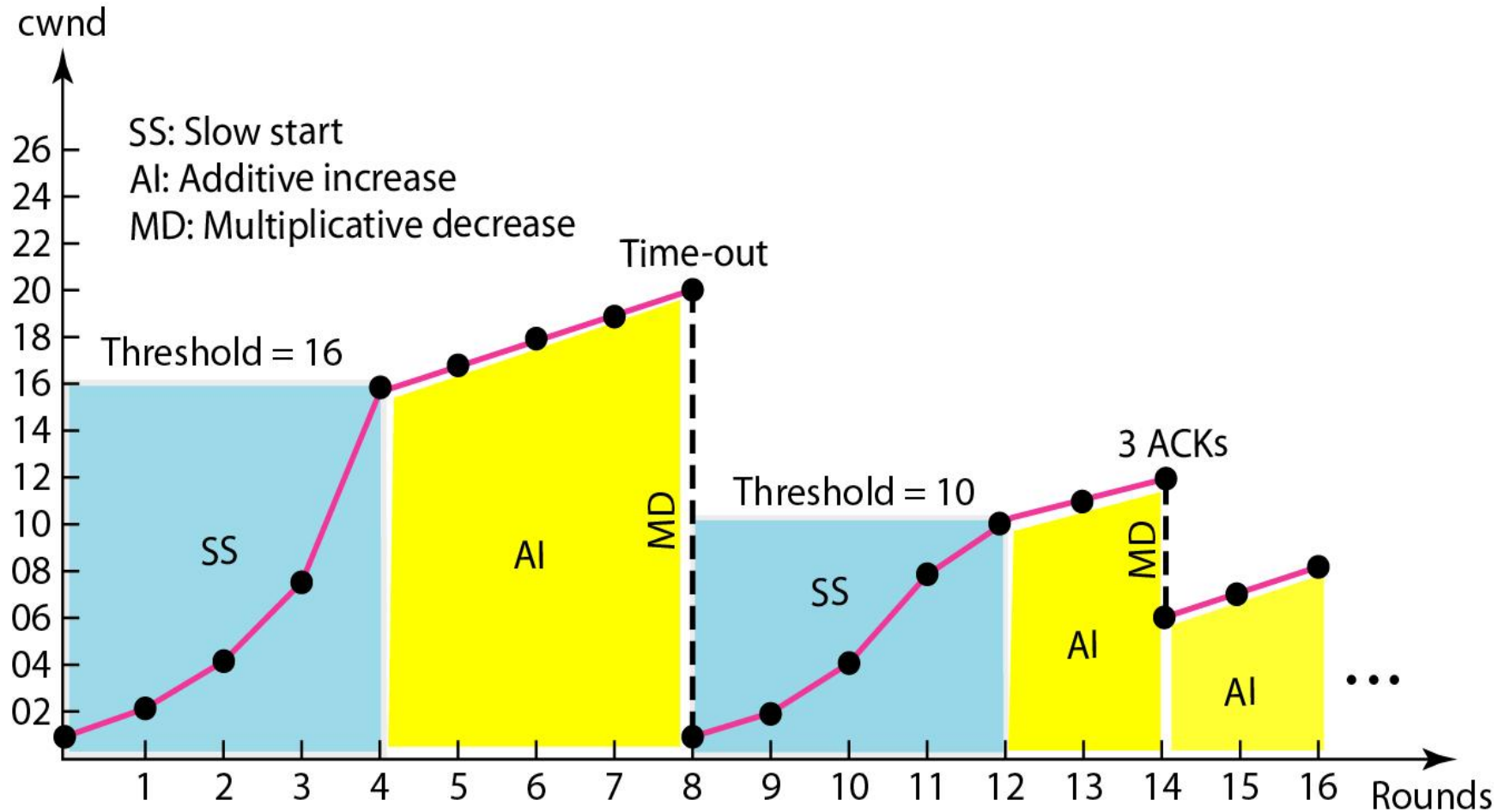
- **Fast Recovery**: For 3 duplicate ACKs, **TCP Reno** will halve the congestion window, perform a "fast recovery", and enter a new congestion avoidance phase:

Fast Recovery

- ◆ threshold = $\frac{1}{2}$ window
- ◆ cwnd = threshold



TCP Congestion Control Example



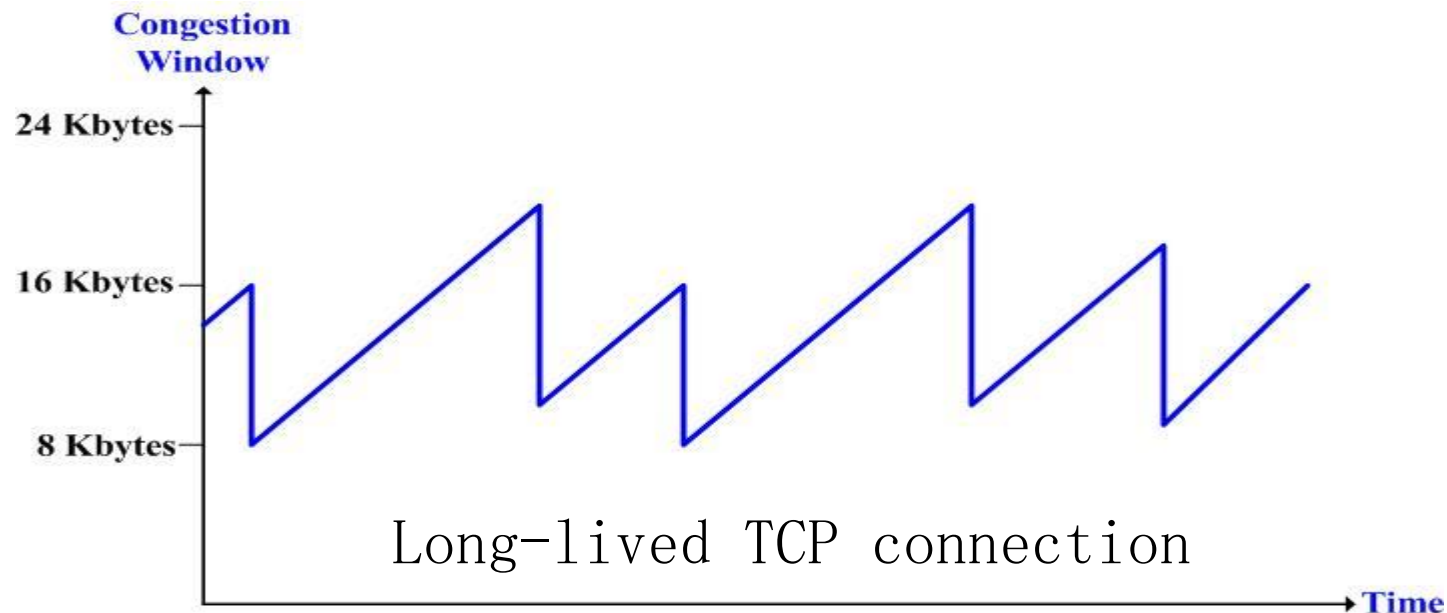
AIMD Effect

■ Addictive Increase

- ◆ Increasing Congestion Window by **1** when a full window of packets has been sent without loss of packets
- ◆ *Linear increasing* to avoid congestion

■ Multiplicative decrease

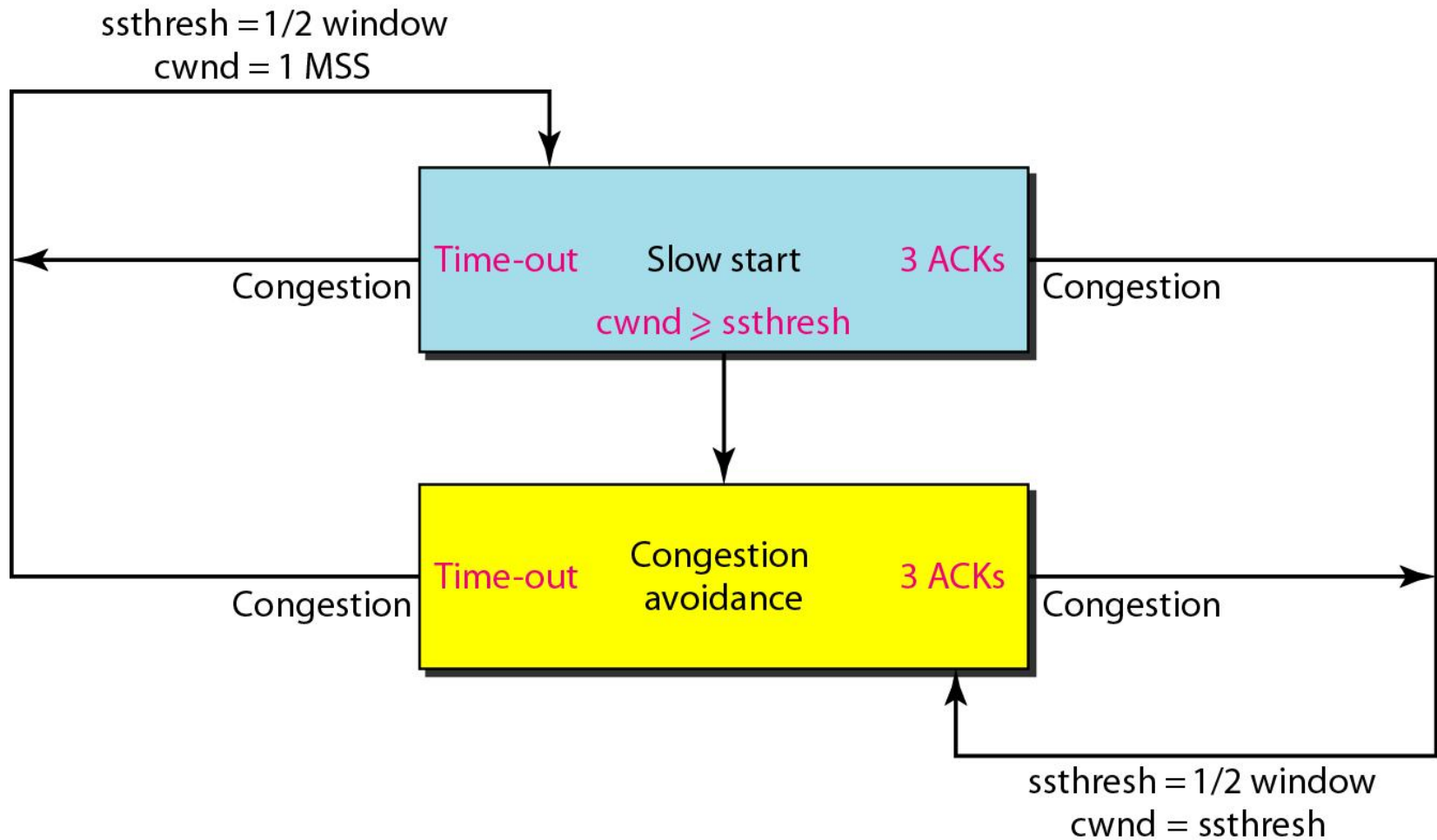
- ◆ Cutting Congestion Window in **half** after packet loss



Summary of TCP Congestion Control (1)

- When Congestion Window (Congwin) < Threshold, sender in **slow start** phase, window grows exponentially.
- When CongWin > Threshold, sender is in **congestion avoidance** phase, window grows linearly.
- When a **3-duplicate-ACK** occurs, sender is in **congestion avoidance** phase
 - ◆ $\text{Threshold} = \text{CongWin} / 2$
 - ◆ $\text{CongWin set} = \text{Threshold}$
- When **timeout** occurs, sender in **slow start** phase
 - ◆ $\text{Threshold} = \text{CongWin} / 2$
 - ◆ $\text{CongWin} = 1 \text{ MSS}$

Summary of TCP Congestion Control(2)

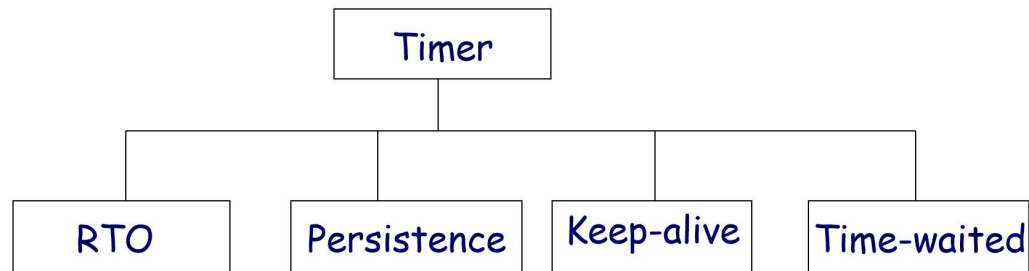


Review of TCP(1)

- 提供面向连接的可靠字节流服务
- 采用三次握手建立连接(SYN, SYN+ACK, ACK)
- 采用三次握手或四步释放关闭连接(FIN, FIN+ACK, ACK)
- 差错控制: ARQ机制、Fast Retransmission
- 流量控制: 动态滑动窗口
 - ◆ Window size
 - ◆ Nagle: 发送方使用, 一次尽可能发送较多的数据
 - ◆ Clark: 接收方使用, 在有较大空闲缓存时才发送窗口更新通知, 解决傻瓜窗口问题

■ 定时器

■ 超时重传时间RTO



◆ Jacobson算法, SRTT

$$\text{RTT 估值} = \alpha \times \text{RTT旧估值} + (1 - \alpha) \times \text{RTT 采样值}$$

Review of TCP(2)

■ 拥塞控制

- ◆ 开始时采用慢启动slow start，拥塞窗口cwnd初值为1MSS，每轮发送成功之后加倍
- ◆ 达到Threshold之后，采用拥塞避免，拥塞窗口按照AI递增，每轮发送成功之后增加一个MSS
- ◆ 一旦出现定时器超时，则采用MD， $\text{Threshold} = 1/2 \text{cwnd}$ (新阈值为超时发生时的拥塞窗口值的一半), $\text{cwnd} = 1$, 开始新的慢启动
- ◆ 当收到3个重复的ACK时，采用快速重传，从当前拥塞窗口值的一半开始新的拥塞避免，按照AI递增 ($\text{Threshold} = 1/2 \text{cwnd}$, $\text{cwnd} = \text{Threshold}$)

The transport features provided by TCP are:

- o connection-oriented transport with feature negotiation and application-to-port mapping (implemented using SYN segments and the TCP Option field to negotiate features),
- o unicast transport (though anycast TCP is implemented, at risk of instability due to rerouting),
- o port multiplexing,
- o unidirectional or bidirectional communication,
- o stream-oriented delivery in a single stream,
- o fully reliable delivery (implemented using ACKs sent from the receiver to confirm delivery),
- o error detection (implemented using a segment checksum to verify delivery to the correct endpoint and integrity of the data and options),
- o segmentation,
- o data bundling (optional; uses Nagle's algorithm to coalesce data sent within the same RTT into full-sized segments),
- o flow control (implemented using a window-based mechanism where the receiver advertises the window that it is willing to buffer), and
- o congestion control (usually implemented using a window-based mechanism and four algorithms for different phases of the transmission: slow start, congestion avoidance, fast retransmit, and fast recovery [RFC5681]).

Key Points(1)

■ Functions and Services of Transport Layer

- ◆ Process-to-process delivery
- ◆ Reliable delivery, congestion control

■ Design issues

- ◆ Data link layer vs. transport layer
- ◆ Addressing: port number
- ◆ Connection management: 3-way handshaking
- ◆ Congestion control: min-max fairness, AIMD

■ UDP

- ◆ Fast and efficient, best-effort service
- ◆ Multiplexing over IP

■ TCP

- ◆ Connection setup: 3-way handshaking
- ◆ Connection release: 3-way handshaking or 4-step
- ◆ Error control: checksum, ARQ, Fast retransmission

Key Points(2): TCP

- Flow control: byte-oriented sliding window
 - ◆ $\text{sending window} = \min(\text{recv-window}, \text{congestion window})$
 - ◆ Nagles algorithm: solution at sender
 - ◆ Silly Window Syndrome: Clark's solution, solution at receiver
- Estimation of RTT and RTO
- Congestion control
 - ◆ Slow Start: congestion window exponentially increases until threshold reached
 - ◆ Congestion Avoidance(AI): congestion window linearly increases
 - ◆ MD: starting a new Slow Start when ACK times out
 - ◆ Fast retransmit: starting a new Congestion Avoidance when 3 duplicated ACKs received

Terms

缩写	英文全称	中文
ACK	Acknowledgement	确认位
	addressing	编址/寻址
AIMD	Additive Increase Multiplicative Decrease	加法增乘法减
API	Application Program Interface	应用编程接口
ARQ	Automatic Retransmission reQuest	自动请求重发
	checksum	校验和/检查和
	Client-Server model	客户/服务器模型
	Congestion Avoidance	拥塞避免
	congestion control	拥塞控制
	congestion window	拥塞窗口
	connectionless service	无连接服务
	connection-oriented service	面向连接的服务

缩写	英文全称	中文
CWR	Congestion Window Reduced	拥塞窗口缩减
	Cumulative ACK	累积ACK
	datagram	数据报
ECE	Explicit Congestion Notification-ECHO	显示拥塞通知-回声
ECN	Explicit Congestion Notification	显式拥塞通知
	Fast Recovery	快速恢复
	Fast Retransmission	快速重传
	flow control	流量控制
FIN	Finish	终止(连接)位
	Internet	因特网

缩写	英文全称	中文
	Max-Min Fairness	最大最小公平性
	Maximum Segment Size	最大报文段长度
MTU	Maximum Transmission Unit	最大传输单元
	Multiplexing/Demultiplexing	复用/解复用(分用)
	packet	分组/包
	process	进程
	Pseudoheader	伪报头
PSH	push	推送(数据)位
QoS	Quality of Service	服务质量
RST	Reset	(连接)重启位

缩写	英文全称	中文
RTT	Round-Trip Time	环回时间
RTO	Retransmission Time-Out	重传超时时间
	TCP segment	TCP报文段
SACK	Selective ACK	选择确认
SAP	Service Access Point	服务访问点
SCTP	Stream Control Transmission Protocol	流控制传输协议
	Service Primitives	服务原语
	Silly Window Syndrome	傻瓜窗口症状
	sliding window	滑动窗口
	Slow Start	慢启动
	Smoothed Round-Trip Time	平滑的环回时间
SYN	Synchronize	(序号)同步位
TCP	Transmission Control Protocol	传输控制协议
UDP	User Datagram Protocol	用户数据报协议
URG	Urgent	紧急位
WAN	Wide Area Network	广域网

Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
 - ◆ Lecture notes of book "Computer Networks, a Top-down Approach", J.Kurose and W.Ross, 3rd Edition, which are marked with [Kurose]
 - ◆ "Data communications and networking", B.A.Forouzan, which are marked with [Forouzan]
 - ◆ "计算机网络", 谢希仁, 第五版, 电子工业出版社, 2008年, which are marked with [谢]

Berkeley Sockets (套接字)

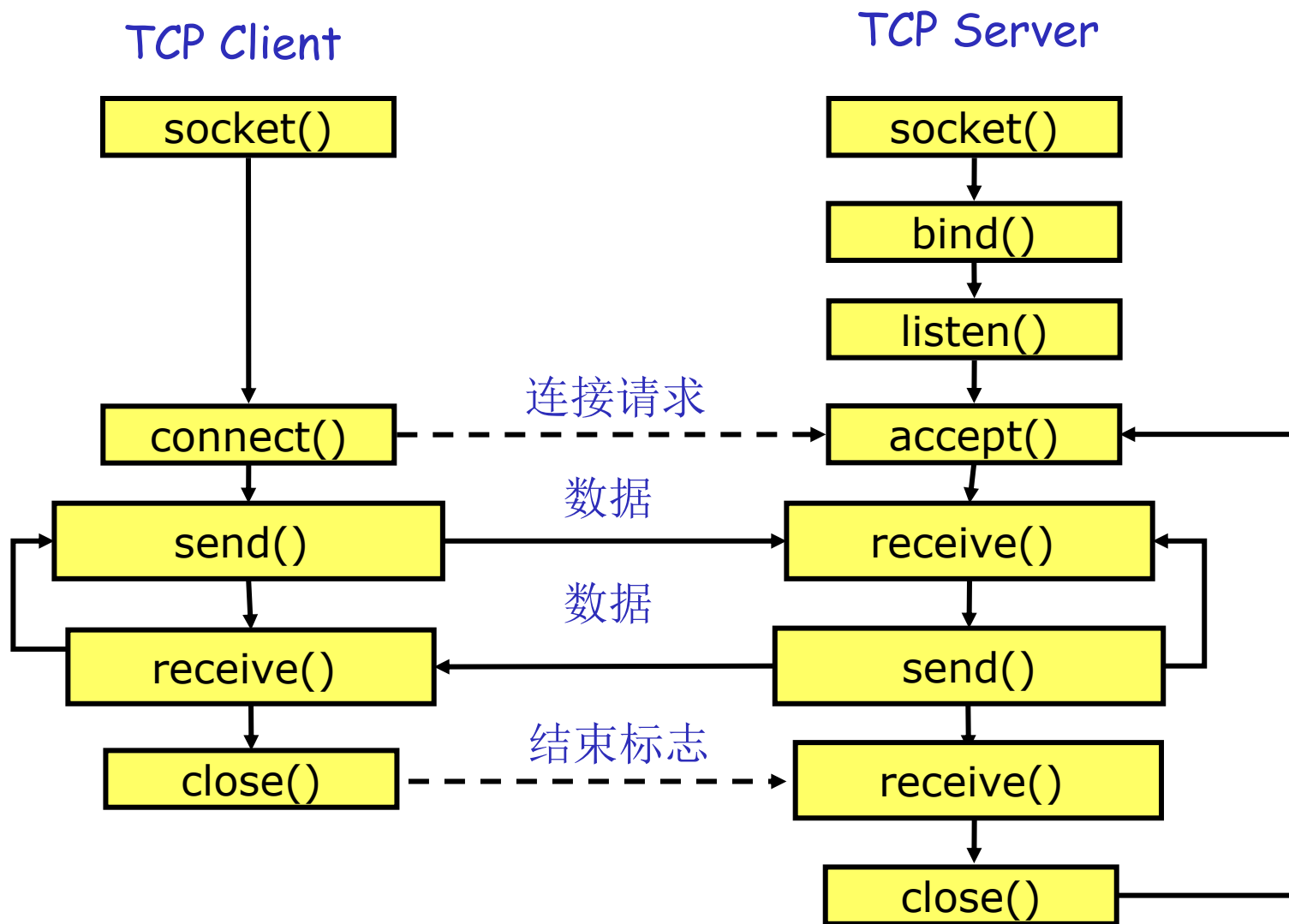
■ Widely used API in Internet

IP address + Port number(端口号)

Primitive	Meaning
SOCKET	Create a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Figure 6-5. The socket primitives for TCP.

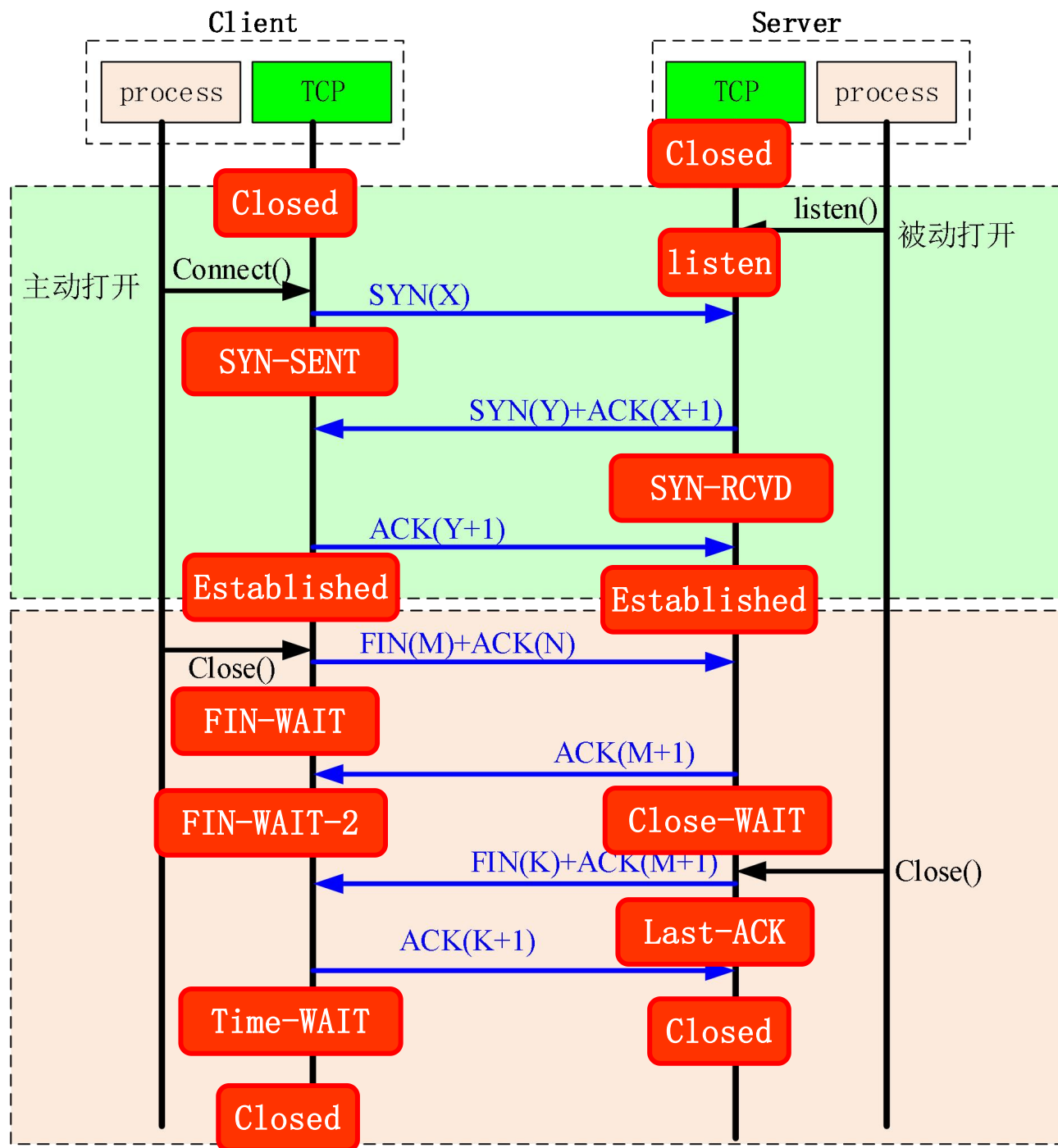
基于TCP的Socket程序流程



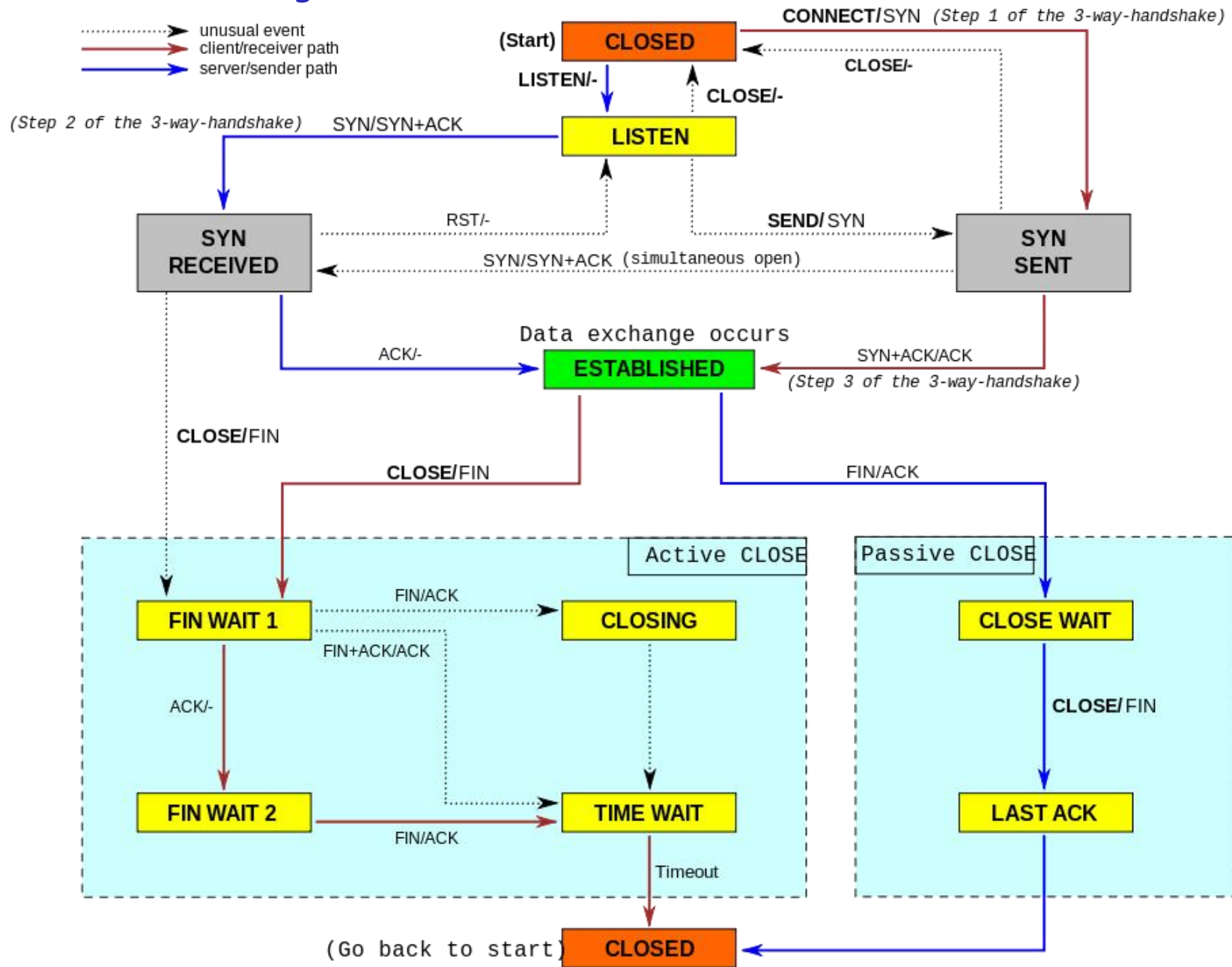
TCP Connection Management Modeling

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

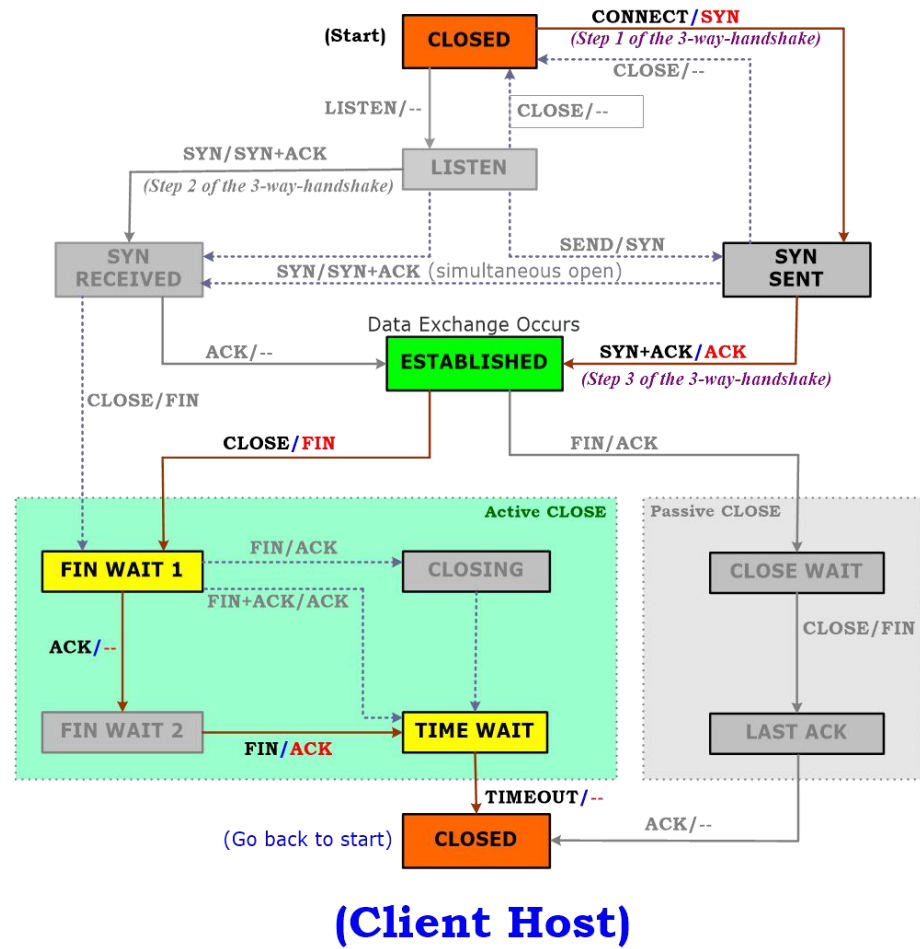
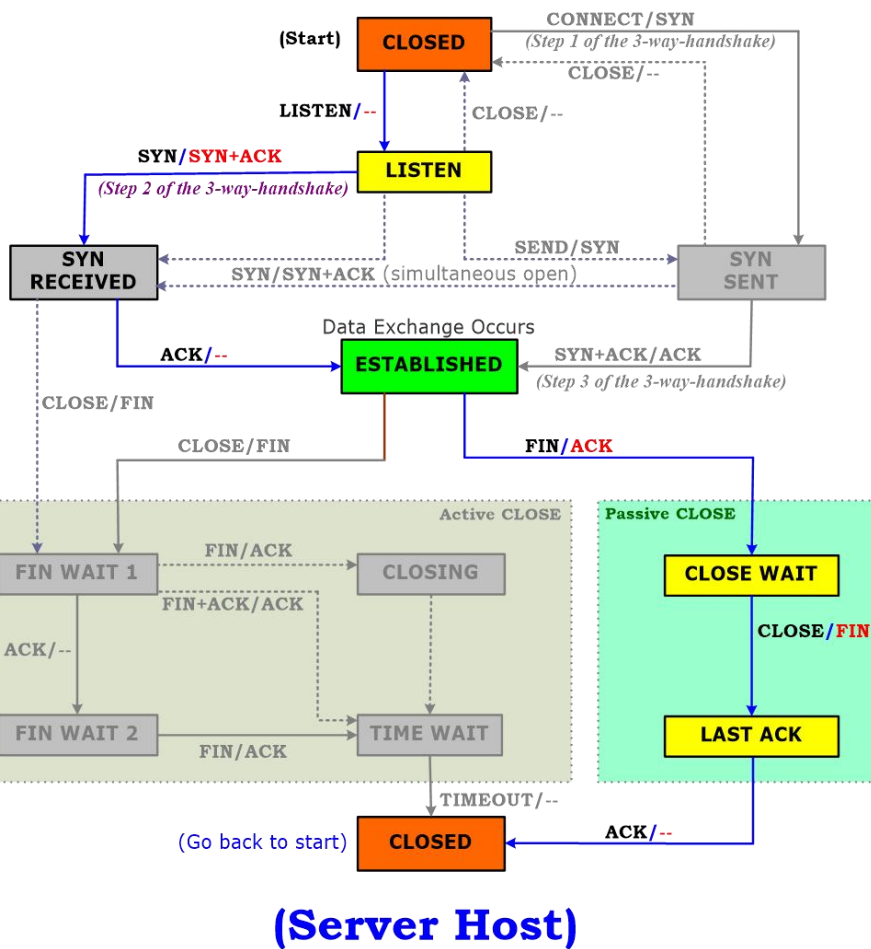
States used in the TCP connection management finite state machine



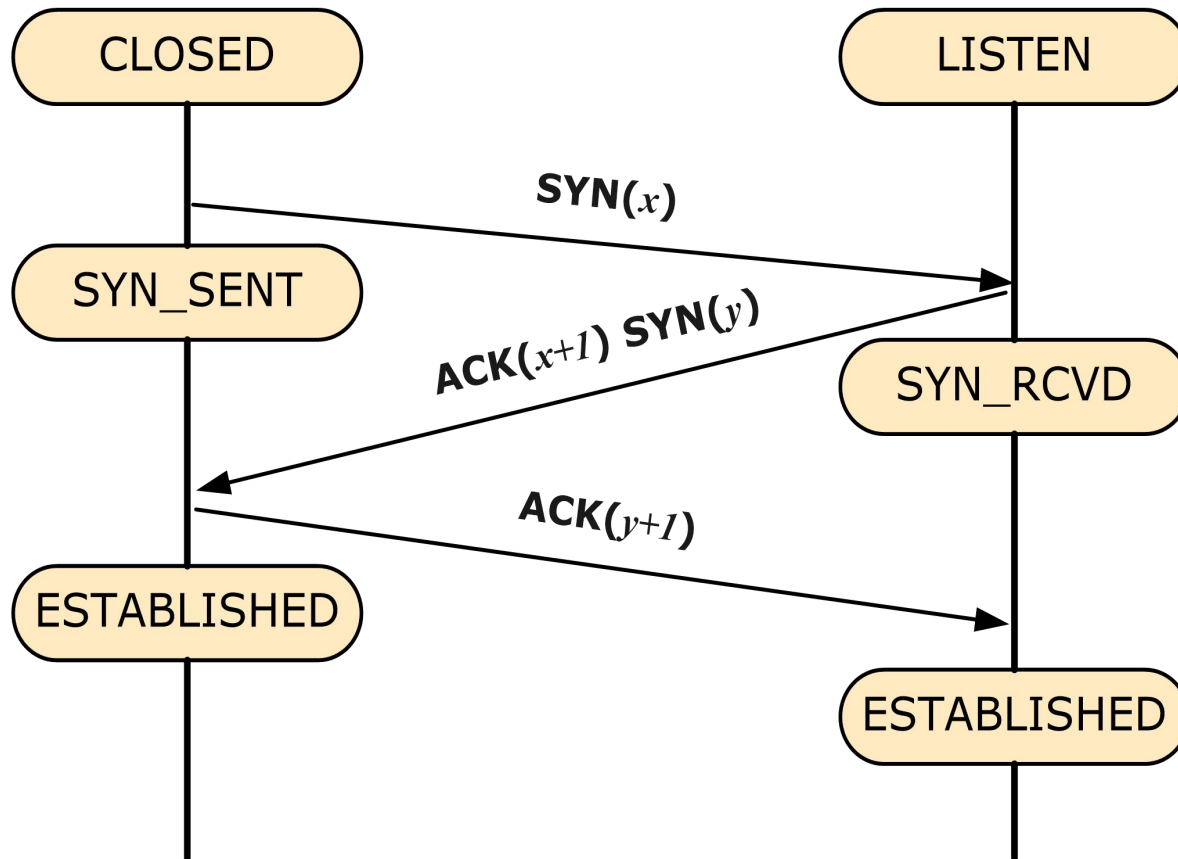
TCP connection management finite state machine.



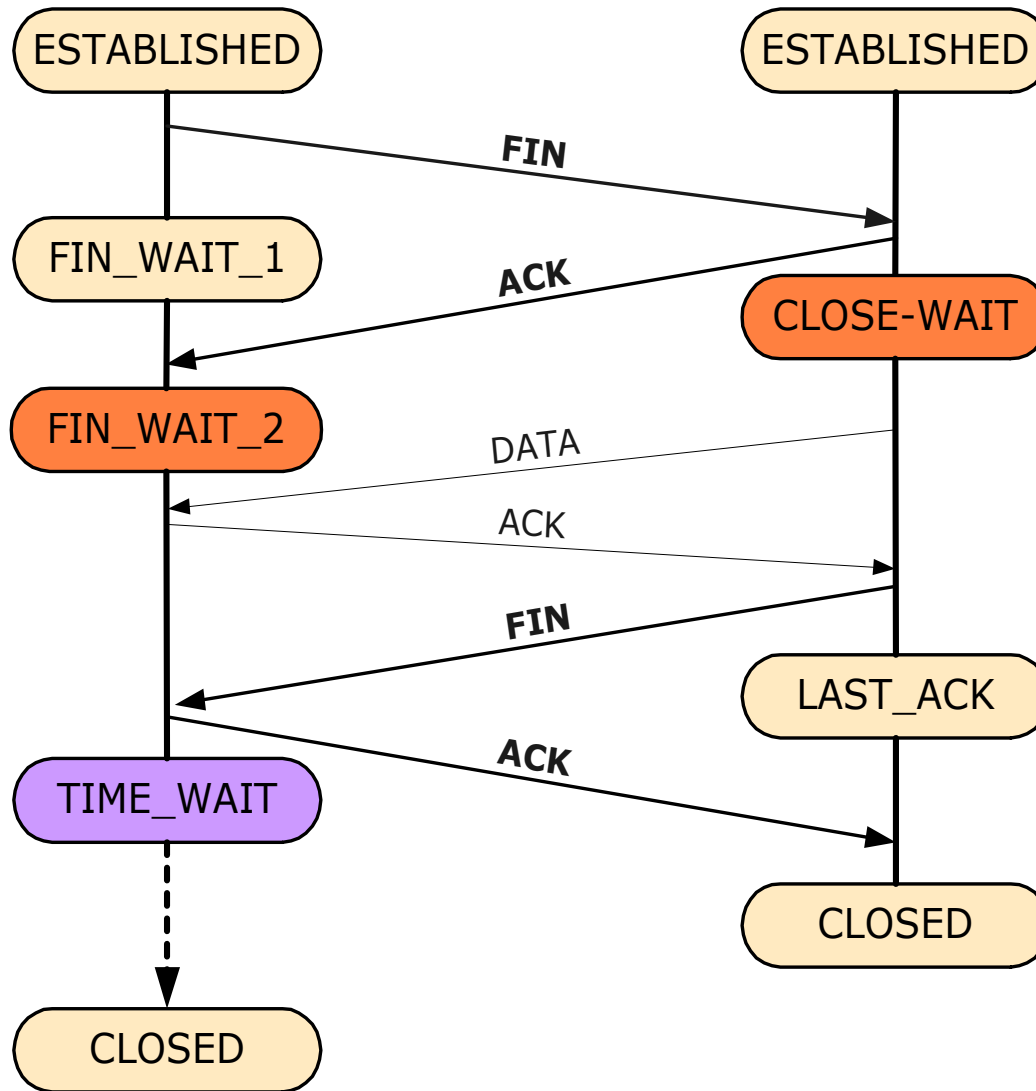
TCP Finite State Machine



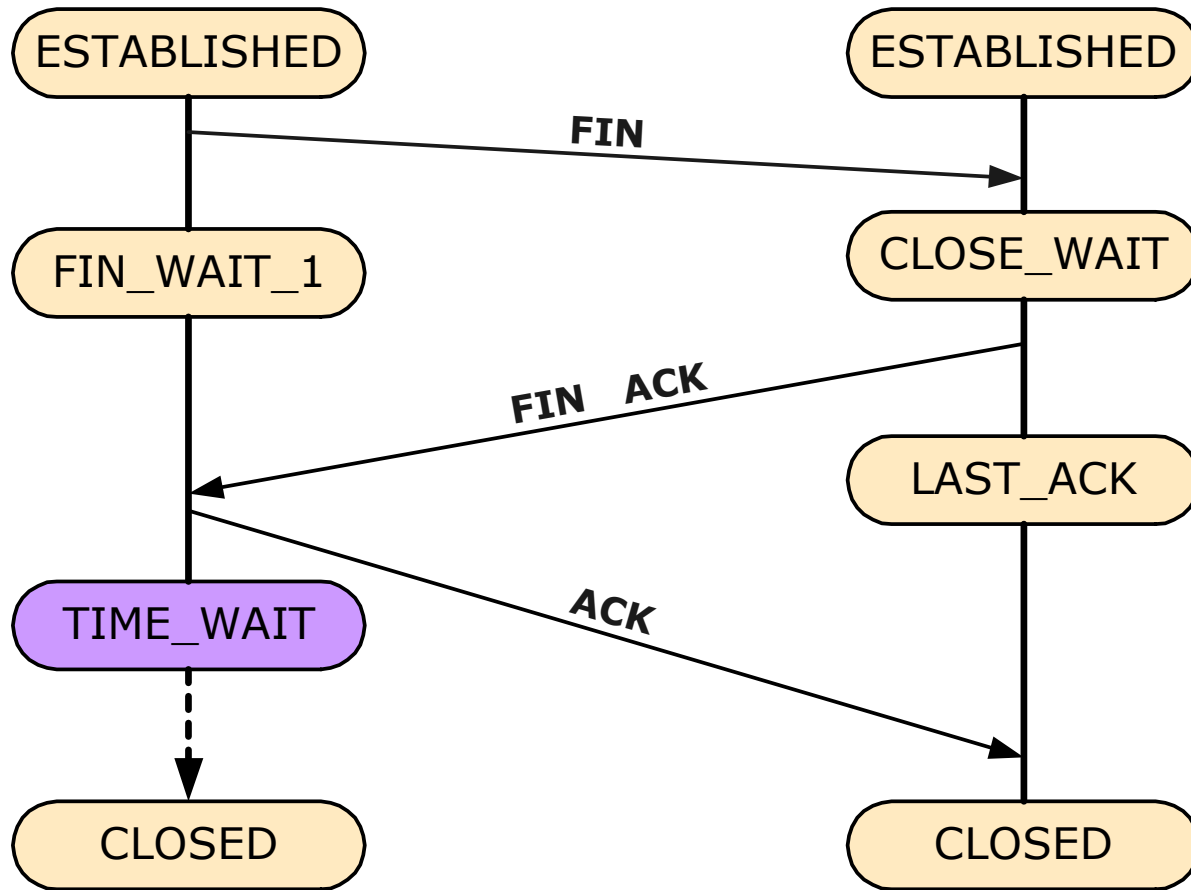
TCP Connection Establishment



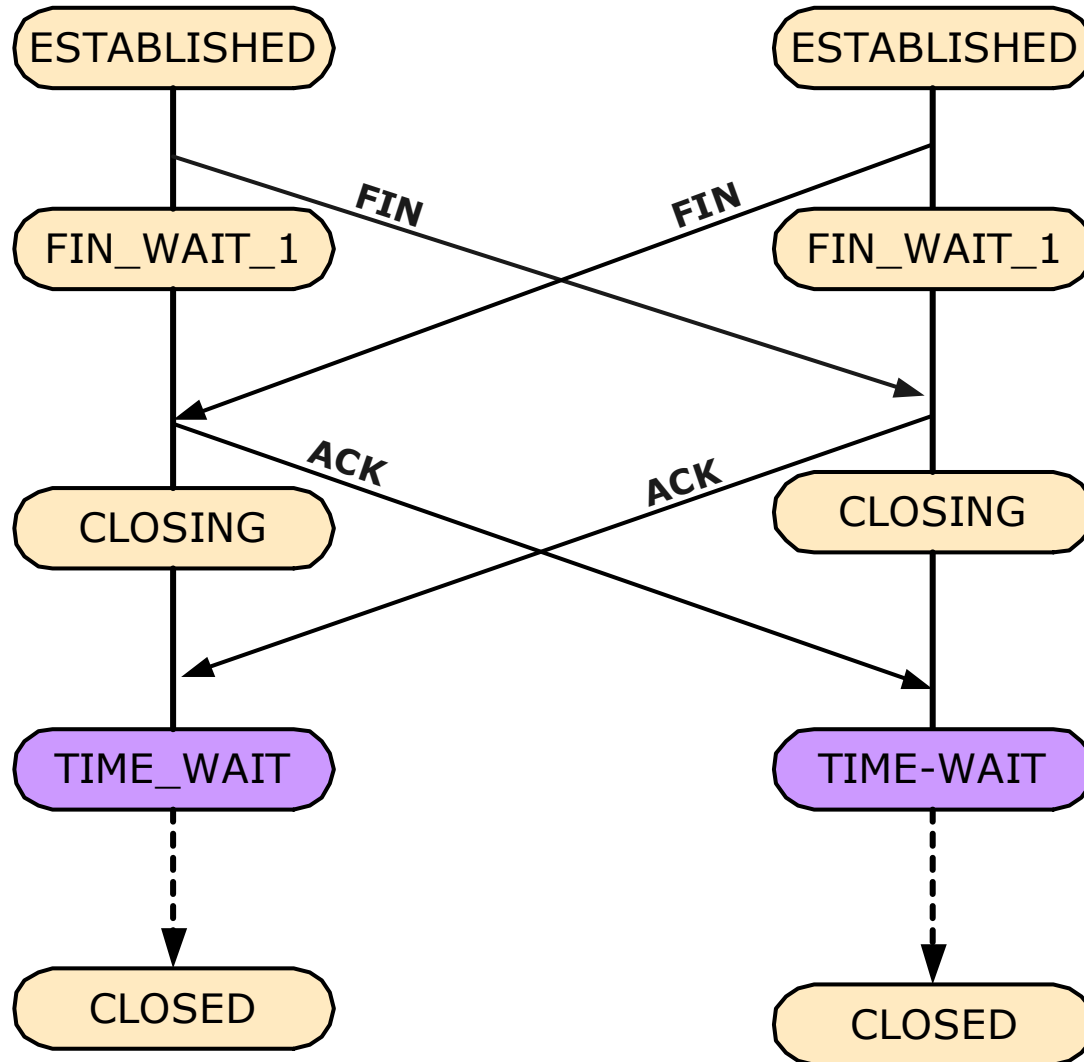
TCP Connection Release (1)



TCP Connection Release(2)

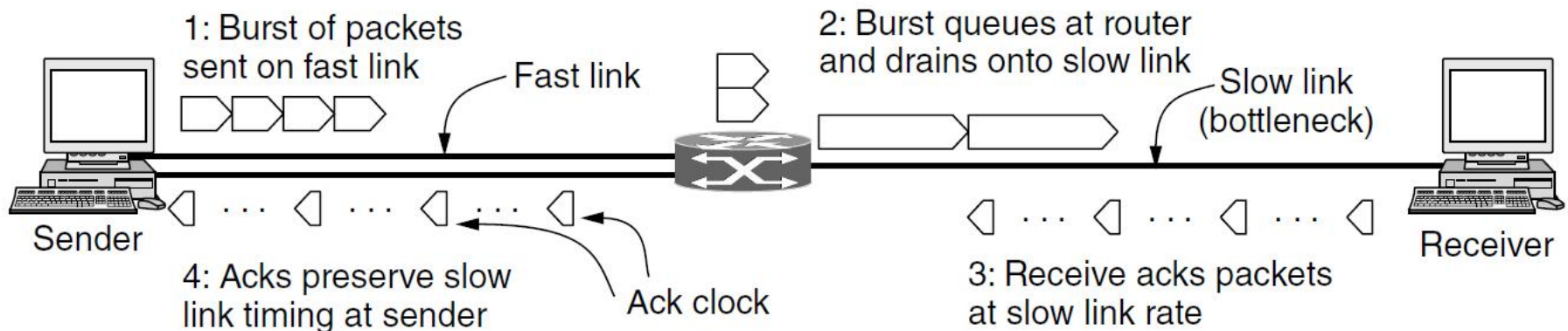


TCP Connection Release (3)

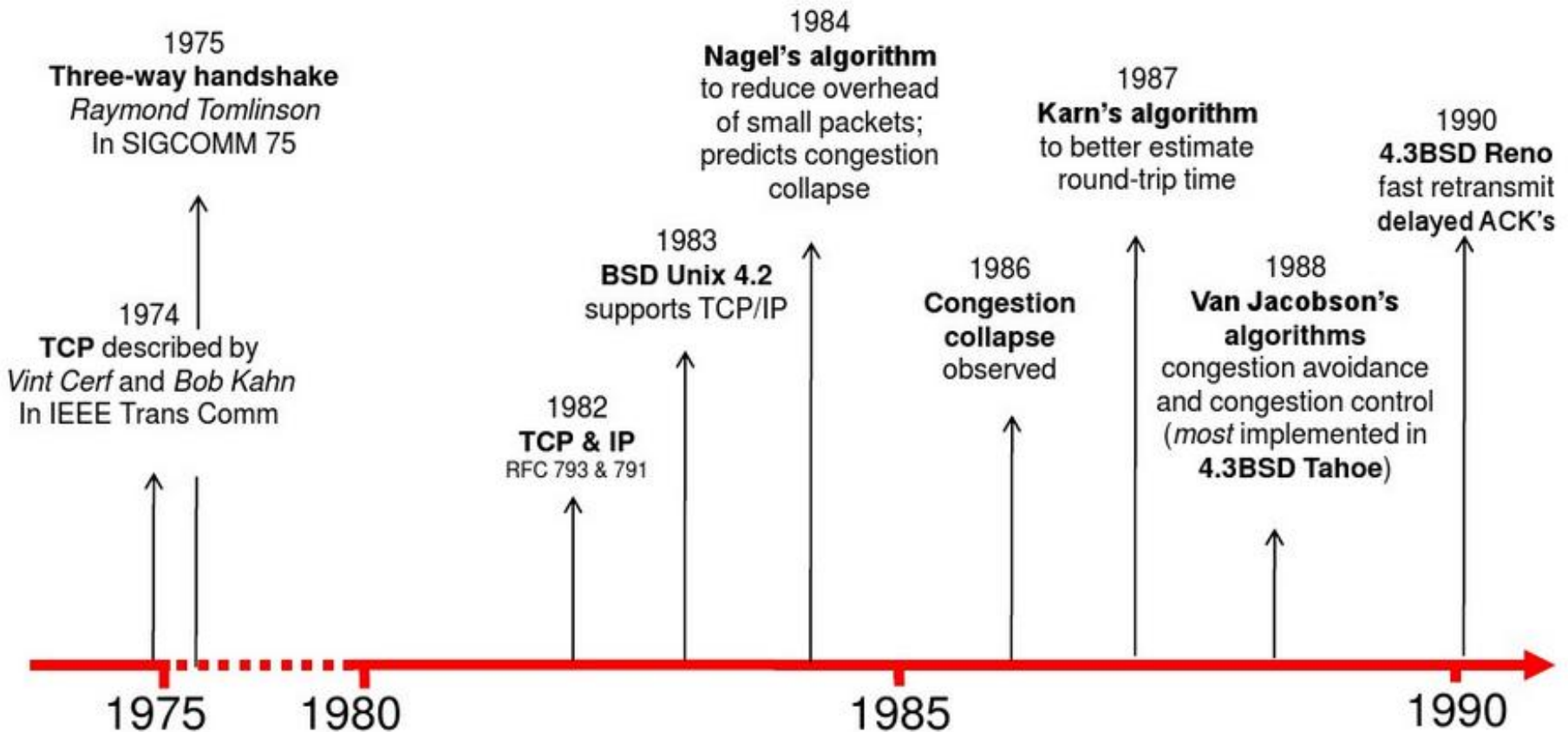


ACK Clock

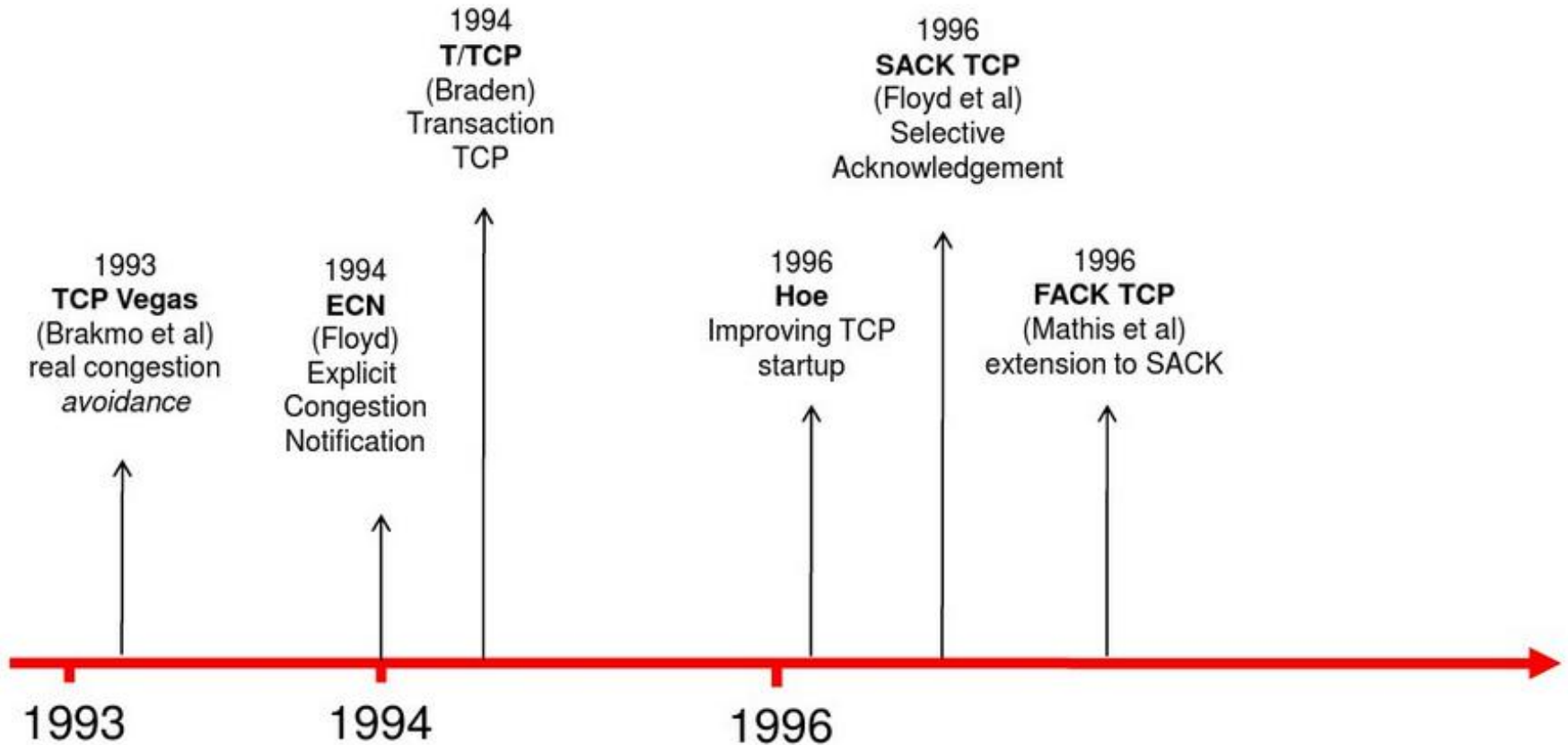
- The times for the ACKs **reflect** the times at which the packets arrived at the receiver after crossing the slow link
- As the ACKs travel over the net and back to the sender, routers **preserve** this timing
- If TCP injects packets **at the rate of ACKs returned**, packets will be sent as fast as the slow link permits, they will not queue up and congest any router along the path
- By using an **ACK clock**, TCP smoothes out traffic and avoids unnecessary queues at routers



TCP Evolution(1)



TCP Evolution(2)



编号	IP 包的全部内容																							
1	45 00	00 30	82 fc	40 00	80 06	f5 a5	c0 a8 00 15	c0 a8 00 c0	06 64	31 ba	22 68 b9 90	00 00 00 00	70 02	ff ff	ec e2 00 00	02 04	05 b4	01 01	04 02					
2	45 00	00 2f	00 07	40 00	40 01	24 42	c0 a8 00 65	da 20 7b 57	08 00	69 5a	36 6f 00 07	73 48 5b 49	37 5c 04 00	08 09 0a 0b	0c 0d	0e 0f	10 11	12						
3	45 00	00 30	00 00	40 00	40 06	b8 a2	c0 a8 00 c0	c0 a8 00 15	31 ba	06 64	5b 9f f7 1c	22 68 b9 91	70 12	20 00	83 45 00 00	02 04	05 b4	01 01	04 02					
4	45 00	00 28	82 fd	40 00	80 06	f5 ac	c0 a8 00 15	c0 a8 00 c0	06 64	31 ba	22 68 b9 91	5b 9f f7 1d	50 10	ff ff	c6 d9 00 00									
5	45 00	00 38	82 fe	40 00	80 06	f5 9b	c0 a8 00 15	c0 a8 00 c0	06 64	31 ba	22 68 b9 91	5b 9f f7 1d	50 18	ff ff	bc b7 00 00	f8 9f	e3 e3	2c 12	c2 89	24 34	6a 13	55 b7	65 59	
6	45 00	00 28	3f 28	40 00	40 06	79 82	c0 a8 00 c0	c0 a8 00 15	31 ba	06 64	5b 9f f7 1d	22 68 b9 a1	50 10	20 00	af f9 00 00									
7	45 00	00 38	83 0b	40 00	80 06	f5 8e	c0 a8 00 15	c0 a8 00 c0	06 64	31 ba	22 68 b9 a1	5b 9f f7 1d	50 18	ff ff	bc a7 00 00	f8 9f	e3 e3	2c 12	c2 89	24 34	6a 13	55 b7	65 59	
8	45 00	00 48	83 3e	00 00	80 06	35 4c	c0 a8 00 15	c0 a8 00 c0	06 64	31 ba	22 68 b9 a1	5b 9f f7 1d	50 18	ff ff	b2 8d 00 00	f8 9f	e3 e3	2c 12	c2 89	24 34	6a 13	55 b7	65 59	dd 47 2c 3a
			b1 0c	9a f1	75 1b	4f 75	62 df	03 19																